

PPSE Firmware

v0.0.1-24-g2f8d409

Generated by Doxygen 1.9.8

1 RP2040 GPS & Environment Logger	1
1.1 Core Features	1
1.1.1 GPS & Telemetry Engine	1
1.1.2 Hybrid Storage System	1
1.1.3 UI & UX Framework	1
1.2 Risorse e Documentazione	2
2 Topic Index	3
2.1 Topics	3
3 Data Structure Index	5
3.1 Data Structures	5
4 File Index	7
4.1 File List	7
5 Topic Documentation	9
5.1 Gestione Errori	9
5.1.1 Detailed Description	9
5.1.2 Enumeration Type Documentation	9
5.1.2.1 error_category_t	9
5.1.2.2 error_code_t	10
5.2 Protocollo Dati	10
5.2.1 Detailed Description	11
5.2.2 Enumeration Type Documentation	11
5.2.2.1 GpsStatus_t	11
5.2.3 Function Documentation	11
5.2.3.1 __attribute__((__aligned__))	11
5.2.4 Variable Documentation	11
5.2.4.1 SystemDataPacket	11
5.3 Gestione Memoria	11
5.3.1 Detailed Description	12
5.3.2 Function Documentation	12
5.3.2.1 debug_dump_gps_log()	12
5.3.2.2 storage_format_all()	12
5.3.2.3 storage_init()	12
5.3.2.4 storage_is_busy_by_usb()	13
5.3.2.5 storage_load_params()	13
5.3.2.6 storage_load_weather()	13
5.3.2.7 storage_log_append()	13
5.3.2.8 storage_log_gps_fix()	13
5.3.2.9 storage_read_last_gps()	14
5.3.2.10 storage_read_line_with_tag()	14
5.3.2.11 storage_save_params()	14

5.3.2.12 storage_save_weather()	15
5.4 Gestione Sistema	15
5.4.1 Detailed Description	15
5.4.2 Function Documentation	15
5.4.2.1 storage_load_weather()	15
5.4.2.2 storage_save_weather()	15
5.4.2.3 sys_is_core1_ready()	15
5.4.2.4 sys_manager_clear_error()	16
5.4.2.5 sys_manager_handle_serial()	16
5.4.2.6 sys_manager_init()	16
5.4.2.7 sys_manager_receive_data()	16
5.4.2.8 sys_manager_report_error()	16
5.4.2.9 sys_manager_send_data()	16
5.4.2.10 sys_manager_update_gps()	17
5.4.2.11 sys_manager_update_weather()	17
5.4.2.12 sys_set_core1_ready()	17
5.5 Telemetria	18
5.5.1 Detailed Description	18
5.5.2 Function Documentation	18
5.5.2.1 telemetry_get_frame()	18
5.5.2.2 telemetry_init()	18
5.5.2.3 telemetry_is_healthy()	18
5.5.2.4 telemetry_update()	19
5.6 Mappatura Pin Hardware	19
5.6.1 Detailed Description	20
5.6.2 Macro Definition Documentation	20
5.6.2.1 ACC_INT1_PIN	20
5.6.2.2 ACC_INT2_PIN	20
5.6.2.3 ANALOG_BATT_PIN	20
5.6.2.4 BTN_BACK_PIN	20
5.6.2.5 BTN_DOWN_PIN	20
5.6.2.6 BTN_LEFT_PIN	20
5.6.2.7 BTN_OK_PIN	20
5.6.2.8 BTN_RIGHT_PIN	20
5.6.2.9 BTN_UP_PIN	20
5.6.2.10 BUCK_5V_EN_PIN	21
5.6.2.11 BUZZER_PIN	21
5.6.2.12 ESP_EN_PIN	21
5.6.2.13 ESP_RX_PIN	21
5.6.2.14 ESP_TX_PIN	21
5.6.2.15 GPS_RX_PIN	21
5.6.2.16 GPS_TX_PIN	21

5.6.2.17 GPSBAUD	21
5.6.2.18 I2C0_SCL_PIN	21
5.6.2.19 I2C0_SDA_PIN	21
5.6.2.20 LED_ALIVE_PIN	22
5.6.2.21 NUM_LEDS	22
5.6.2.22 OUTPUT_USB_EN_PIN	22
5.6.2.23 RGB_LED_PIN	22
5.6.2.24 SAM_EN_PIN	22
5.6.2.25 SAM_PPS_PIN	22
5.6.2.26 SAM_RST_PIN	22
5.6.2.27 SERVO_1_PIN	22
5.6.2.28 SERVO_2_PIN	22
5.6.2.29 SERVO_3_PIN	22
5.6.2.30 TEMP_SENS_PIN	23
5.7 Driver Display OLED	23
5.7.1 Detailed Description	23
5.7.2 Macro Definition Documentation	23
5.7.2.1 SCREEN_ADDRESS	23
5.7.2.2 SCREEN_HEIGHT	23
5.7.2.3 SCREEN_WIDTH	23
5.7.3 Function Documentation	24
5.7.3.1 display_clear()	24
5.7.3.2 display_draw_pixel()	24
5.7.3.3 display_hw_init()	24
5.7.3.4 display_is_alive()	24
5.7.3.5 display_set_brightness()	24
5.7.3.6 display_set_power()	25
5.7.3.7 display_show()	25
5.7.3.8 get_display_driver()	25
5.8 Gestione Input	25
5.8.1 Detailed Description	26
5.8.2 Typedef Documentation	26
5.8.2.1 button_t	26
5.8.3 Enumeration Type Documentation	26
5.8.3.1 button_state_t	26
5.8.3.2 ButtonId	26
5.8.4 Function Documentation	27
5.8.4.1 inputs_get_last_press()	27
5.8.4.2 inputs_init()	27
5.8.4.3 inputs_is_pressed()	27
5.8.4.4 inputs_update()	27
5.9 Driver Periferiche	27

5.9.1 Detailed Description	28
5.9.2 Enumeration Type Documentation	28
5.9.2.1 FeedbackType	28
5.9.2.2 LedAnimState	28
5.9.3 Function Documentation	29
5.9.3.1 peripherals_auto_feedback()	29
5.9.3.2 peripherals_beep()	29
5.9.3.3 peripherals_get_temperature()	29
5.9.3.4 peripherals_init()	29
5.9.3.5 peripherals_init_leds()	30
5.9.3.6 peripherals_play_click()	30
5.9.3.7 peripherals_set_buzzer()	30
5.9.3.8 peripherals_set_led()	30
5.9.3.9 peripherals_set_leds()	30
5.9.3.10 peripherals_trigger_feedback()	30
5.9.3.11 peripherals_update_led_fx()	30
5.10 Gestione Sensori	31
5.10.1 Detailed Description	31
5.10.2 Function Documentation	31
5.10.2.1 sensors_accel_calibrate()	31
5.10.2.2 sensors_accel_get_data()	31
5.10.2.3 sensors_i2c_init()	32
5.10.2.4 sensors_i2c_is_healthy()	32
5.10.2.5 sensors_read_battery_v()	32
5.10.2.6 sensors_read_temperature_c()	32
5.11 Master Control	33
5.11.1 Detailed Description	33
5.11.2 Function Documentation	33
5.11.2.1 core0_loop()	33
5.11.2.2 core0_setup()	33
5.12 RealTimeWorker	33
5.12.1 Detailed Description	33
5.12.2 Function Documentation	33
5.12.2.1 core1_loop()	33
5.12.2.2 core1_setup()	34
5.13 Gestione Task	34
5.13.1 Detailed Description	34
5.13.2 Variable Documentation	34
5.13.2.1 task_health	34
5.13.2.2 task_input	34
5.13.2.3 task_serial	34
5.13.2.4 task_ui	34

5.14 Animazioni Avvio	35
5.14.1 Detailed Description	35
5.14.2 Function Documentation	35
5.14.2.1 run_hacker_boot_sequence()	35
5.15 Gestione UI	35
5.15.1 Detailed Description	35
5.15.2 Enumeration Type Documentation	35
5.15.2.1 view_id_t	35
5.15.3 Function Documentation	36
5.15.3.1 ui_manager_dispatch_input()	36
5.15.3.2 ui_manager_get_current_view()	36
5.15.3.3 ui_manager_init()	36
5.15.3.4 ui_manager_navigate_to()	36
5.15.3.5 ui_manager_update()	37
5.16 Definizioni Viste	37
5.16.1 Detailed Description	37
5.16.2 Variable Documentation	37
5.16.2.1 view_gps	37
5.16.2.2 view_home	37
5.16.2.3 view_info	37
5.16.2.4 view_meteo	37
5.16.2.5 view_settings	38
5.17 Gestore Task	38
5.17.1 Detailed Description	38
5.17.2 Typedef Documentation	38
5.17.2.1 TaskCallback	38
5.17.3 Function Documentation	38
5.17.3.1 run_task()	38
6 Data Structure Documentation	39
6.1 error_report_t Struct Reference	39
6.1.1 Detailed Description	39
6.1.2 Field Documentation	39
6.1.2.1 category	39
6.1.2.2 code	39
6.1.2.3 is_critical	39
6.1.2.4 timestamp	39
6.2 SystemConfig Struct Reference	40
6.2.1 Detailed Description	40
6.2.2 Field Documentation	40
6.2.2.1 buzzer_enabled	40
6.2.2.2 gps_refresh_rate	40

6.2.2.3 leds_enabled	40
6.2.2.4 oled_brightness	40
6.3 Task Struct Reference	40
6.3.1 Detailed Description	41
6.3.2 Field Documentation	41
6.3.2.1 callback	41
6.3.2.2 interval	41
6.3.2.3 last_run	41
6.4 Vector3D_t Struct Reference	41
6.4.1 Detailed Description	41
6.4.2 Field Documentation	41
6.4.2.1 timestamp	41
6.4.2.2 x	42
6.4.2.3 y	42
6.4.2.4 z	42
6.5 view_interface_t Struct Reference	42
6.5.1 Detailed Description	42
6.5.2 Field Documentation	42
6.5.2.1 on_enter	42
6.5.2.2 on_exit	42
6.5.2.3 on_input	43
6.5.2.4 on_update	43
6.6 WeatherDataPacket Struct Reference	43
6.6.1 Detailed Description	43
6.6.2 Field Documentation	43
6.6.2.1 city	43
6.6.2.2 humidity	44
6.6.2.3 temp_ext	44
6.6.2.4 valid	44
6.6.2.5 weather_code	44
6.6.2.6 wind_speed	44
7 File Documentation	45
7.1 firm/include/core/config.h File Reference	45
7.1.1 Detailed Description	46
7.1.2 Function Documentation	46
7.1.2.1 config_init()	46
7.1.2.2 config_save()	46
7.1.3 Variable Documentation	46
7.1.3.1 global_cfg	46
7.2 config.h	46
7.3 firm/include/core/error_codes.h File Reference	47

7.3.1 Detailed Description	48
7.4 error_codes.h	48
7.5 firm/include/core/messages.h File Reference	48
7.5.1 Detailed Description	49
7.6 messages.h	50
7.7 firm/include/core/power_manager.h File Reference	50
7.7.1 Detailed Description	52
7.7.2 Enumeration Type Documentation	52
7.7.2.1 power_mode_t	52
7.7.3 Function Documentation	52
7.7.3.1 power_get_battery_level()	52
7.7.3.2 power_get_battery_voltage()	52
7.7.3.3 power_manager_init()	52
7.7.3.4 power_manager_update()	52
7.8 power_manager.h	53
7.9 firm/include/core/storage.h File Reference	53
7.9.1 Detailed Description	55
7.10 storage.h	55
7.11 firm/include/core/system_manager.h File Reference	55
7.11.1 Detailed Description	57
7.12 system_manager.h	57
7.13 firm/include/core/telemetry.h File Reference	57
7.13.1 Detailed Description	58
7.14 telemetry.h	59
7.15 firm/include/drivers/config_pins.h File Reference	59
7.15.1 Detailed Description	60
7.16 config_pins.h	60
7.17 firm/include/drivers/display_ssd1306.h File Reference	61
7.17.1 Detailed Description	62
7.18 display_ssd1306.h	62
7.19 firm/include/drivers/inputs.h File Reference	62
7.19.1 Detailed Description	63
7.20 inputs.h	63
7.21 firm/include/drivers/peripherals.h File Reference	64
7.21.1 Detailed Description	65
7.22 peripherals.h	65
7.23 firm/include/drivers/radio_uart.h File Reference	66
7.23.1 Detailed Description	67
7.23.2 Function Documentation	67
7.23.2.1 radio_esp_send()	67
7.23.2.2 radio_gps_send()	67
7.23.2.3 radio_uart_init()	67

7.24 radio_uart.h	67
7.25 firm/include/drivers/sensors_i2c.h File Reference	68
7.25.1 Detailed Description	69
7.26 sensors_i2c.h	69
7.27 firm/include/pipeline/core0_manager.h File Reference	69
7.27.1 Detailed Description	70
7.28 core0_manager.h	70
7.29 firm/include/pipeline/core1_manager.h File Reference	70
7.29.1 Detailed Description	71
7.30 core1_manager.h	71
7.31 firm/include/pipeline/tasks.h File Reference	71
7.31.1 Detailed Description	72
7.32 tasks.h	73
7.33 firm/include/ui/boot_anim.h File Reference	73
7.33.1 Detailed Description	73
7.34 boot_anim.h	73
7.35 firm/include/ui/ui_manager.h File Reference	74
7.35.1 Detailed Description	74
7.36 ui_manager.h	75
7.37 firm/include/ui/view_definitions.h File Reference	75
7.37.1 Detailed Description	76
7.38 view_definitions.h	76
7.39 firm/include/util/scheduler.h File Reference	76
7.39.1 Detailed Description	77
7.40 scheduler.h	78
7.41 target.h	78
7.42 config.cpp	80
7.43 power_manager.cpp	80
7.44 storage.cpp	80
7.45 system_manager.cpp	82
7.46 telemetry.cpp	84
7.47 firm/src/drivers/display_ssd1306.cpp File Reference	87
7.47.1 Detailed Description	88
7.48 display_ssd1306.cpp	88
7.49 inputs.cpp	89
7.50 firm/src/drivers/peripherals.cpp File Reference	89
7.50.1 Detailed Description	90
7.50.2 Variable Documentation	90
7.50.2.1 global_cfg	90
7.51 peripherals.cpp	91
7.52 radio_uart.cpp	93
7.53 sensors_i2c.cpp	93

7.54 main.cpp	94
7.55 core0_manager.cpp	94
7.56 core1_manager.cpp	96
7.57 tasks.cpp	97
7.58 firm/src/ui/boot_anim.cpp File Reference	97
7.58.1 Detailed Description	98
7.58.2 Function Documentation	98
7.58.2.1 run_hacker_boot_sequence()	98
7.59 boot_anim.cpp	98
7.60 firm/src/ui/ui_manager.cpp File Reference	99
7.61 ui_manager.cpp	99
7.62 view_gps.cpp	101
7.63 view_home.cpp	102
7.64 view_info.cpp	102
7.65 view_meteo.cpp	103
7.66 view_settings.cpp	104

Chapter 1

RP2040 GPS & Environment Logger

Firmware per **RP2040** dedicato al logging geospaziale, monitoraggio ambientale e telemetria in tempo reale.

1.1 Core Features

1.1.1 GPS & Telemetry Engine

- **Precision Navigation:** Integrazione con moduli **u-blox** (SAM-M8Q) per il parsing di protocolli NMEA e UBX.
- **Multi-Core Telemetry:** Gestione dell'acquisizione dati e del parsing sul **Core 1** per garantire l'integrità del flusso senza latenze nell'interfaccia utente.
- **Real-time Snapshot:** Utilizzo della struttura atomica `SystemDataPacket` per tracciare coordinate, altitudine, velocità e stato dei satelliti.
- **Fix Diagnostics:** Monitoraggio dello stato del segnale (IDLE, No Fix, 2D/3D) con calcolo dell'età dell'ultimo fix per la sicurezza del volo.

1.1.2 Hybrid Storage System

- **Data Integrity:** Utilizzo di **LittleFS** come filesystem nativo per la persistenza dei log GNSS e della telemetria, scelto per la sua resilienza ai crash energetici (*copy-on-write*) e per l'efficace gestione del *wear-leveling* sulla memoria Flash.
- **USB Mass Storage:** Implementazione di un layer di interfaccia USB (tramite *TinyUSB*) che espone i log contenuti nella Flash come un volume rimovibile leggibile su PC, garantendo la portabilità dei dati senza necessità di driver esterni.
- **Safety Lock:** Protezione del filesystem con rilevamento automatico della connessione USB per inibire le operazioni di scrittura del firmware su Flash durante la modalità Mass Storage, prevenendo conflitti e corruzioni dei dati.

1.1.3 UI & UX Framework

- **Agnostic Display Driver:** Interfaccia grafica su OLED **SSD1306** (128x64) con astrazione completa dalle librerie hardware per facilitare il porting.
- **State Machine UI:** Gestione delle schermate tramite un orchestratore (`ui_manager`) che gestisce la navigazione tra Home, GPS, Meteo e Impostazioni.
- **Multisensory Feedback:**
 - **Visual:** LED RGB (WS2812B) per indicazioni di stato (Success, Error, Neutral).
 - **Acoustic:** Buzzer piezoelettrico per allarmi critici e feedback alla pressione dei tasti.
 - **Input:** Gestione di 4 tasti fisici con debounce software integrato.

1.2 Risorse e Documentazione

Il progetto adotta un approccio ****"Documentation as Code"****. Tutte le risorse tecniche sono centralizzate e aggiornate automaticamente ad ogni release tramite il portale ufficiale:

- **Project Documentation Portal** *Punto di accesso unico per consultare il report accademico, l'architettura del sistema e lo stato dell'hardware.*
- **Project Wiki** *Guida rapida all'installazione, comandi del Makefile e workflow di sviluppo per nuovi contributori.*
- **API Reference (Doxygen)** *Documentazione tecnica generata direttamente dal codice sorgente per classi, driver e task RTOS.*

Chapter 2

Topic Index

2.1 Topics

Here is a list of all topics with brief descriptions:

Gestione Errori	9
Protocollo Dati	10
Gestione Memoria	11
Gestione Sistema	15
Telemetria	18
Mappatura Pin Hardware	19
Driver Display OLED	23
Gestione Input	25
Driver Periferiche	27
Gestione Sensori	31
Master Control	33
RealTimeWorker	33
Gestione Task	34
Animazioni Avvio	35
Gestione UI	35
Definizioni Viste	37
Gestore Task	38

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

error_report_t	Struttura dati per il report di un errore	39
SystemConfig	Parametri di configurazione globale del dispositivo	40
Task	Struttura dati che definisce un task periodico	40
Vector3D_t	Struttura dati per vettori inerziali (Accelerazione/Giroscopio)	41
view_interface_t	Struttura che definisce l'interfaccia funzionale di una schermata (View)	42
WeatherDataPacket	Pacchetto contenente le informazioni meteo correnti	43

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

firm/include/core/config.h	
Gestione delle configurazioni di sistema persistenti	45
firm/include/core/error_codes.h	
Definizioni degli stati di errore e diagnostica di sistema	47
firm/include/core/messages.h	
Definizione delle strutture dati per il protocollo di comunicazione	48
firm/include/core/power_manager.h	
Gestore agnostico dell'alimentazione e dei carichi hardware	50
firm/include/core/storage.h	
Interfaccia per la gestione della persistenza dati su file system (LittleFS)	53
firm/include/core/system_manager.h	
Gestione del middleware di sistema e comunicazione IPC tra core	55
firm/include/core/telemetry.h	
Gestione telemetria con accesso atomico per architetture multi-core	57
firm/include/drivers/config_pins.h	
Mappatura hardware GLOBALE e REALE per RP2040 (PPSE Lab Edition)	59
firm/include/drivers/display_ssd1306.h	
Driver di basso livello agnostico per il controllo dell'OLED SSD1306	61
firm/include/drivers/inputs.h	
Driver agnostico per la gestione degli input fisici (Pulsanti)	62
firm/include/drivers/peripherals.h	
Driver agnostico unificato per feedback (LED, Buzzer) e sensori	64
firm/include/drivers/radio_uart.h	
Driver agnostico per la gestione delle interfacce seriali (UART0 e UART1)	66
firm/include/drivers/sensors_i2c.h	
Driver agnostico per il bus I2C0 e la sensoristica inerziale/ambientale	68
firm/include/pipeline/core0_manager.h	
Punto di ingresso e ciclo di controllo principale per il Core 0	69
firm/include/pipeline/core1_manager.h	
Punto di ingresso e ciclo di elaborazione per il Core 1	70
firm/include/pipeline/tasks.h	
Dichiarazione dei task di sistema per lo scheduler	71
firm/include/ui/boot_anim.h	
Gestione delle sequenze grafiche di avvio (Boot Animations)	73
firm/include/ui/ui_manager.h	
Orchestratore agnostico dell'interfaccia utente	74
firm/include/ui/view_definitions.h	
Definizione del contratto per le View dell'interfaccia utente	75
firm/include/util/scheduler.h	
Motore di esecuzione multitasking cooperativo per task periodici	76

firm/include/util/ target.h	78
firm/src/main.cpp	94
firm/src/core/ config.cpp	80
firm/src/core/ power_manager.cpp	80
firm/src/core/ storage.cpp	80
firm/src/core/ system_manager.cpp	82
firm/src/core/ telemetry.cpp	84
firm/src/drivers/ display_ssd1306.cpp	
Implementazione driver OLED via I2C0 nativo per PCB PPSE Lab	87
firm/src/drivers/ inputs.cpp	89
firm/src/drivers/ peripherals.cpp	
Implementazione driver con logica di feedback a stati e boot sequence	89
firm/src/drivers/ radio_uart.cpp	93
firm/src/drivers/ sensors_i2c.cpp	93
firm/src/pipeline/ core0_manager.cpp	94
firm/src/pipeline/ core1_manager.cpp	96
firm/src/pipeline/ tasks.cpp	97
firm/src/ui/ boot_anim.cpp	
Animazione di boot hacker-style con rendering asincrono dei LED	97
firm/src/ui/ ui_manager.cpp	99
firm/src/ui/views/ view_gps.cpp	101
firm/src/ui/views/ view_home.cpp	102
firm/src/ui/views/ view_info.cpp	102
firm/src/ui/views/ view_meteo.cpp	103
firm/src/ui/views/ view_settings.cpp	104

Chapter 5

Topic Documentation

5.1 Gestione Errori

Data Structures

- struct `error_report_t`

Struttura dati per il report di un errore.

Enumerations

- enum `error_category_t` {
 `ERR_CAT_NONE` = 0x00 , `ERR_CAT_SYS` = 0x01 , `ERR_CAT_HW` = 0x02 , `ERR_CAT_SENSORS` = 0x03
 ,
 `ERR_CAT_POWER` = 0x04 , `ERR_CAT_STORAGE` = 0x05 }
Categorie principali di errore.
- enum `error_code_t` {
 `SUCCESS` = 0 , `ERR_SYS_IPC_TIMEOUT` = 0x10 , `ERR_SYS_WATCHDOG` = 0x11 , `ERR_SYS_OUT_OF_MEMORY` = 0x12 ,
 `ERR_HW_I2C_BUS_LOCK` = 0x20 , `ERR_HW_DISPLAY_LOST` = 0x21 , `ERR_HW_UART_OVERRUN` = 0x22 , `ERR_SENS_GPS_NO_DATA` = 0x30 ,
 `ERR_SENS_IMU_NOT_CAL` = 0x31 , `ERR_SENS_GPS_BAD_FIX` = 0x32 , `ERR_PWR_BATT_LOW` = 0x40 , `ERR_PWR_BATT_CRITICAL` = 0x41 ,
 `ERR_PWR_USB_OVERVOLT` = 0x42 , `ERR_STG_SD_NOT_FOUND` = 0x50 , `ERR_STG_WRITE_FAIL` = 0x51 , `ERR_STG_FS_CORRUPT` = 0x52 }
Codici errore specifici.

5.1.1 Detailed Description

5.1.2 Enumeration Type Documentation

5.1.2.1 `error_category_t`

enum `error_category_t`

Categorie principali di errore.

Utilizzate per classificare l'origine dell'anomalia.

Enumerator

<code>ERR_CAT_NONE</code>	Nessun errore.
<code>ERR_CAT_SYS</code>	Errori critici di sistema (Kernel/IPC).
<code>ERR_CAT_HW</code>	Errori driver o periferiche fisiche.
<code>ERR_CAT_SENSORS</code>	Errori acquisizione dati (GPS/IMU).
<code>ERR_CAT_POWER</code>	Anomalie elettriche o batteria.
<code>ERR_CAT_STORAGE</code>	Errori filesystem o SD Card.

Definition at line 20 of file [error_codes.h](#).

5.1.2.2 error_code_t

enum [error_code_t](#)

Codici errore specifici.

La numerazione è suddivisa per categorie (es. 0x1x per System, 0x2x per HW). Questo design facilita il debug logico sui bus di comunicazione IPC.

Enumerator

SUCCESS	Operazione completata correttamente.
ERR_SYS_IPC_TIMEOUT	Il Core partner non risponde.
ERR_SYS_WATCHDOG	Reset causato da Watchdog.
ERR_SYS_OUT_OF_MEMORY	Fallimento allocazione RAM.
ERR_HW_I2C_BUS_LOCK	Bus I2C bloccato (SDA/SCL low).
ERR_HW_DISPLAY_LOST	Display SSD1306 non risponde.
ERR_HW_UART_OVERRUN	Buffer seriale pieno (Perdita dati).
ERR_SENS_GPS_NO_DATA	Nessun byte ricevuto dal modulo GPS.
ERR_SENS_IMU_NOT_CAL	Accelerometro non calibrato.
ERR_SENS_GPS_BAD_FIX	Segnale GPS degradato (HDOP alto).
ERR_PWR_BATT_LOW	Batteria sotto la soglia di sicurezza.
ERR_PWR_BATT_CRITICAL	Tensione critica: atterraggio immediato.
ERR_PWR_USB_OVERVOLT	Sovratensione rilevata su porta USB.
ERR_STG_SD_NOT_FOUND	SD Card non inserita.
ERR_STG_WRITE_FAIL	Fallimento scrittura log (Card full/error).
ERR_STG_FS_CORRUPT	Filesystem corrotto.

Definition at line 34 of file [error_codes.h](#).

5.2 Protocollo Dati

Data Structures

- struct [WeatherDataPacket](#)

Pacchetto contenente le informazioni meteo correnti.

Enumerations

- enum [GpsStatus_t](#) { [GPS_STATUS_IDLE](#) = 0 , [GPS_STATUS_NO_FIX](#) , [GPS_STATUS_FIX_2D](#) , [GPS_STATUS_FIX_3D](#) }

Enumerazione degli stati operativi del modulo GPS.

Functions

- struct [__attribute__](#) ((packed))

Pacchetto principale di telemetria di sistema.

Variables

- [SystemDataPacket](#)

5.2.1 Detailed Description

5.2.2 Enumeration Type Documentation

5.2.2.1 GpsStatus_t

enum [GpsStatus_t](#)

Enumerazione degli stati operativi del modulo GPS.

Enumerator

GPS_STATUS_IDLE	Modulo in attesa o spento.
GPS_STATUS_NO_FIX	Modulo attivo, ma nessun fix ottenuto.
GPS_STATUS_FIX_2D	Fix 2D ottenuto (lat/lon).
GPS_STATUS_FIX_3D	Fix 3D ottenuto (lat/lon/alt).

Definition at line 20 of file [messages.h](#).

5.2.3 Function Documentation

5.2.3.1 __attribute__()

```
struct __attribute__ (
    (packed) )
```

Pacchetto principale di telemetria di sistema.

Struttura 'packed' per garantire un layout di memoria fisso, essenziale per la serializzazione tra sistemi eterogenei.

< Latitudine (gradi decimali).

< Longitudine (gradi decimali).

< Altitudine in metri.

< Velocità rispetto al suolo in m/s.

< Numero di satelliti utilizzati nel fix.

< Stato del fix ([GpsStatus_t](#)).

< Angolo di beccheggio (gradi).

< Angolo di rollio (gradi).

< Angolo di imbardata (gradi).

< Tensione batteria in Volt.

< Temperatura interna in °C.

< Tempo di accensione in secondi.

< Dati meteo associati alla posizione.

< Ultimo report di errore registrato.

Flag di stato operativo del sistema (bit-field).

< Sistema armato.

< Logging su storage attivo.

< Modulo Wi-Fi attivo.

< Stato di errore critico persistente.

Definition at line 1 of file [messages.h](#).

5.2.4 Variable Documentation

5.2.4.1 SystemDataPacket

[SystemDataPacket](#)

Definition at line 79 of file [messages.h](#).

5.3 Gestione Memoria

Functions

- bool [storage_init](#) (void)

- Inizializza il file system LittleFS.*
- void [storage_log_append](#) (const char *entry)
Aggiunge una riga di log al file delle telemetrie.
- void **storage_sync** (void)
Sincronizza i dati in sospeso con il supporto fisico.
- bool [storage_read_line_with_tag](#) (const char *filename, const char *tag, char *buffer, size_t len)
Cerca una riga contenente uno specifico tag in un file.
- bool [storage_save_params](#) (const void *data, size_t size)
Salva i parametri di configurazione in formato binario.
- bool [storage_load_params](#) (void *data, size_t size)
Carica i parametri di configurazione dalla flash.
- bool [storage_is_busy_by_usb](#) (void)
Verifica se lo storage è impegnato dal protocollo USB (MTP/MSC).
- void [storage_log_gps_fix](#) (float lat, float lon)
Logga una coordinata GPS nel file di log dedicato.
- bool [storage_read_last_gps](#) (float *lat, float *lon)
Recupera l'ultimo fix GPS salvato in memoria.
- void [debug_dump_gps_log](#) (void)
Esegue un dump del file GPS sulla seriale per debug.
- bool [storage_format_all](#) (void)
Formatta l'intera partizione Flash LittleFS.
- bool [storage_save_weather](#) (const [WeatherDataPacket](#) *w)
- bool [storage_load_weather](#) ([WeatherDataPacket](#) *w)

5.3.1 Detailed Description

5.3.2 Function Documentation

5.3.2.1 [debug_dump_gps_log\(\)](#)

```
void debug_dump_gps_log (
    void )
```

Esegue un dump del file GPS sulla seriale per debug.

Definition at line [141](#) of file [storage.cpp](#).

5.3.2.2 [storage_format_all\(\)](#)

```
bool storage_format_all (
    void )
```

Formatta l'intera partizione Flash LittleFS.

Warning

Questa operazione è distruttiva: cancella tutti i file salvati.

Returns

true se la formattazione e il ri-mount hanno avuto successo.

Definition at line [164](#) of file [storage.cpp](#).

5.3.2.3 [storage_init\(\)](#)

```
bool storage_init (
    void )
```

Inizializza il file system LittleFS.

Returns

true se il mount ha avuto successo, false altrimenti.

Definition at line [10](#) of file [storage.cpp](#).

5.3.2.4 storage_is_busy_by_usb()

```
bool storage_is_busy_by_usb (
    void )
```

Verifica se lo storage è impegnato dal protocollo USB (MTP/MSC).

Returns

true se il dispositivo è montato su PC.

Definition at line 73 of file [storage.cpp](#).

5.3.2.5 storage_load_params()

```
bool storage_load_params (
    void * data,
    size_t size )
```

Carica i parametri di configurazione dalla flash.

Parameters

<i>data</i>	Puntatore alla struct di destinazione.
<i>size</i>	Dimensione prevista.

Returns

true se lettura avvenuta con successo.

Definition at line 63 of file [storage.cpp](#).

5.3.2.6 storage_load_weather()

```
bool storage_load_weather (
    WeatherDataPacket * w )
```

Definition at line 193 of file [storage.cpp](#).

5.3.2.7 storage_log_append()

```
void storage_log_append (
    const char * entry )
```

Aggiunge una riga di log al file delle telemetrie.

Parameters

<i>entry</i>	Stringa di testo da accodare (es. riga CSV).
--------------	--

Definition at line 33 of file [storage.cpp](#).

5.3.2.8 storage_log_gps_fix()

```
void storage_log_gps_fix (
    float lat,
    float lon )
```

Logga una coordinata GPS nel file di log dedicato.

Parameters

<i>lat</i>	Latitudine.
<i>lon</i>	Longitudine.

Definition at line 77 of file [storage.cpp](#).

5.3.2.9 `storage_read_last_gps()`

```
bool storage_read_last_gps (
    float * lat,
    float * lon )
```

Recupera l'ultimo fix GPS salvato in memoria.

Parameters

<i>lat</i>	Puntatore dove scrivere la latitudine.
<i>lon</i>	Puntatore dove scrivere la longitudine.

Returns

true se il fix è stato trovato e parsato.

Definition at line 97 of file [storage.cpp](#).

5.3.2.10 `storage_read_line_with_tag()`

```
bool storage_read_line_with_tag (
    const char * filename,
    const char * tag,
    char * buffer,
    size_t len )
```

Cerca una riga contenente uno specifico tag in un file.

Parameters

<i>filename</i>	Percorso del file.
<i>tag</i>	Tag da cercare (es. "FIX").
<i>buffer</i>	Buffer di destinazione.
<i>len</i>	Lunghezza massima del buffer.

Returns

true se trovato, false altrimenti.

5.3.2.11 `storage_save_params()`

```
bool storage_save_params (
    const void * data,
    size_t size )
```

Salva i parametri di configurazione in formato binario.

Parameters

<i>data</i>	Puntatore ai dati da salvare.
<i>size</i>	Dimensione della struttura dati.

Returns

true se scrittura avvenuta con successo.

Definition at line 46 of file [storage.cpp](#).

5.3.2.12 storage_save_weather()

```
bool storage_save_weather (
    const WeatherDataPacket * w )
```

Definition at line 174 of file [storage.cpp](#).

5.4 Gestione Sistema

Functions

- void [sys_manager_init](#) (void)
Inizializza i meccanismi di comunicazione inter-core.
- bool [sys_manager_send_data](#) (SystemDataPacket *packet)
Invia un pacchetto di telemetria al core partner.
- bool [sys_manager_receive_data](#) (SystemDataPacket *buffer)
Riceve un pacchetto di telemetria dal core partner.
- void [sys_manager_report_error](#) (error_category_t cat, error_code_t code, bool critical)
Notifica un evento di errore al sistema di diagnostica.
- bool [sys_is_core1_ready](#) (void)
Verifica se il Core 1 è pronto a ricevere comandi o dati.
- void [sys_set_core1_ready](#) (bool ready)
Imposta lo stato di disponibilità del Core 1.
- void [sys_manager_handle_serial](#) (void)
Gestisce il parsing dei comandi in arrivo dalla porta seriale.
- bool [sys_manager_update_weather](#) (const WeatherDataPacket &weather)
Aggiorna localmente i dati meteo di sistema.
- bool [sys_manager_update_gps](#) (float lat, float lon, bool fix, uint8_t sats)
Aggiorna le coordinate GPS correnti.
- void [sys_manager_clear_error](#) (void)
Resetta lo stato di errore globale, rimuovendo notifiche pendenti.
- bool [storage_save_weather](#) (const WeatherDataPacket *w)
- bool [storage_load_weather](#) (WeatherDataPacket *w)

5.4.1 Detailed Description

5.4.2 Function Documentation

5.4.2.1 storage_load_weather()

```
bool storage_load_weather (
    WeatherDataPacket * w )
```

Definition at line 193 of file [storage.cpp](#).

5.4.2.2 storage_save_weather()

```
bool storage_save_weather (
    const WeatherDataPacket * w )
```

Definition at line 174 of file [storage.cpp](#).

5.4.2.3 sys_is_core1_ready()

```
bool sys_is_core1_ready (
    void )
```

Verifica se il Core 1 è pronto a ricevere comandi o dati.

Returns

true se il Core 1 è nello stato 'ready'.

Definition at line 66 of file [system_manager.cpp](#).

5.4.2.4 sys_manager_clear_error()

```
void sys_manager_clear_error (
    void )
```

Resetta lo stato di errore globale, rimuovendo notifiche pendenti.

Definition at line 55 of file [system_manager.cpp](#).

5.4.2.5 sys_manager_handle_serial()

```
void sys_manager_handle_serial (
    void )
```

Gestisce il parsing dei comandi in arrivo dalla porta seriale.

Definition at line 69 of file [system_manager.cpp](#).

5.4.2.6 sys_manager_init()

```
void sys_manager_init (
    void )
```

Inizializza i meccanismi di comunicazione inter-core.

Definition at line 11 of file [system_manager.cpp](#).

5.4.2.7 sys_manager_receive_data()

```
bool sys_manager_receive_data (
    SystemDataPacket * buffer )
```

Riceve un pacchetto di telemetria dal core partner.

Parameters

<i>buffer</i>	Buffer di destinazione dove memorizzare i dati ricevuti.
---------------	--

Returns

true se il pacchetto è stato ricevuto e validato.

Definition at line 33 of file [system_manager.cpp](#).

5.4.2.8 sys_manager_report_error()

```
void sys_manager_report_error (
    error_category_t cat,
    error_code_t code,
    bool critical )
```

Notifica un evento di errore al sistema di diagnostica.

Parameters

<i>cat</i>	Categoria dell'errore (error_category_t).
<i>code</i>	Codice specifico dell'errore (error_code_t).
<i>critical</i>	Se true, innescia procedure di Failsafe immediate.

Definition at line 40 of file [system_manager.cpp](#).

5.4.2.9 sys_manager_send_data()

```
bool sys_manager_send_data (
    SystemDataPacket * packet )
```

Invia un pacchetto di telemetria al core partner.

Parameters

<i>packet</i>	Puntatore alla struttura <code>SystemDataPacket</code> da trasmettere.
---------------	--

Returns

true se l'invio è avvenuto con successo.

Definition at line 26 of file [system_manager.cpp](#).

5.4.2.10 sys_manager_update_gps()

```
bool sys_manager_update_gps (
    float lat,
    float lon,
    bool fix,
    uint8_t sats )
```

Aggiorna le coordinate GPS correnti.

Parameters

<i>lat</i>	Latitudine.
<i>lon</i>	Longitudine.
<i>fix</i>	Flag di validità del fix.
<i>sats</i>	Numero di satelliti rilevati.

Returns

true se i dati sono stati correttamente aggiornati.

Definition at line 149 of file [system_manager.cpp](#).

5.4.2.11 sys_manager_update_weather()

```
bool sys_manager_update_weather (
    const WeatherDataPacket & weather )
```

Aggiorna localmente i dati meteo di sistema.

Parameters

<i>weather</i>	Riferimento alla struttura WeatherDataPacket aggiornata.
----------------	--

Returns

true se l'aggiornamento è andato a buon fine.

Definition at line 141 of file [system_manager.cpp](#).

5.4.2.12 sys_set_core1_ready()

```
void sys_set_core1_ready (
    bool ready )
```

Imposta lo stato di disponibilità del Core 1.

Parameters

<i>ready</i>	Stato di disponibilità.
--------------	-------------------------

Definition at line 67 of file [system_manager.cpp](#).

5.5 Telemetria

Functions

- void [telemetry_init](#) (void)
Inizializza i buffer di telemetria e i meccanismi di sincronizzazione.
- void [telemetry_update](#) (void)
Aggiorna il buffer di telemetria interno con i dati di sistema correnti.
- bool [telemetry_get_frame](#) (SystemDataPacket *dest)
Copia il frame di telemetria corrente nella destinazione fornita.
- bool [telemetry_is_healthy](#) (void)
Verifica lo stato di salute dei dati di telemetria.

5.5.1 Detailed Description

5.5.2 Function Documentation

5.5.2.1 [telemetry_get_frame\(\)](#)

```
bool telemetry_get_frame (
    SystemDataPacket * dest )
```

Copia il frame di telemetria corrente nella destinazione fornita.

Utilizza un meccanismo di accesso sicuro per evitare la lettura di dati parzialmente aggiornati (data tearing) su architetture multi-core.

Parameters

<i>dest</i>	Puntatore alla struttura SystemDataPacket di destinazione.
-------------	--

Returns

true se il frame è stato copiato correttamente.

Definition at line 209 of file [telemetry.cpp](#).

5.5.2.2 [telemetry_init\(\)](#)

```
void telemetry_init (
    void )
```

Inizializza i buffer di telemetria e i meccanismi di sincronizzazione.

< Pin 7 RP2040 (DRX_SAM)

< Pin 6 RP2040 (KITX_SAM)

Definition at line 30 of file [telemetry.cpp](#).

5.5.2.3 [telemetry_is_healthy\(\)](#)

```
bool telemetry_is_healthy (
    void )
```

Verifica lo stato di salute dei dati di telemetria.

Returns

true se il sistema è in grado di fornire telemetria valida.

Definition at line 222 of file [telemetry.cpp](#).

5.5.2.4 telemetry_update()

```
void telemetry_update (
    void )
```

Aggiorna il buffer di telemetria interno con i dati di sistema correnti.

Dovrebbe essere chiamata periodicamente dai task di sistema per mantenere i dati aggiornati.

Definition at line 162 of file [telemetry.cpp](#).

5.6 Mappatura Pin Hardware

Bus di Comunicazione (I2C & UART)

- #define [I2C0_SDA_PIN](#) 0
- #define [I2C0_SCL_PIN](#) 1
- #define [GPS_TX_PIN](#) 4
- *UART1: Modulo GPS SAM-M8Q.*
- #define [GPS_RX_PIN](#) 5
- #define [GPSBAUD](#) 9600
- #define [ESP_TX_PIN](#) 16
- *UART0: Modulo WiFi ESP-01.*
- #define [ESP_RX_PIN](#) 17

Controllo Alimentazione e Reset Moduli

- #define [SAM_EN_PIN](#) 18
- #define [SAM_RST_PIN](#) 12
- #define [ESP_EN_PIN](#) 3
- #define [BUCK_5V_EN_PIN](#) 11
- #define [OUTPUT_USB_EN_PIN](#) 27

Input Utente (Pulsanti e Interruttori)

- #define [BTN_UP_PIN](#) 22
- #define [BTN_DOWN_PIN](#) 23
- #define [BTN_RIGHT_PIN](#) 24
- #define [BTN_LEFT_PIN](#) 25
- #define [BTN_OK_PIN](#) [BTN_RIGHT_PIN](#)
- #define [BTN_BACK_PIN](#) [BTN_LEFT_PIN](#)

Feedback, Stato e Linee di Interrupt

- #define [NUM_LEDS](#) 8
- #define [RGB_LED_PIN](#) 13
- #define [BUZZER_PIN](#) 21
- #define [LED_ALIVE_PIN](#) 20
- #define [ACC_INT1_PIN](#) 14
- #define [ACC_INT2_PIN](#) 15
- #define [SAM_PPS_PIN](#) 19

Attuatori PWM (Servo)

- #define [SERVO_1_PIN](#) 8
- #define [SERVO_2_PIN](#) 9
- #define [SERVO_3_PIN](#) 7

Canali Analogici (ADC Hardware)

- `#define TEMP_SENS_PIN 26`
- `#define ANALOG_BATT_PIN 28`

5.6.1 Detailed Description

5.6.2 Macro Definition Documentation

5.6.2.1 ACC_INT1_PIN

`#define ACC_INT1_PIN 14`
Interrupt 1 Accelerometro
Definition at line 61 of file [config_pins.h](#).

5.6.2.2 ACC_INT2_PIN

`#define ACC_INT2_PIN 15`
Interrupt 2 Accelerometro
Definition at line 62 of file [config_pins.h](#).

5.6.2.3 ANALOG_BATT_PIN

`#define ANALOG_BATT_PIN 28`
Monitoraggio tensione batteria (partitore)
Definition at line 80 of file [config_pins.h](#).

5.6.2.4 BTN_BACK_PIN

`#define BTN_BACK_PIN BTN_LEFT_PIN`
Aliasing per logica UI
Definition at line 50 of file [config_pins.h](#).

5.6.2.5 BTN_DOWN_PIN

`#define BTN_DOWN_PIN 23`
Definition at line 46 of file [config_pins.h](#).

5.6.2.6 BTN_LEFT_PIN

`#define BTN_LEFT_PIN 25`
Definition at line 48 of file [config_pins.h](#).

5.6.2.7 BTN_OK_PIN

`#define BTN_OK_PIN BTN_RIGHT_PIN`
Aliasing per logica UI
Definition at line 49 of file [config_pins.h](#).

5.6.2.8 BTN_RIGHT_PIN

`#define BTN_RIGHT_PIN 24`
Definition at line 47 of file [config_pins.h](#).

5.6.2.9 BTN_UP_PIN

`#define BTN_UP_PIN 22`
Definition at line 45 of file [config_pins.h](#).

5.6.2.10 BUCK_5V_EN_PIN

```
#define BUCK_5V_EN_PIN 11
```

Controllo abilitazione step-down 5V
Definition at line 37 of file [config_pins.h](#).

5.6.2.11 BUZZER_PIN

```
#define BUZZER_PIN 21
```

Uscita PWM per feedback sonoro
Definition at line 59 of file [config_pins.h](#).

5.6.2.12 ESP_EN_PIN

```
#define ESP_EN_PIN 3
```

Abilitazione modulo WiFi
Definition at line 36 of file [config_pins.h](#).

5.6.2.13 ESP_RX_PIN

```
#define ESP_RX_PIN 17
```

ESP-01 RX (verso RP2040 TX)
Definition at line 27 of file [config_pins.h](#).

5.6.2.14 ESP_TX_PIN

```
#define ESP_TX_PIN 16
```

UART0: Modulo WiFi ESP-01.
ESP-01 TX (verso RP2040 RX)
Definition at line 26 of file [config_pins.h](#).

5.6.2.15 GPS_RX_PIN

```
#define GPS_RX_PIN 5
```

Pin 6 RP2040 (KITX_SAM)
Definition at line 22 of file [config_pins.h](#).

5.6.2.16 GPS_TX_PIN

```
#define GPS_TX_PIN 4
```

UART1: Modulo GPS SAM-M8Q.
Pin 7 RP2040 (DRX_SAM)
Definition at line 21 of file [config_pins.h](#).

5.6.2.17 GPSBAUD

```
#define GPSBAUD 9600
```

Baud rate standard modulo SAM-M8Q
Definition at line 23 of file [config_pins.h](#).

5.6.2.18 I2C0_SCL_PIN

```
#define I2C0_SCL_PIN 1
```

Bus I2C: Clock
Definition at line 18 of file [config_pins.h](#).

5.6.2.19 I2C0_SDA_PIN

```
#define I2C0_SDA_PIN 0
```

Bus I2C: Dati
Definition at line 17 of file [config_pins.h](#).

5.6.2.20 LED_ALIVE_PIN

```
#define LED_ALIVE_PIN 20
```

LED di stato sistema (Heartbeat)
Definition at line 60 of file [config_pins.h](#).

5.6.2.21 NUM_LEDS

```
#define NUM_LEDS 8
```

Numero di LED indirizzabili WS2812B
Definition at line 57 of file [config_pins.h](#).

5.6.2.22 OUTPUT_USB_EN_PIN

```
#define OUTPUT_USB_EN_PIN 27
```

Abilitazione porta USB di output
Definition at line 38 of file [config_pins.h](#).

5.6.2.23 RGB_LED_PIN

```
#define RGB_LED_PIN 13
```

Segnale data per strip LED
Definition at line 58 of file [config_pins.h](#).

5.6.2.24 SAM_EN_PIN

```
#define SAM_EN_PIN 18
```

Abilitazione modulo GPS
Definition at line 34 of file [config_pins.h](#).

5.6.2.25 SAM_PPS_PIN

```
#define SAM_PPS_PIN 19
```

Pulse Per Second GPS (Time sync)
Definition at line 63 of file [config_pins.h](#).

5.6.2.26 SAM_RST_PIN

```
#define SAM_RST_PIN 12
```

Reset fisico modulo GPS
Definition at line 35 of file [config_pins.h](#).

5.6.2.27 SERVO_1_PIN

```
#define SERVO_1_PIN 8
```

Definition at line 70 of file [config_pins.h](#).

5.6.2.28 SERVO_2_PIN

```
#define SERVO_2_PIN 9
```

Definition at line 71 of file [config_pins.h](#).

5.6.2.29 SERVO_3_PIN

```
#define SERVO_3_PIN 7
```

Definition at line 72 of file [config_pins.h](#).

5.6.2.30 TEMP_SENS_PIN

```
#define TEMP_SENS_PIN 26
```

Ingresso sensore temperatura

Definition at line 79 of file [config_pins.h](#).

5.7 Driver Display OLED

Functions

- bool [display_hw_init](#) (void)
Inizializza l'hardware I2C e il controller OLED.
- bool [display_is_alive](#) (void)
Verifica se il display è fisicamente connesso al bus I2C.
- void [display_clear](#) (void)
Pulisce il buffer interno (RAM) del display.
- void [display_show](#) (void)
Invia il buffer corrente al display fisico (Refresh).
- void [display_set_power](#) (bool on)
Gestione energetica del pannello.
- void [display_set_brightness](#) (uint8_t level)
Regola il contrasto (luminosità) dei pixel OLED.
- void [display_draw_pixel](#) (int16_t x, int16_t y, uint16_t color)
Funzione primitiva di disegno agnostica.
- void * [get_display_driver](#) (void)
Fornisce l'accesso al driver sottostante (uso avanzato).

Costanti Fisiche Pannello

- - #define [SCREEN_WIDTH](#) 128
 - #define [SCREEN_HEIGHT](#) 64
 - #define [SCREEN_ADDRESS](#) 0x3C

5.7.1 Detailed Description

5.7.2 Macro Definition Documentation

5.7.2.1 SCREEN_ADDRESS

```
#define SCREEN_ADDRESS 0x3C
```

Indirizzo I2C standard SSD1306.

Definition at line 21 of file [display_ssd1306.h](#).

5.7.2.2 SCREEN_HEIGHT

```
#define SCREEN_HEIGHT 64
```

Altezza display in pixel.

Definition at line 20 of file [display_ssd1306.h](#).

5.7.2.3 SCREEN_WIDTH

```
#define SCREEN_WIDTH 128
```

Larghezza display in pixel.

Definition at line 19 of file [display_ssd1306.h](#).

5.7.3 Function Documentation

5.7.3.1 `display_clear()`

```
void display_clear (
    void )
```

Pulisce il buffer interno (RAM) del display.
Riempie il buffer con pixel "spenti" (nero).
Definition at line 46 of file [display_ssd1306.cpp](#).

5.7.3.2 `display_draw_pixel()`

```
void display_draw_pixel (
    int16_t x,
    int16_t y,
    uint16_t color )
```

Funzione primitiva di disegno agnostica.

Parameters

<i>x</i>	Coordinata orizzontale.
<i>y</i>	Coordinata verticale.
<i>color</i>	1 per acceso, 0 per spento.

Definition at line 65 of file [display_ssd1306.cpp](#).

5.7.3.3 `display_hw_init()`

```
bool display_hw_init (
    void )
```

Inizializza l'hardware I2C e il controller OLED.
Configura il bus I2C0 dell'RP2040 e alloca il buffer di memoria.

Returns

true se il display è stato trovato e inizializzato correttamente.

< Bus I2C: Dati
< Bus I2C: Clock
< Indirizzo I2C standard SSD1306.
< Indirizzo I2C standard SSD1306.

Definition at line 16 of file [display_ssd1306.cpp](#).

5.7.3.4 `display_is_alive()`

```
bool display_is_alive (
    void )
```

Verifica se il display è fisicamente connesso al bus I2C.

Returns

true se il device risponde all'indirizzo SCREEN_ADDRESS.

< Indirizzo I2C standard SSD1306.

Definition at line 40 of file [display_ssd1306.cpp](#).

5.7.3.5 `display_set_brightness()`

```
void display_set_brightness (
    uint8_t level )
```

Regola il contrasto (luminosità) dei pixel OLED.

Parameters

<i>level</i>	Valore da 0 (minimo contrasto) a 255 (massimo).
--------------	---

Definition at line 59 of file [display_ssd1306.cpp](#).

5.7.3.6 display_set_power()

```
void display_set_power (
    bool on )
```

Gestione energetica del pannello.

Parameters

<i>on</i>	true per attivare il display, false per metterlo in Sleep (consumo < 10uA).
-----------	---

Definition at line 54 of file [display_ssd1306.cpp](#).

5.7.3.7 display_show()

```
void display_show (
    void )
```

Invia il buffer corrente al display fisico (Refresh).

Questa è l'operazione più onerosa in termini di tempo sul bus I2C.

Definition at line 50 of file [display_ssd1306.cpp](#).

5.7.3.8 get_display_driver()

```
void * get_display_driver (
    void )
```

Fornisce l'accesso al driver sottostante (uso avanzato).

Restituisce un puntatore generico (void*) per mantenere l'header agnostico da librerie specifiche (es. Adafruit GFX).

Note

Il chiamante dovrà effettuare il cast esplicito a (Adafruit_SSD1306*) nel proprio file .cpp.

Returns

Puntatore all'oggetto driver hardware.

Definition at line 69 of file [display_ssd1306.cpp](#).

5.8 Gestione Input

Typedefs

- typedef [ButtonId](#) [button_t](#)
Alias per compatibilità con il framework UI.

Enumerations

- enum [ButtonId](#) {
 [BTN_NONE](#) = 0 , [BTN_UP](#) , [BTN_DOWN](#) , [BTN_OK](#) ,
 [BTN_BACK](#) , [BTN_COUNT](#) }
Identificativi logici dei pulsanti del sistema.
- enum [button_state_t](#) { [BTN_PRESSED](#) , [BTN_RELEASED](#) }
Stati possibili di un evento pulsante.

Functions

- void `inputs_init` (void)
Inizializza i GPIO e le strutture di debounce per i pulsanti.
- `ButtonId inputs_get_last_press` (void)
Recupera l'ultimo evento di pressione (Logica a consumo).
- void `inputs_update` (void)
Task di aggiornamento periodico (Debounce).
- bool `inputs_is_pressed` (`ButtonId` btn)
Verifica lo stato fisico attuale (istantaneo) di un pulsante.

5.8.1 Detailed Description

5.8.2 Typedef Documentation

5.8.2.1 `button_t`

`typedef ButtonId button_t`

Alias per compatibilità con il framework UI.

•

Definition at line 30 of file `inputs.h`.

5.8.3 Enumeration Type Documentation

5.8.3.1 `button_state_t`

`enum button_state_t`

Stati possibili di un evento pulsante.

Enumerator

BTN_PRESSED	Tasto appena premuto (Leading edge).
BTN_RELEASED	Tasto rilasciato (Trailing edge - trigger standard UI).

Definition at line 35 of file `inputs.h`.

5.8.3.2 `ButtonId`

`enum ButtonId`

Identificativi logici dei pulsanti del sistema.

Enumerator

BTN_NONE	Nessun evento registrato.
BTN_UP	Pulsante Navigazione SU / Incremento.
BTN_DOWN	Pulsante Navigazione GIÙ / Decremento.
BTN_OK	Pulsante Selezione / Conferma / Arm motori.
BTN_BACK	Pulsante Indietro / Annulla / Disarm.
BTN_COUNT	Helper per la dimensione degli array di stato.

Definition at line 19 of file `inputs.h`.

5.8.4 Function Documentation

5.8.4.1 inputs_get_last_press()

```
ButtonId inputs_get_last_press (
    void )
```

Recupera l'ultimo evento di pressione (Logica a consumo).

Questa funzione consuma l'evento: dopo la lettura, il buffer interno viene resettato a BTN_NONE.

Returns

ButtonId L'identificativo dell'ultimo pulsante rilevato.

Definition at line 38 of file [inputs.cpp](#).

5.8.4.2 inputs_init()

```
void inputs_init (
    void )
```

Inizializza i GPIO e le strutture di debounce per i pulsanti.

< Aliasing per logica UI

< Aliasing per logica UI

< Aliasing per logica UI

< Aliasing per logica UI

Definition at line 22 of file [inputs.cpp](#).

5.8.4.3 inputs_is_pressed()

```
bool inputs_is_pressed (
    ButtonId btn )
```

Verifica lo stato fisico attuale (istantaneo) di un pulsante.

Parameters

<i>btn</i>	Identificativo del pulsante da interrogare.
------------	---

Returns

true se il pulsante è attualmente mantenuto premuto.

< Aliasing per logica UI

< Aliasing per logica UI

Definition at line 44 of file [inputs.cpp](#).

5.8.4.4 inputs_update()

```
void inputs_update (
    void )
```

Task di aggiornamento periodico (Debounce).

Implementa la logica di campionamento temporale per eliminare i rimbalzi elettromeccanici. Deve essere richiamato con un timer costante (es. 20ms).

5.9 Driver Periferiche

Enumerations

- enum [FeedbackType](#) { [FEEDBACK_SUCCESS](#) , [FEEDBACK_ERROR](#) , [FEEDBACK_NEUTRAL](#) , [FEEDBACK_OFF](#) }

Tipologie di feedback utente per azioni UI.

- enum `LedAnimState` {
`LED_ANIM_NONE` , `LED_ANIM_BOOT` , `LED_ANIM_BOOT_DONE` , `LED_ANIM_SEARCHING` ,
`LED_ANIM_FIX_OK` , `LED_ANIM_SIGNAL_LOW` }

Stati di animazione LED per sistema di feedback visuale.

Inizializzazione

- void `peripherals_init` (void)
- void `peripherals_init_leds` (void)

Controllo Fisico Base

- void `peripherals_set_leds` (bool enabled)
- void `peripherals_set_buzzer` (bool enabled)

Feedback Visivo e Acustico

- void `peripherals_update_led_fx` (`LedAnimState` state, float val)
Aggiorna l'animazione LED basata sullo stato di sistema.
- void `peripherals_auto_feedback` (const `SystemDataPacket` *data)
Esegue un feedback automatico basato sui dati di telemetria.
- void `peripherals_trigger_feedback` (`FeedbackType` type)
Attiva un feedback sonoro/visivo basato su un evento.
- void `peripherals_set_led` (uint8_t r, uint8_t g, uint8_t b)
Imposta colore RGB statico per i LED.
- void `peripherals_beep` (uint32_t freq_hz, uint32_t duration_ms)
Esegue un breve suono (beep).
- void `peripherals_play_click` (void)
Esegue il tipico suono di "click" per conferma UI.

Acquisizione Dati Ambientali

- float `peripherals_get_temperature` (void)
- float `peripherals_get_battery_voltage` (void)

5.9.1 Detailed Description

5.9.2 Enumeration Type Documentation

5.9.2.1 FeedbackType

enum `FeedbackType`

Tipologie di feedback utente per azioni UI.

Enumerator

<code>FEEDBACK_SUCCESS</code>	Feedback positivo (es. OK salvato).
<code>FEEDBACK_ERROR</code>	Feedback negativo (es. errore operazione).
<code>FEEDBACK_NEUTRAL</code>	Feedback informativo neutro.
<code>FEEDBACK_OFF</code>	Disabilita feedback.

Definition at line 20 of file `peripherals.h`.

5.9.2.2 LedAnimState

enum `LedAnimState`

Stati di animazione LED per sistema di feedback visuale.

Enumerator

LED_ANIM_NONE	Nessuna animazione.
LED_ANIM_BOOT	Animazione fase di avvio.
LED_ANIM_BOOT_DONE	Segnalazione completamento boot.
LED_ANIM_SEARCHING	Ricerca segnale (es. GPS/Wi-Fi).
LED_ANIM_FIX_OK	Segnale acquisito correttamente.
LED_ANIM_SIGNAL_LOW	Segnale debole o degradato.

Definition at line 30 of file [peripherals.h](#).

5.9.3 Function Documentation

5.9.3.1 peripherals_auto_feedback()

```
void peripherals_auto_feedback (
    const SystemDataPacket * data )
```

Esegue un feedback automatico basato sui dati di telemetria.

Parameters

<i>data</i>	Pacchetto telemetrico da analizzare.
-------------	--------------------------------------

< Numero di LED indirizzabili WS2812B

Definition at line 130 of file [peripherals.cpp](#).

5.9.3.2 peripherals_beep()

```
void peripherals_beep (
    uint32_t freq_hz,
    uint32_t duration_ms )
```

Esegue un breve suono (beep).

Parameters

<i>freq_hz</i>	Frequenza in Hertz.
<i>duration_ms</i>	Durata in millisecondi.

< Uscita PWM per feedback sonoro

Definition at line 174 of file [peripherals.cpp](#).

5.9.3.3 peripherals_get_temperature()

```
float peripherals_get_temperature (
    void )
```

< Ingresso sensore temperatura

Definition at line 208 of file [peripherals.cpp](#).

5.9.3.4 peripherals_init()

```
void peripherals_init (
    void )
```

< LED di stato sistema (Heartbeat)

< LED di stato sistema (Heartbeat)

< LED di stato sistema (Heartbeat)

< LED di stato sistema (Heartbeat)

< Controllo abilitazione step-down 5V

< Controllo abilitazione step-down 5V
 < Ingresso sensore temperatura
 Definition at line 20 of file [peripherals.cpp](#).

5.9.3.5 peripherals_init_leds()

```
void peripherals_init_leds (
    void )
```

< Segnale data per strip LED
 < Numero di LED indirizzabili WS2812B
 < Numero di LED indirizzabili WS2812B
 Definition at line 35 of file [peripherals.cpp](#).

5.9.3.6 peripherals_play_click()

```
void peripherals_play_click (
    void )
```

Esegue il tipico suono di "click" per conferma UI.
 Definition at line 59 of file [peripherals.cpp](#).

5.9.3.7 peripherals_set_buzzer()

```
void peripherals_set_buzzer (
    bool enabled )
```

< Uscita PWM per feedback sonoro
 Definition at line 52 of file [peripherals.cpp](#).

5.9.3.8 peripherals_set_led()

```
void peripherals_set_led (
    uint8_t r,
    uint8_t g,
    uint8_t b )
```

Imposta colore RGB statico per i LED.
 < Numero di LED indirizzabili WS2812B
 Definition at line 203 of file [peripherals.cpp](#).

5.9.3.9 peripherals_set_leds()

```
void peripherals_set_leds (
    bool enabled )
```

< Numero di LED indirizzabili WS2812B
 Definition at line 43 of file [peripherals.cpp](#).

5.9.3.10 peripherals_trigger_feedback()

```
void peripherals_trigger_feedback (
    FeedbackType type )
```

Attiva un feedback sonoro/visivo basato su un evento.

Parameters

<i>type</i>	Tipologia di feedback (FeedbackType).
-------------	---------------------------------------

Definition at line 194 of file [peripherals.cpp](#).

5.9.3.11 peripherals_update_led_fx()

```
void peripherals_update_led_fx (
```

```
LedAnimState state,
float val )
```

Aggiorna l'animazione LED basata sullo stato di sistema.

Parameters

<i>state</i>	Stato dell'animazione desiderato (LedAnimState).
<i>val</i>	Valore opzionale (es. luminosità 0.0-1.0 o intensità).

Aggiorna l'animazione LED basata sullo stato di sistema. < Numero di LED indirizzabili WS2812B

< Numero di LED indirizzabili WS2812B

< Numero di LED indirizzabili WS2812B

< Numero di LED indirizzabili WS2812B

< Numero di LED indirizzabili WS2812B

< Numero di LED indirizzabili WS2812B

< Numero di LED indirizzabili WS2812B

Definition at line 68 of file [peripherals.cpp](#).

5.10 Gestione Sensori

Data Structures

- struct [Vector3D_t](#)
Struttura dati per vettori inerziali (Accelerazione/Giroscopio).

Functions

- bool [sensors_i2c_init](#) (void)
Inizializza il bus I2C0 e i sensori collegati.
- bool [sensors_accel_get_data](#) ([Vector3D_t](#) *data)
Legge i dati dall'accelerometro.
- bool [sensors_i2c_is_healthy](#) (void)
Esegue una scansione del bus I2C0 per diagnostica.
- void [sensors_accel_calibrate](#) (void)
Calibra il sensore inerziale.
- float [sensors_read_temperature_c](#) (void)
Legge la temperatura corrente.
- float [sensors_read_battery_v](#) (void)
Legge la tensione della batteria.

5.10.1 Detailed Description

5.10.2 Function Documentation

5.10.2.1 [sensors_accel_calibrate\(\)](#)

```
void sensors_accel_calibrate (
    void )
```

Calibra il sensore inerziale.

Calcola l'offset statico (bias). Il sistema deve essere in posizione piana durante l'operazione.

5.10.2.2 [sensors_accel_get_data\(\)](#)

```
bool sensors_accel_get_data (
    Vector3D\_t * data )
```

Legge i dati dall'accelerometro.

Astrae il modello specifico del sensore (es. LIS3DH).

Parameters

<i>data</i>	Puntatore alla struttura dove salvare i dati.
-------------	---

Returns

true se la lettura è andata a buon fine.

Definition at line 34 of file [sensors_i2c.cpp](#).

5.10.2.3 sensors_i2c_init()

```
bool sensors_i2c_init (  
    void )
```

Inizializza il bus I2C0 e i sensori collegati.

Returns

true se il bus e i sensori critici sono pronti.

Definition at line 6 of file [sensors_i2c.cpp](#).

5.10.2.4 sensors_i2c_is_healthy()

```
bool sensors_i2c_is_healthy (  
    void )
```

Esegue una scansione del bus I2C0 per diagnostica.

Utile in fase di boot per verificare che display e sensori siano correttamente alimentati.

Returns

true se almeno un device risponde sul bus.

Definition at line 12 of file [sensors_i2c.cpp](#).

5.10.2.5 sensors_read_battery_v()

```
float sensors_read_battery_v (  
    void )
```

Legge la tensione della batteria.

Returns

Voltaggio letto dal partitore analogico (Volt).

< Monitoraggio tensione batteria (partitore)

Definition at line 27 of file [sensors_i2c.cpp](#).

5.10.2.6 sensors_read_temperature_c()

```
float sensors_read_temperature_c (  
    void )
```

Legge la temperatura corrente.

Returns

Temperatura in gradi Celsius (°C).

< Ingresso sensore temperatura

Definition at line 20 of file [sensors_i2c.cpp](#).

5.11 Master Control

Functions

- void [core0_setup](#) (void)
Setup iniziale del Core 0.
- void [core0_loop](#) (void)
Ciclo principale di esecuzione del Core 0.

5.11.1 Detailed Description

5.11.2 Function Documentation

5.11.2.1 [core0_loop\(\)](#)

```
void core0_loop (
    void )
```

Ciclo principale di esecuzione del Core 0.

Esegue la logica di polling della UI, l'aggiornamento della telemetria e la gestione dei task di sistema non bloccanti.

< LED di stato sistema (Heartbeat)

Definition at line [173](#) of file [core0_manager.cpp](#).

5.11.2.2 [core0_setup\(\)](#)

```
void core0_setup (
    void )
```

Setup iniziale del Core 0.

Inizializza i driver fondamentali (Display, Storage, Periferiche) e prepara lo stack di sistema prima dell'avvio del ciclo principale. < LED di stato sistema (Heartbeat)

< LED di stato sistema (Heartbeat)

< Controllo abilitazione step-down 5V

< Controllo abilitazione step-down 5V

Definition at line [100](#) of file [core0_manager.cpp](#).

5.12 RealTimeWorker

Functions

- void [core1_setup](#) (void)
Setup iniziale del Core 1.
- void [core1_loop](#) (void)
Ciclo principale di elaborazione del Core 1.

5.12.1 Detailed Description

5.12.2 Function Documentation

5.12.2.1 [core1_loop\(\)](#)

```
void core1_loop (
    void )
```

Ciclo principale di elaborazione del Core 1.

Gestisce il processing continuo dei dati sensoristici e mantiene aggiornato il buffer di telemetria condiviso con il Core 0 tramite IPC.

Definition at line [40](#) of file [core1_manager.cpp](#).

5.12.2.2 core1_setup()

```
void core1_setup (
    void )
```

Setup iniziale del Core 1.

Configura le periferiche real-time, i bus di comunicazione (I2C/UART) e sincronizza lo stato con il Core 0 prima di entrare nel loop. < Abilitazione modulo GPS

< Abilitazione modulo GPS

Definition at line 9 of file [core1_manager.cpp](#).

5.13 Gestione Task

Variables

- [Task task_serial](#)
Task di gestione comunicazione seriale.
- [Task task_input](#)
Task di acquisizione input utente.
- [Task task_ui](#)
Task di aggiornamento dell'interfaccia utente (UI).
- [Task task_health](#)
Task di monitoraggio integrità sistema (Health Check).

5.13.1 Detailed Description

5.13.2 Variable Documentation

5.13.2.1 task_health

[Task task_health](#) [extern]

[Task](#) di monitoraggio integrità sistema (Health Check).

Controlla lo stato di carica della batteria, temperature e diagnostica errori per prevenire guasti critici.

Definition at line 22 of file [tasks.cpp](#).

5.13.2.2 task_input

[Task task_input](#) [extern]

[Task](#) di acquisizione input utente.

Si occupa del polling e del debounce dei pulsanti fisici.

Definition at line 9 of file [tasks.cpp](#).

5.13.2.3 task_serial

[Task task_serial](#) [extern]

[Task](#) di gestione comunicazione seriale.

Gestisce il parsing dei comandi in arrivo e l'invio della telemetria.

Definition at line 7 of file [tasks.cpp](#).

5.13.2.4 task_ui

[Task task_ui](#) [extern]

[Task](#) di aggiornamento dell'interfaccia utente (UI).

Gestisce il render dei menu sul display OLED e la navigazione tra le viste.

Definition at line 14 of file [tasks.cpp](#).

5.14 Animazioni Avvio

Functions

- void [run_hacker_boot_sequence](#) (void(*update_callback)(void))
Esegue la sequenza di avvio "Hacker-style".

5.14.1 Detailed Description

5.14.2 Function Documentation

5.14.2.1 run_hacker_boot_sequence()

```
void run_hacker_boot_sequence (
    void(*) (void)  update_callback )
```

Esegue la sequenza di avvio "Hacker-style".

Questa funzione esegue l'animazione di boot sul display. Utilizza una funzione di callback per permettere al sistema di eseguire operazioni (come il check delle periferiche) durante l'animazione.

Parameters

update_callback	Puntatore a funzione invocato durante ogni frame dell'animazione per mantenere attivi i task di sistema.
---------------------------------	--

•

5.15 Gestione UI

Enumerations

- enum [view_id_t](#) {
 [VIEW_ID_HOME](#) = 0 , [VIEW_ID_GPS](#) , [VIEW_ID_METEO](#) , [VIEW_ID_SETTINGS](#) ,
 [VIEW_ID_INFO](#) , [VIEW_ID_COUNT](#) }
Identificativi univoci per le schermate (view) del sistema.

Functions

- void [ui_manager_init](#) (void)
Inizializza il sistema UI.
- void [ui_manager_update](#) (const void *system_data)
Task di aggiornamento periodico dell'interfaccia.
- void [ui_manager_dispatch_input](#) (uint8_t input_id)
Dispaccia un evento di input alla schermata attiva.
- void [ui_manager_navigate_to](#) ([view_id_t](#) new_view_id)
Cambia la schermata attiva (Navigazione).
- [view_id_t](#) [ui_manager_get_current_view](#) (void)
Restituisce la schermata correntemente visualizzata.

5.15.1 Detailed Description

5.15.2 Enumeration Type Documentation

5.15.2.1 view_id_t

```
enum view_id_t
```

Identificativi univoci per le schermate (view) del sistema.

Enumerator

VIEW_ID_HOME	Schermata principale (Dashboard).
VIEW_ID_GPS	Schermata dati di navigazione.
VIEW_ID_METEO	Schermata dati meteorologici.
VIEW_ID_SETTINGS	Schermata configurazione parametri.
VIEW_ID_INFO	Schermata informazioni di sistema.
VIEW_ID_COUNT	Helper per la dimensione dello stack UI.

Definition at line 19 of file [ui_manager.h](#).

5.15.3 Function Documentation

5.15.3.1 ui_manager_dispatch_input()

```
void ui_manager_dispatch_input (
    uint8_t input_id )
```

Dispaccia un evento di input alla schermata attiva.

Instrada l'evento (es. pressione pulsante) verso la view correntemente visualizzata.

Parameters

<i>input_id</i>	Identificativo dell'input (basato su ButtonId in inputs.h).
-----------------	--

Definition at line 76 of file [ui_manager.cpp](#).

5.15.3.2 ui_manager_get_current_view()

```
view_id_t ui_manager_get_current_view (
    void )
```

Restituisce la schermata correntemente visualizzata.

Returns

L'ID della vista attiva.

Definition at line 138 of file [ui_manager.cpp](#).

5.15.3.3 ui_manager_init()

```
void ui_manager_init (
    void )
```

Inizializza il sistema UI.

Configura il driver del display, inizializza le strutture grafiche e imposta VIEW_ID_HOME come schermata iniziale.

Definition at line 22 of file [ui_manager.cpp](#).

5.15.3.4 ui_manager_navigate_to()

```
void ui_manager_navigate_to (
    view_id_t new_view_id )
```

Cambia la schermata attiva (Navigazione).

Parameters

<i>new_view_id</i>	L'ID della nuova schermata da caricare.
--------------------	---

Definition at line 112 of file [ui_manager.cpp](#).

5.15.3.5 ui_manager_update()

```
void ui_manager_update (
    const void * system_data )
```

Task di aggiornamento periodico dell'interfaccia.

Da richiamare nel loop principale del Core 0. Gestisce il render dei widget basandosi sui dati di telemetria.

Parameters

<code>system_data</code>	Puntatore al pacchetto dati (SystemDataPacket) ricevuto dal Core 1.
--------------------------	---

Definition at line 50 of file [ui_manager.cpp](#).

5.16 Definizioni Viste

Data Structures

- struct [view_interface_t](#)

Struttura che definisce l'interfaccia funzionale di una schermata (View).

Istanze delle Viste di Sistema

Esportazione delle interfacce implementate nei relativi moduli.

- const [view_interface_t](#) [view_home](#)
- const [view_interface_t](#) [view_gps](#)
- const [view_interface_t](#) [view_meteo](#)
- const [view_interface_t](#) [view_settings](#)
- const [view_interface_t](#) [view_info](#)

5.16.1 Detailed Description

5.16.2 Variable Documentation

5.16.2.1 view_gps

```
const view\_interface\_t view_gps [extern]
```

Vista dati di navigazione.

Definition at line 56 of file [view_gps.cpp](#).

5.16.2.2 view_home

```
const view\_interface\_t view_home [extern]
```

Vista Dashboard principale.

Definition at line 62 of file [view_home.cpp](#).

5.16.2.3 view_info

```
const view\_interface\_t view_info [extern]
```

Vista informazioni di sistema.

Definition at line 59 of file [view_info.cpp](#).

5.16.2.4 view_meteo

```
const view\_interface\_t view_meteo [extern]
```

Vista meteo.

Definition at line 71 of file [view_meteo.cpp](#).

5.16.2.5 view_settings

```
const view_interface_t view_settings [extern]
```

Vista menu impostazioni.

Definition at line 103 of file [view_settings.cpp](#).

5.17 Gestore Task

Data Structures

- struct [Task](#)

Struttura dati che definisce un task periodico.

Typedefs

- typedef void(* [TaskCallback](#)) (void)

Firma della funzione callback per un task.

Functions

- void [run_task](#) ([Task](#) *t)

Verifica ed esegue il task se l'intervallo è scaduto.

5.17.1 Detailed Description

5.17.2 Typedef Documentation

5.17.2.1 TaskCallback

```
typedef void(* TaskCallback) (void)
```

Firma della funzione callback per un task.

Definition at line 18 of file [scheduler.h](#).

5.17.3 Function Documentation

5.17.3.1 run_task()

```
void run_task (  
    Task * t ) [inline]
```

Verifica ed esegue il task se l'intervallo è scaduto.

Implementa la logica di controllo non bloccante. Confronta il tempo corrente (millis) con l'ultimo tempo di esecuzione memorizzato.

Parameters

<i>t</i>	Puntatore alla struttura Task da gestire.
----------	---

Definition at line 35 of file [scheduler.h](#).

Chapter 6

Data Structure Documentation

6.1 `error_report_t` Struct Reference

Struttura dati per il report di un errore.

```
#include <error_codes.h>
```

Data Fields

- `error_category_t` `category`
- `error_code_t` `code`
- `uint32_t` `timestamp`
- `bool` `is_critical`

6.1.1 Detailed Description

Struttura dati per il report di un errore.

Utilizzata per inviare notifiche di errore attraverso il bus IPC.

Definition at line 67 of file [error_codes.h](#).

6.1.2 Field Documentation

6.1.2.1 `category`

```
error_category_t error_report_t::category
```

Categoria di appartenenza.

Definition at line 68 of file [error_codes.h](#).

6.1.2.2 `code`

```
error_code_t error_report_t::code
```

Codice errore specifico.

Definition at line 69 of file [error_codes.h](#).

6.1.2.3 `is_critical`

```
bool error_report_t::is_critical
```

Se true, richiede intervento immediato (es. Failsafe).

Definition at line 71 of file [error_codes.h](#).

6.1.2.4 `timestamp`

```
uint32_t error_report_t::timestamp
```

Timestamp in millisecondi dall'avvio (millis).

Definition at line 70 of file [error_codes.h](#).

The documentation for this struct was generated from the following file:

- [firm/include/core/error_codes.h](#)

6.2 SystemConfig Struct Reference

Parametri di configurazione globale del dispositivo.

```
#include <config.h>
```

Data Fields

- `uint8_t oled_brightness`
Luminosità display OLED (range: 0-255)
- `bool buzzer_enabled`
Stato del cicalino (true = attivo)
- `bool leds_enabled`
Stato dei LED di sistema (true = attivi)
- `uint8_t gps_refresh_rate`
Frequenza di aggiornamento del modulo GPS (in Hz)

6.2.1 Detailed Description

Parametri di configurazione globale del dispositivo.

Definition at line 19 of file [config.h](#).

6.2.2 Field Documentation

6.2.2.1 buzzer_enabled

```
bool SystemConfig::buzzer_enabled
```

Stato del cicalino (true = attivo)

Definition at line 21 of file [config.h](#).

6.2.2.2 gps_refresh_rate

```
uint8_t SystemConfig::gps_refresh_rate
```

Frequenza di aggiornamento del modulo GPS (in Hz)

Definition at line 23 of file [config.h](#).

6.2.2.3 leds_enabled

```
bool SystemConfig::leds_enabled
```

Stato dei LED di sistema (true = attivi)

Definition at line 22 of file [config.h](#).

6.2.2.4 oled_brightness

```
uint8_t SystemConfig::oled_brightness
```

Luminosità display OLED (range: 0-255)

Definition at line 20 of file [config.h](#).

The documentation for this struct was generated from the following file:

- [firm/include/core/config.h](#)

6.3 Task Struct Reference

Struttura dati che definisce un task periodico.

```
#include <scheduler.h>
```

Data Fields

- [TaskCallback](#) `callback`
- `uint32_t` `interval`
- `uint32_t` `last_run`

6.3.1 Detailed Description

Struttura dati che definisce un task periodico.
Definition at line 23 of file [scheduler.h](#).

6.3.2 Field Documentation

6.3.2.1 callback

[TaskCallback](#) `Task::callback`
Puntatore alla funzione da eseguire.
Definition at line 24 of file [scheduler.h](#).

6.3.2.2 interval

`uint32_t` `Task::interval`
Intervallo di esecuzione in millisecondi.
Definition at line 25 of file [scheduler.h](#).

6.3.2.3 last_run

`uint32_t` `Task::last_run`
Timestamp dell'ultima esecuzione (ms).
Definition at line 26 of file [scheduler.h](#).
The documentation for this struct was generated from the following file:

- `firm/include/util/scheduler.h`

6.4 Vector3D_t Struct Reference

Struttura dati per vettori inerziali (Accelerazione/Giroscopio).
`#include <sensors_i2c.h>`

Data Fields

- `float` `x`
- `float` `y`
- `float` `z`
- `uint32_t` `timestamp`

6.4.1 Detailed Description

Struttura dati per vettori inerziali (Accelerazione/Giroscopio).
Utilizza float per rappresentare i valori in unità fisiche (g o m/s²).
Definition at line 19 of file [sensors_i2c.h](#).

6.4.2 Field Documentation

6.4.2.1 timestamp

`uint32_t` `Vector3D_t::timestamp`
Tempo di acquisizione (ms) per calcoli PID/Integrazione.
Definition at line 23 of file [sensors_i2c.h](#).

6.4.2.2 x

```
float Vector3D_t::x
```

Asse X.

Definition at line 20 of file [sensors_i2c.h](#).

6.4.2.3 y

```
float Vector3D_t::y
```

Asse Y.

Definition at line 21 of file [sensors_i2c.h](#).

6.4.2.4 z

```
float Vector3D_t::z
```

Asse Z.

Definition at line 22 of file [sensors_i2c.h](#).

The documentation for this struct was generated from the following file:

- [firm/include/drivers/sensors_i2c.h](#)

6.5 view_interface_t Struct Reference

Struttura che definisce l'interfaccia funzionale di una schermata (View).

```
#include <view_definitions.h>
```

Data Fields

- void(* [on_enter](#))(void)
Chiamata una sola volta all'attivazione della view (es. reset stato).
- void(* [on_update](#))(const void *data)
Rendering grafico e logica di aggiornamento.
- void(* [on_input](#))(button_t btn, button_state_t state)
Gestore degli eventi di input inviati dall'orchestratore.
- void(* [on_exit](#))(void)
Chiamata prima di disattivare la view corrente per passare a un'altra.

6.5.1 Detailed Description

Struttura che definisce l'interfaccia funzionale di una schermata (View).

Implementa il pattern 'Interface' in C tramite puntatori a funzione. Ogni vista deve esporre queste quattro funzioni per il lifecycle.

Definition at line 21 of file [view_definitions.h](#).

6.5.2 Field Documentation

6.5.2.1 on_enter

```
void(* view_interface_t::on_enter) (void)
```

Chiamata una sola volta all'attivazione della view (es. reset stato).

Definition at line 23 of file [view_definitions.h](#).

6.5.2.2 on_exit

```
void(* view_interface_t::on_exit) (void)
```

Chiamata prima di disattivare la view corrente per passare a un'altra.

Definition at line 37 of file [view_definitions.h](#).

6.5.2.3 on_input

```
void(* view_interface_t::on_input) (button_t btn, button_state_t state)
```

Gestore degli eventi di input inviati dall'orchestratore.

*

Parameters

<i>btn</i>	Il tasto fisico (es. BTN_UP).
<i>state</i>	Lo stato (es. BTN_RELEASED).

Definition at line 34 of file [view_definitions.h](#).

6.5.2.4 on_update

```
void(* view_interface_t::on_update) (const void *data)
```

Rendering grafico e logica di aggiornamento.

*

Parameters

<i>data</i>	Puntatore al pacchetto di sistema (SystemDataPacket*).
-------------	--

Definition at line 28 of file [view_definitions.h](#).

The documentation for this struct was generated from the following file:

- [firm/include/ui/view_definitions.h](#)

6.6 WeatherDataPacket Struct Reference

Pacchetto contenente le informazioni meteo correnti.

```
#include <messages.h>
```

Data Fields

- char [city](#) [24]
- float [temp_ext](#)
- float [wind_speed](#)
- int [humidity](#)
- int [weather_code](#)
- bool [valid](#)

6.6.1 Detailed Description

Pacchetto contenente le informazioni meteo correnti.

Definition at line 30 of file [messages.h](#).

6.6.2 Field Documentation

6.6.2.1 city

```
char WeatherDataPacket::city[24]
```

Nome della città (max 23 caratteri + terminatore).

Definition at line 31 of file [messages.h](#).

6.6.2.2 humidity

`int WeatherDataPacket::humidity`
Umidità relativa in %.
Definition at line 34 of file [messages.h](#).

6.6.2.3 temp_ext

`float WeatherDataPacket::temp_ext`
Temperatura esterna in °C.
Definition at line 32 of file [messages.h](#).

6.6.2.4 valid

`bool WeatherDataPacket::valid`
Flag di validità dei dati meteo.
Definition at line 36 of file [messages.h](#).

6.6.2.5 weather_code

`int WeatherDataPacket::weather_code`
Codice meteo (es. OpenMeteo).
Definition at line 35 of file [messages.h](#).

6.6.2.6 wind_speed

`float WeatherDataPacket::wind_speed`
Velocità del vento in m/s o km/h.
Definition at line 33 of file [messages.h](#).

The documentation for this struct was generated from the following file:

- [firm/include/core/messages.h](#)

Chapter 7

File Documentation

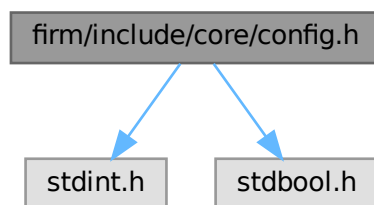
7.1 `firm/include/core/config.h` File Reference

Gestione delle configurazioni di sistema persistenti.

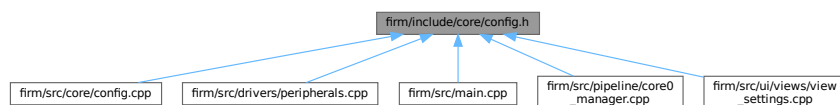
```
#include <stdint.h>
```

```
#include <stdbool.h>
```

Include dependency graph for `config.h`:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [SystemConfig](#)

Parametri di configurazione globale del dispositivo.

Functions

- void [config_init](#) (void)

Inizializza il modulo di configurazione.

- void [config_save](#) (void)

Salva la configurazione corrente nella memoria flash.

Variables

- [SystemConfig global_cfg](#)

Istanza globale della configurazione di sistema.

7.1.1 Detailed Description

Gestione delle configurazioni di sistema persistenti.

- Questo modulo definisce la struttura [SystemConfig](#) che contiene i parametri operativi del dispositivo e fornisce le interfacce per inizializzare e salvare tali configurazioni nella memoria flash.

Definition in file [config.h](#).

7.1.2 Function Documentation

7.1.2.1 config_init()

```
void config_init (
    void )
```

Inizializza il modulo di configurazione.

- Carica i parametri dalla memoria flash (LittleFS) nella variabile `global_cfg`. Se il file di configurazione non esiste, ripristina i valori di default.

Definition at line 5 of file [config.cpp](#).

7.1.2.2 config_save()

```
void config_save (
    void )
```

Salva la configurazione corrente nella memoria flash.

- Serializza la struct `global_cfg` nel file di configurazione persistente. Deve essere chiamata ogni volta che viene modificato un parametro utente.

7.1.3 Variable Documentation

7.1.3.1 global_cfg

```
SystemConfig global_cfg [extern]
```

Istanza globale della configurazione di sistema.

- Questa variabile mantiene lo stato corrente dei settaggi. Viene inizializzata durante il boot del sistema.

Definition at line 3 of file [config.cpp](#).

7.2 config.h

[Go to the documentation of this file.](#)

```
00001
00009 #ifndef CONFIG_H
00010 #define CONFIG_H
00011
00012 #include <stdint.h>
00013 #include <stdbool.h>
00014
00019 typedef struct {
00020     uint8_t    oled_brightness;
00021     bool       buzzer_enabled;
00022     bool       leds_enabled;
00023     uint8_t    gps_refresh_rate;
00024 } SystemConfig;
00025
00031 extern SystemConfig global_cfg;
00032
```

```

00038 void config_init(void);
00039
00045 void config_save(void);
00046
00047 #endif // CONFIG_H

```

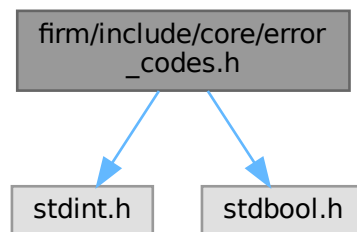
7.3 firm/include/core/error_codes.h File Reference

Definizioni degli stati di errore e diagnostica di sistema.

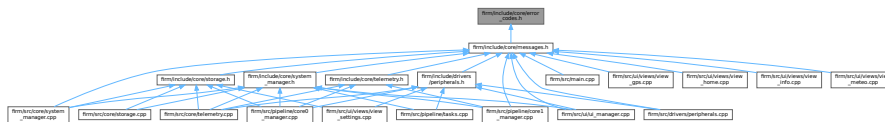
```
#include <stdint.h>
```

```
#include <stdbool.h>
```

Include dependency graph for error_codes.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [error_report_t](#)

Struttura dati per il report di un errore.

Enumerations

- enum [error_category_t](#) {
[ERR_CAT_NONE](#) = 0x00 , [ERR_CAT_SYS](#) = 0x01 , [ERR_CAT_HW](#) = 0x02 , [ERR_CAT_SENSORS](#) = 0x03
, [ERR_CAT_POWER](#) = 0x04 , [ERR_CAT_STORAGE](#) = 0x05 }
Categorie principali di errore.
- enum [error_code_t](#) {
[SUCCESS](#) = 0 , [ERR_SYS_IPC_TIMEOUT](#) = 0x10 , [ERR_SYS_WATCHDOG](#) = 0x11 , [ERR_SYS_OUT_OF_MEMORY](#) = 0x12 ,
[ERR_HW_I2C_BUS_LOCK](#) = 0x20 , [ERR_HW_DISPLAY_LOST](#) = 0x21 , [ERR_HW_UART_OVERRUN](#) = 0x22 , [ERR_SENS_GPS_NO_DATA](#) = 0x30 ,
[ERR_SENS_IMU_NOT_CAL](#) = 0x31 , [ERR_SENS_GPS_BAD_FIX](#) = 0x32 , [ERR_PWR_BATT_LOW](#) = 0x40 , [ERR_PWR_BATT_CRITICAL](#) = 0x41 ,
[ERR_PWR_USB_OVERVOLT](#) = 0x42 , [ERR_STG_SD_NOT_FOUND](#) = 0x50 , [ERR_STG_WRITE_FAIL](#) = 0x51 , [ERR_STG_FS_CORRUPT](#) = 0x52 }

Codici errore specifici.

7.3.1 Detailed Description

Definizioni degli stati di errore e diagnostica di sistema.

Fornisce un linguaggio comune per tutti i moduli (Core 0 e Core 1) per segnalare anomalie hardware o logiche.

•

Definition in file [error_codes.h](#).

7.4 error_codes.h

[Go to the documentation of this file.](#)

```
00001
00010 #ifndef ERROR_CODES_H
00011 #define ERROR_CODES_H
00012
00013 #include <stdint.h>
00014 #include <stdbool.h>
00015
00020 typedef enum {
00021     ERR_CAT_NONE      = 0x00,
00022     ERR_CAT_SYS       = 0x01,
00023     ERR_CAT_HW        = 0x02,
00024     ERR_CAT_SENSORS   = 0x03,
00025     ERR_CAT_POWER     = 0x04,
00026     ERR_CAT_STORAGE   = 0x05
00027 } error_category_t;
00028
00034 typedef enum {
00035     SUCCESS = 0,
00037     // --- Sistema (0x10-0x1F) ---
00038     ERR_SYS_IPC_TIMEOUT      = 0x10,
00039     ERR_SYS_WATCHDOG        = 0x11,
00040     ERR_SYS_OUT_OF_MEMORY   = 0x12,
00042     // --- Hardware (0x20-0x2F) ---
00043     ERR_HW_I2C_BUS_LOCK     = 0x20,
00044     ERR_HW_DISPLAY_LOST     = 0x21,
00045     ERR_HW_UART_OVERRUN     = 0x22,
00047     // --- Sensori (0x30-0x3F) ---
00048     ERR_SENS_GPS_NO_DATA    = 0x30,
00049     ERR_SENS_IMU_NOT_CAL    = 0x31,
00050     ERR_SENS_GPS_BAD_FIX    = 0x32,
00052     // --- Power (0x40-0x4F) ---
00053     ERR_PWR_BATT_LOW        = 0x40,
00054     ERR_PWR_BATT_CRITICAL   = 0x41,
00055     ERR_PWR_USB_OVERVOLT    = 0x42,
00057     // --- Storage (0x50-0x5F) ---
00058     ERR_STG_SD_NOT_FOUND    = 0x50,
00059     ERR_STG_WRITE_FAIL      = 0x51,
00060     ERR_STG_FS_CORRUPT      = 0x52
00061 } error_code_t;
00062
00067 typedef struct {
00068     error_category_t category;
00069     error_code_t code;
00070     uint32_t timestamp;
00071     bool is_critical;
00072 } error_report_t;
00073
// fine del gruppo ErrorHandling 00075
00076 #endif /* ERROR_CODES_H */
```

7.5 firm/include/core/messages.h File Reference

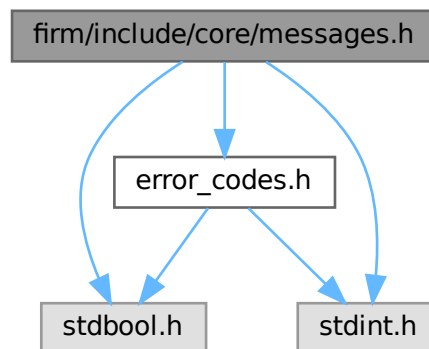
Definizione delle strutture dati per il protocollo di comunicazione.

```
#include <stdbool.h>
```

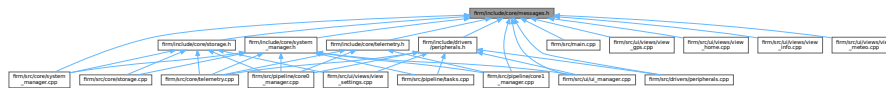
```
#include <stdint.h>
```

```
#include "error_codes.h"
```

Include dependency graph for messages.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [WeatherDataPacket](#)

Pacchetto contenente le informazioni meteo correnti.

Enumerations

- enum [GpsStatus_t](#) { [GPS_STATUS_IDLE](#) = 0 , [GPS_STATUS_NO_FIX](#) , [GPS_STATUS_FIX_2D](#) , [GPS_STATUS_FIX_3D](#) }

Enumerazione degli stati operativi del modulo GPS.

Functions

- struct [__attribute__\(\(packed\)\)](#)

Pacchetto principale di telemetria di sistema.

Variables

- [SystemDataPacket](#)

7.5.1 Detailed Description

Definizione delle strutture dati per il protocollo di comunicazione.

Questo file contiene i pacchetti di telemetria e stato del sistema. Le strutture sono ottimizzate per essere trasmesse via bus seriale o IPC.

Definition in file [messages.h](#).

7.6 messages.h

[Go to the documentation of this file.](#)

```

00001
00010 #ifndef MESSAGES_H
00011 #define MESSAGES_H
00012
00013 #include <stdbool.h>
00014 #include <stdint.h>
00015 #include "error_codes.h"
00016
00020 typedef enum {
00021     GPS_STATUS_IDLE = 0,
00022     GPS_STATUS_NO_FIX,
00023     GPS_STATUS_FIX_2D,
00024     GPS_STATUS_FIX_3D
00025 } GpsStatus_t;
00026
00030 typedef struct {
00031     char city[24];
00032     float temp_ext;
00033     float wind_speed;
00034     int humidity;
00035     int weather_code;
00036     bool valid;
00037 } WeatherDataPacket;
00038
00044 typedef struct __attribute__((packed)) {
00045     // --- Navigazione & GPS ---
00046     double latitude;
00047     double longitude;
00048     float altitude_m;
00049     float speed_ms;
00050     uint8_t satellites;
00051     uint8_t gps_status;
00053     // --- Assetto ---
00054     float pitch;
00055     float roll;
00056     float yaw;
00058     // --- Salute Hardware ---
00059     float battery_v;
00060     float temp_c;
00061     uint32_t uptime_s;
00063     // --- Dati Meteo ---
00064     WeatherDataPacket weather;
00066     // --- Diagnostica ---
00067     error_report_t last_error;
00072     struct {
00073         uint8_t is_armed : 1;
00074         uint8_t is_logging : 1;
00075         uint8_t wifi_active : 1;
00076         uint8_t error_active : 1;
00077     } flags;
00078
00079 } SystemDataPacket;
00080
// fine del gruppo ProtocolDef 00082
00083 #endif // MESSAGES_H

```

7.7 firm/include/core/power_manager.h File Reference

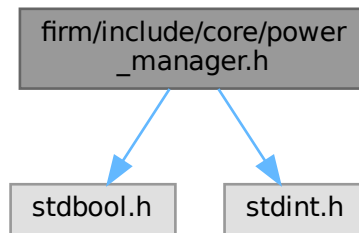
Gestore agnostico dell'alimentazione e dei carichi hardware.

```

#include <stdbool.h>
#include <stdint.h>

```

Include dependency graph for power_manager.h:



Enumerations

- enum `power_mode_t` { `PWR_MODE_PERFORMANCE` = 0 , `PWR_MODE_BALANCED` , `PWR_MODE_LOW_POWER` , `PWR_MODE_CRITICAL` }

Profili energetici del sistema.

Functions

- void `power_manager_init` (void)
Inizializza i controller di potenza (Buck, Enable pins).
- void `power_manager_update` (void)
Task di monitoraggio energetico.

Controllo Moduli Radio

- void `power_gps_set_state` (bool enable)
Abilita/Disabilita l'alimentazione del modulo GPS (SAM-M8Q).
- void `power_gps_reset` (void)
Esegue un reset hardware del modulo GPS.
- void `power_wifi_set_state` (bool enable)
Abilita/Disabilita l'alimentazione del modulo WiFi (ESP-01).

Gestione Carichi di Potenza

- void `power_usb_output_enable` (bool enable)
Abilita l'uscita di alimentazione per periferiche USB esterne.
- void `power_main_buck_enable` (bool enable)
Controllo del convertitore Buck principale della board.

Telemetria e Diagnostica

- float `power_get_battery_voltage` (void)
Restituisce la tensione della batteria filtrata.
- int8_t `power_get_battery_level` (void)
Restituisce la percentuale di carica residua della LiPo.
- bool `power_source_is_usb` (void)
Verifica se il sistema è alimentato via cavo USB.
- void `power_set_mode` (`power_mode_t` mode)
Imposta manualmente il profilo energetico.

7.7.1 Detailed Description

Gestore agnostico dell'alimentazione e dei carichi hardware.

Centralizza il controllo dei moduli (GPS, WiFi, USB) e implementa logiche di protezione per la batteria LiPo.

Definition in file [power_manager.h](#).

7.7.2 Enumeration Type Documentation

7.7.2.1 power_mode_t

enum [power_mode_t](#)

Profili energetici del sistema.

*

Definiscono il comportamento dei carichi in base alla missione.

Enumerator

PWR_MODE_PERFORMANCE	Massima potenza: tutti i moduli attivi.
PWR_MODE_BALANCED	Risparmio dinamico: WiFi on-demand.
PWR_MODE_LOW_POWER	Solo telemetria essenziale e GPS.
PWR_MODE_CRITICAL	Emergenza: spegnimento carichi non vitali (Failsafe).

Definition at line 17 of file [power_manager.h](#).

7.7.3 Function Documentation

7.7.3.1 power_get_battery_level()

```
int8_t power_get_battery_level (
    void )
```

Restituisce la percentuale di carica residua della LiPo.

Returns

int8_t Percentuale (0-100), o -1 se non rilevata.

7.7.3.2 power_get_battery_voltage()

```
float power_get_battery_voltage (
    void )
```

Restituisce la tensione della batteria filtrata.

Returns

float Tensione in Volt (V).

7.7.3.3 power_manager_init()

```
void power_manager_init (
    void )
```

Inizializza i controller di potenza (Buck, Enable pins).

Configura i GPIO di uscita e verifica la tensione di avvio.

7.7.3.4 power_manager_update()

```
void power_manager_update (
    void )
```

[Task](#) di monitoraggio energetico.

Da chiamare periodicamente per gestire le transizioni di stato basate sulla tensione della batteria.

7.8 power_manager.h

[Go to the documentation of this file.](#)

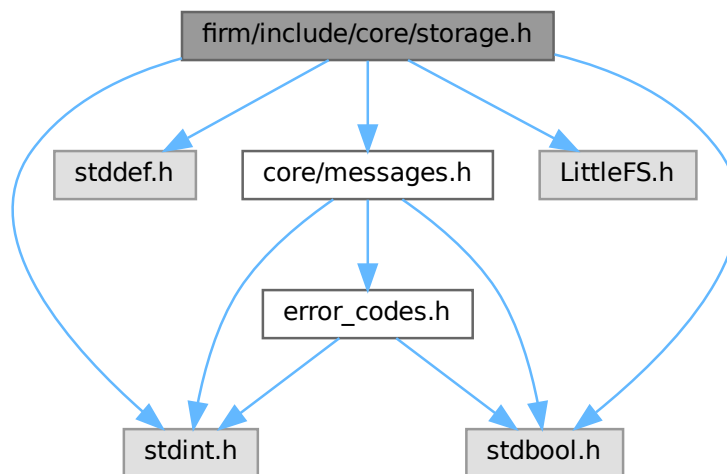
```
00001
00008 #ifndef POWER_MANAGER_H
00009 #define POWER_MANAGER_H
00010
00011 #include <stdbool.h>
00012 #include <stdint.h>
00013
00017 typedef enum {
00018     PWR_MODE_PERFORMANCE = 0,
00019     PWR_MODE_BALANCED,
00020     PWR_MODE_LOW_POWER,
00021     PWR_MODE_CRITICAL
00022 } power_mode_t;
00023
00028 void power_manager_init(void);
00029
00035 void power_manager_update(void);
00036
00041 void power_gps_set_state(bool enable);
00042
00044 void power_gps_reset(void);
00045
00047 void power_wifi_set_state(bool enable);
00048
00055 void power_usb_output_enable(bool enable);
00056
00058 void power_main_buck_enable(bool enable);
00059
00067 float power_get_battery_voltage(void);
00068
00071 int8_t power_get_battery_level(void);
00072
00074 bool power_source_is_usb(void);
00075
00077 void power_set_mode(power_mode_t mode);
00078
00081 #endif /* POWER_MANAGER_H */
```

7.9 firm/include/core/storage.h File Reference

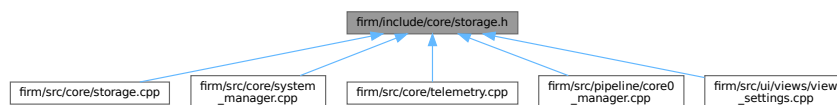
Interfaccia per la gestione della persistenza dati su file system (LittleFS).

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include <LittleFS.h>
#include "core/messages.h"
```

Include dependency graph for storage.h:



This graph shows which files directly or indirectly include this file:



Functions

- bool `storage_init` (void)
Inizializza il file system LittleFS.
- void `storage_log_append` (const char *entry)
Aggiunge una riga di log al file delle telemetrie.
- void `storage_sync` (void)
Sincronizza i dati in sospeso con il supporto fisico.
- bool `storage_read_line_with_tag` (const char *filename, const char *tag, char *buffer, size_t len)
Cerca una riga contenente uno specifico tag in un file.
- bool `storage_save_params` (const void *data, size_t size)
Salva i parametri di configurazione in formato binario.
- bool `storage_load_params` (void *data, size_t size)
Carica i parametri di configurazione dalla flash.
- bool `storage_is_busy_by_usb` (void)
Verifica se lo storage è impegnato dal protocollo USB (MTP/MSC).
- void `storage_log_gps_fix` (float lat, float lon)
Logga una coordinata GPS nel file di log dedicato.
- bool `storage_read_last_gps` (float *lat, float *lon)
Recupera l'ultimo fix GPS salvato in memoria.
- void `debug_dump_gps_log` (void)

Esegue un dump del file GPS sulla seriale per debug.

- bool `storage_format_all` (void)

Formatta l'intera partizione Flash LittleFS.

- bool `storage_save_weather` (const `WeatherDataPacket` *w)
- bool `storage_load_weather` (`WeatherDataPacket` *w)

7.9.1 Detailed Description

Interfaccia per la gestione della persistenza dati su file system (LittleFS).

Gestisce il salvataggio dei parametri di configurazione, il logging delle telemetrie e l'archiviazione dei fix GPS.

Definition in file [storage.h](#).

7.10 storage.h

[Go to the documentation of this file.](#)

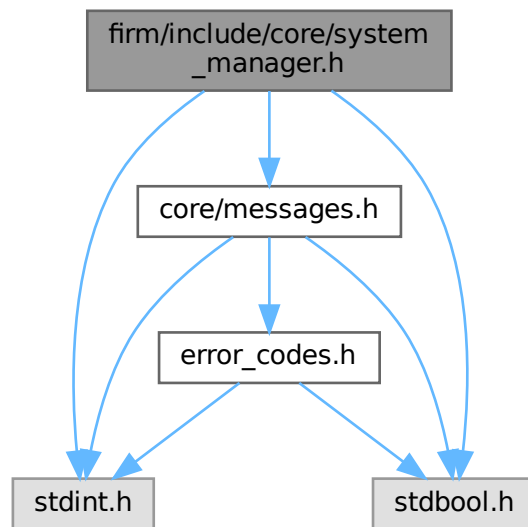
```
00001
00010 #ifndef STORAGE_H
00011 #define STORAGE_H
00012
00013 #include <stdbool.h>
00014 #include <stddef.h>
00015 #include <stdint.h>
00016 #include <LittleFS.h>
00017 #include "core/messages.h" // <-- FONDAMENTALE per far conoscere WeatherDataPacket
00018
00023 bool storage_init(void);
00024
00029 void storage_log_append(const char* entry);
00030
00034 void storage_sync(void);
00035
00044 bool storage_read_line_with_tag(const char* filename, const char* tag, char* buffer, size_t len);
00045
00052 bool storage_save_params(const void* data, size_t size);
00053
00060 bool storage_load_params(void* data, size_t size);
00061
00066 bool storage_is_busy_by_usb(void);
00067
00073 void storage_log_gps_fix(float lat, float lon);
00074
00081 bool storage_read_last_gps(float* lat, float* lon);
00082
00086 void debug_dump_gps_log(void);
00087
00093 bool storage_format_all(void);
00094
00095 bool storage_save_weather(const WeatherDataPacket* w);
00096 bool storage_load_weather(WeatherDataPacket* w);
00097
// fine del gruppo StorageManager 00099
00100 #endif // STORAGE_H
```

7.11 firm/include/core/system_manager.h File Reference

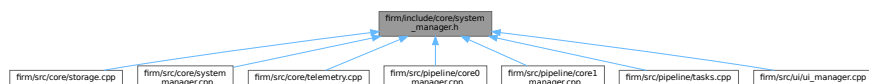
Gestione del middleware di sistema e comunicazione IPC tra core.

```
#include <stdbool.h>
#include <stdint.h>
#include "core/messages.h"
```

Include dependency graph for `system_manager.h`:



This graph shows which files directly or indirectly include this file:



Functions

- void `sys_manager_init` (void)
Inizializza i meccanismi di comunicazione inter-core.
- bool `sys_manager_send_data` (SystemDataPacket *packet)
Invia un pacchetto di telemetria al core partner.
- bool `sys_manager_receive_data` (SystemDataPacket *buffer)
Riceve un pacchetto di telemetria dal core partner.
- void `sys_manager_report_error` (error_category_t cat, error_code_t code, bool critical)
Notifica un evento di errore al sistema di diagnostica.
- bool `sys_is_core1_ready` (void)
Verifica se il Core 1 è pronto a ricevere comandi o dati.
- void `sys_set_core1_ready` (bool ready)
Imposta lo stato di disponibilità del Core 1.
- void `sys_manager_handle_serial` (void)
Gestisce il parsing dei comandi in arrivo dalla porta seriale.
- bool `sys_manager_update_weather` (const WeatherDataPacket &weather)
Aggiorna localmente i dati meteo di sistema.
- bool `sys_manager_update_gps` (float lat, float lon, bool fix, uint8_t sats)
Aggiorna le coordinate GPS correnti.

- void `sys_manager_clear_error` (void)
Resetta lo stato di errore globale, rimuovendo notifiche pendenti.
- bool `storage_save_weather` (const `WeatherDataPacket` *w)
- bool `storage_load_weather` (`WeatherDataPacket` *w)

7.11.1 Detailed Description

Gestione del middleware di sistema e comunicazione IPC tra core.

Coordina lo scambio dei pacchetti di telemetria, la gestione dei report di errore e lo stato di sincronizzazione tra il Core principale e il Core di elaborazione/IO.

Definition in file `system_manager.h`.

7.12 system_manager.h

[Go to the documentation of this file.](#)

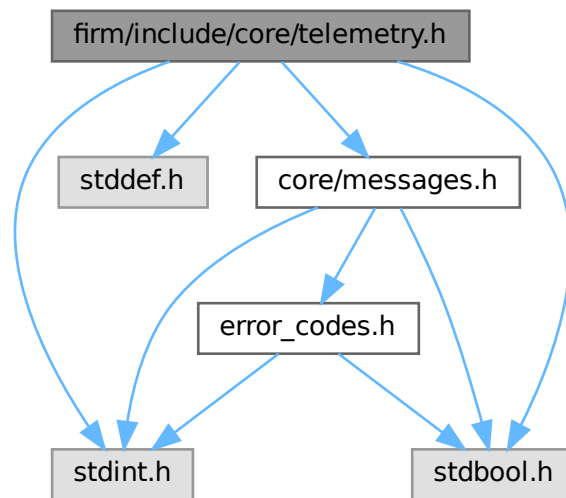
```
00001
00011 #ifndef SYSTEM_MANAGER_H
00012 #define SYSTEM_MANAGER_H
00013
00014 #include <stdbool.h>
00015 #include <stdint.h>
00016 #include "core/messages.h"
00017
00021 void sys_manager_init(void);
00022
00028 bool sys_manager_send_data(SystemDataPacket* packet);
00029
00035 bool sys_manager_receive_data(SystemDataPacket* buffer);
00036
00043 void sys_manager_report_error(error_category_t cat, error_code_t code, bool critical);
00044
00049 bool sys_is_core1_ready(void);
00050
00055 void sys_set_core1_ready(bool ready);
00056
00060 void sys_manager_handle_serial(void);
00061
00067 bool sys_manager_update_weather(const WeatherDataPacket& weather);
00068
00077 bool sys_manager_update_gps(float lat, float lon, bool fix, uint8_t sats);
00078
00082 void sys_manager_clear_error(void);
00083
00084 bool storage_save_weather(const WeatherDataPacket* w);
00085 bool storage_load_weather(WeatherDataPacket* w);
00086
00088 // fine del gruppo SystemManager
00089 #endif /* SYSTEM_MANAGER_H */
```

7.13 firm/include/core/telemetry.h File Reference

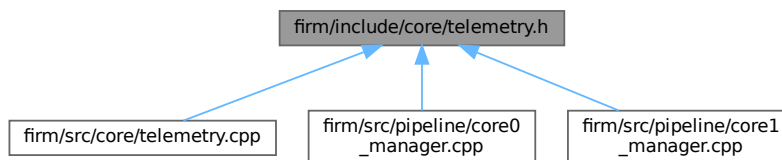
Gestione telemetria con accesso atomico per architetture multi-core.

```
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "core/messages.h"
```

Include dependency graph for `telemetry.h`:



This graph shows which files directly or indirectly include this file:



Functions

- void `telemetry_init` (void)
Inizializza i buffer di telemetria e i meccanismi di sincronizzazione.
- void `telemetry_update` (void)
Aggiorna il buffer di telemetria interno con i dati di sistema correnti.
- bool `telemetry_get_frame` (SystemDataPacket *dest)
Copia il frame di telemetria corrente nella destinazione fornita.
- bool `telemetry_is_healthy` (void)
Verifica lo stato di salute dei dati di telemetria.

7.13.1 Detailed Description

Gestione telemetria con accesso atomico per architetture multi-core.

Fornisce un'interfaccia sicura per la lettura e scrittura dei dati di sistema, garantendo l'integrità dei dati tramite meccanismi di accesso sincronizzato (mutex o lock atomici).

Definition in file [telemetry.h](#).

7.14 telemetry.h

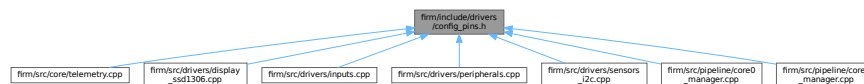
[Go to the documentation of this file.](#)

```
00001
00011 #ifndef TELEMETRY_H
00012 #define TELEMETRY_H
00013
00014 #include <stdbool.h>
00015 #include <stddef.h>
00016 #include <stdint.h>
00017 #include "core/messages.h"
00018
00022 void telemetry_init(void);
00023
00029 void telemetry_update(void);
00030
00038 bool telemetry_get_frame(SystemDataPacket* dest);
00039
00044 bool telemetry_is_healthy(void);
00045
00051 static void check_and_log_gps(float lat, float lon);
00052
// fine del gruppo Telemetry 00054
00055 #endif // TELEMETRY_H
```

7.15 firm/include/drivers/config_pins.h File Reference

Mappatura hardware GLOBALE e REALE per RP2040 (PPSE Lab Edition).

This graph shows which files directly or indirectly include this file:



Macros

Bus di Comunicazione (I2C & UART)

- #define I2C0_SDA_PIN 0
- #define I2C0_SCL_PIN 1
- #define GPS_TX_PIN 4
- *UART1: Modulo GPS SAM-M8Q.*
- #define GPS_RX_PIN 5
- #define GPSBAUD 9600
- #define ESP_TX_PIN 16
- *UART0: Modulo WiFi ESP-01.*
- #define ESP_RX_PIN 17

Controllo Alimentazione e Reset Moduli

- #define SAM_EN_PIN 18
- #define SAM_RST_PIN 12
- #define ESP_EN_PIN 3
- #define BUCK_5V_EN_PIN 11
- #define OUTPUT_USB_EN_PIN 27

Input Utente (Pulsanti e Interruttori)

- #define BTN_UP_PIN 22
- #define BTN_DOWN_PIN 23
- #define BTN_RIGHT_PIN 24
- #define BTN_LEFT_PIN 25
- #define BTN_OK_PIN BTN_RIGHT_PIN
- #define BTN_BACK_PIN BTN_LEFT_PIN

Feedback, Stato e Linee di Interrupt

- `#define NUM_LEDS 8`
- `#define RGB_LED_PIN 13`
- `#define BUZZER_PIN 21`
- `#define LED_ALIVE_PIN 20`
- `#define ACC_INT1_PIN 14`
- `#define ACC_INT2_PIN 15`
- `#define SAM_PPS_PIN 19`

Attuatori PWM (Servo)

- `#define SERVO_1_PIN 8`
- `#define SERVO_2_PIN 9`
- `#define SERVO_3_PIN 7`

Canali Analogici (ADC Hardware)

- `#define TEMP_SENS_PIN 26`
- `#define ANALOG_BATT_PIN 28`

7.15.1 Detailed Description

Mappatura hardware GLOBALE e REALE per RP2040 (PPSE Lab Edition).

Definizione univoca dei pin per il core RP2040. Include interfacce di comunicazione, controllo alimentazione e periferiche di feedback.

Definition in file [config_pins.h](#).

7.16 config_pins.h

[Go to the documentation of this file.](#)

```
00001
00010 #ifndef CONFIG_PINS_H
00011 #define CONFIG_PINS_H
00012
00017 #define I2C0_SDA_PIN      0
00018 #define I2C0_SCL_PIN      1
00021 #define GPS_TX_PIN        4
00022 #define GPS_RX_PIN        5
00023 #define GPSBAUD            9600
00026 #define ESP_TX_PIN        16
00027 #define ESP_RX_PIN        17
00034 #define SAM_EN_PIN        18
00035 #define SAM_RST_PIN        12
00036 #define ESP_EN_PIN        3
00037 #define BUCK_5V_EN_PIN    11
00038 #define OUTPUT_USB_EN_PIN 27
00045 #define BTN_UP_PIN        22
00046 #define BTN_DOWN_PIN      23
00047 #define BTN_RIGHT_PIN     24
00048 #define BTN_LEFT_PIN      25
00049 #define BTN_OK_PIN         BTN_RIGHT_PIN
00050 #define BTN_BACK_PIN       BTN_LEFT_PIN
00057 #define NUM_LEDS          8
00058 #define RGB_LED_PIN        13
00059 #define BUZZER_PIN         21
00060 #define LED_ALIVE_PIN      20
00061 #define ACC_INT1_PIN       14
00062 #define ACC_INT2_PIN       15
00063 #define SAM_PPS_PIN        19
00070 #define SERVO_1_PIN        8
00071 #define SERVO_2_PIN        9
00072 #define SERVO_3_PIN        7
00079 #define TEMP_SENS_PIN      26
00080 #define ANALOG_BATT_PIN     28
// fine del gruppo HardwarePins 00084
00085 #endif /* CONFIG_PINS_H */
```

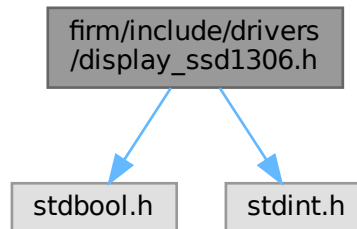

7.17 firm/include/drivers/display_ssd1306.h File Reference

Driver di basso livello agnostico per il controllo dell'OLED SSD1306.

```
#include <stdbool.h>
```

```
#include <stdint.h>
```

Include dependency graph for display_ssd1306.h:



This graph shows which files directly or indirectly include this file:



Macros

Costanti Fisiche Pannello

-
- `#define SCREEN_WIDTH 128`
- `#define SCREEN_HEIGHT 64`
- `#define SCREEN_ADDRESS 0x3C`

Functions

- `bool display_hw_init (void)`
Inizializza l'hardware I2C e il controller OLED.
- `bool display_is_alive (void)`
Verifica se il display è fisicamente connesso al bus I2C.
- `void display_clear (void)`
Pulisce il buffer interno (RAM) del display.
- `void display_show (void)`
Invia il buffer corrente al display fisico (Refresh).
- `void display_set_power (bool on)`
Gestione energetica del pannello.
- `void display_set_brightness (uint8_t level)`
Regola il contrasto (luminosità) dei pixel OLED.
- `void display_draw_pixel (int16_t x, int16_t y, uint16_t color)`
Funzione primitiva di disegno agnostica.
- `void * get_display_driver (void)`
Fornisce l'accesso al driver sottostante (uso avanzato).

7.17.1 Detailed Description

Driver di basso livello agnostico per il controllo dell'OLED SSD1306.

Questa interfaccia astrae le dipendenze da Adafruit, permettendo alla logica superiore di operare indipendentemente dall'implementazione hardware.

Definition in file [display_ssd1306.h](#).

7.18 display_ssd1306.h

[Go to the documentation of this file.](#)

```
00001
00010 #ifndef DISPLAY_SSD1306_H
00011 #define DISPLAY_SSD1306_H
00012
00013 #include <stdbool.h>
00014 #include <stdint.h>
00015
00019 #define SCREEN_WIDTH 128
00020 #define SCREEN_HEIGHT 64
00021 #define SCREEN_ADDRESS 0x3C
00029 bool display_hw_init(void);
00030
00035 bool display_is_alive(void);
00036
00041 void display_clear(void);
00042
00047 void display_show(void);
00048
00053 void display_set_power(bool on);
00054
00059 void display_set_brightness(uint8_t level);
00060
00067 void display_draw_pixel(int16_t x, int16_t y, uint16_t color);
00068
00076 void* get_display_driver(void);
00077
// fine del gruppo DisplayDriver 00079
00080 #endif /* DISPLAY_SSD1306_H */
```

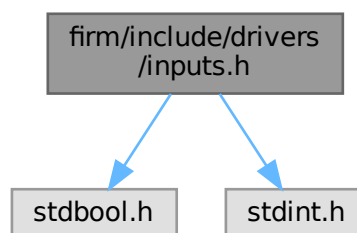
7.19 firm/include/drivers/inputs.h File Reference

Driver agnostico per la gestione degli input fisici (Pulsanti).

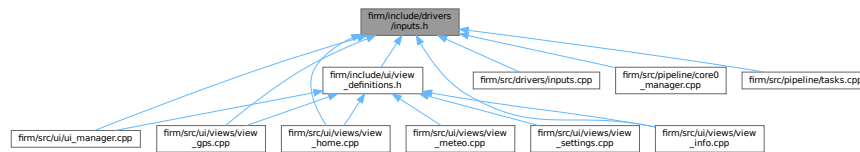
```
#include <stdbool.h>
```

```
#include <stdint.h>
```

Include dependency graph for inputs.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- typedef `ButtonId` `button_t`
Alias per compatibilità con il framework UI.

Enumerations

- enum `ButtonId` {
 `BTN_NONE` = 0 , `BTN_UP` , `BTN_DOWN` , `BTN_OK` ,
 `BTN_BACK` , `BTN_COUNT` }
Identificativi logici dei pulsanti del sistema.
- enum `button_state_t` { `BTN_PRESSED` , `BTN_RELEASED` }
Stati possibili di un evento pulsante.

Functions

- void `inputs_init` (void)
Inizializza i GPIO e le strutture di debounce per i pulsanti.
- `ButtonId` `inputs_get_last_press` (void)
Recupera l'ultimo evento di pressione (Logica a consumo).
- void `inputs_update` (void)
Task di aggiornamento periodico (Debounce).
- bool `inputs_is_pressed` (`ButtonId` btn)
Verifica lo stato fisico attuale (istantaneo) di un pulsante.

7.19.1 Detailed Description

Driver agnostico per la gestione degli input fisici (Pulsanti).

Gestisce il debounce hardware e fornisce una mappatura logica degli eventi di pressione per l'interfaccia utente.

Definition in file `inputs.h`.

7.20 inputs.h

[Go to the documentation of this file.](#)

```

00001
00010 #ifndef INPUTS_H
00011 #define INPUTS_H
00012
00013 #include <stdbool.h>
00014 #include <stdint.h>
00015
00019 typedef enum {
00020     BTN_NONE = 0,
00021     BTN_UP,
00022     BTN_DOWN,
00023     BTN_OK,
00024     BTN_BACK,
00025     BTN_COUNT
00026 } ButtonId;
00027
00030 typedef ButtonId button_t;
  
```

```

00031
00035 typedef enum {
00036     BTN_PRESSED,
00037     BTN_RELEASED
00038 } button_state_t;
00039
00043 void inputs_init(void);
00044
00051 ButtonId inputs_get_last_press(void);
00052
00059 void inputs_update(void);
00060
00066 bool inputs_is_pressed(ButtonId btn);
00067
// fine del gruppo InputDriver 00069
00070 #endif /* INPUTS_H */

```

7.21 firm/include/drivers/peripherals.h File Reference

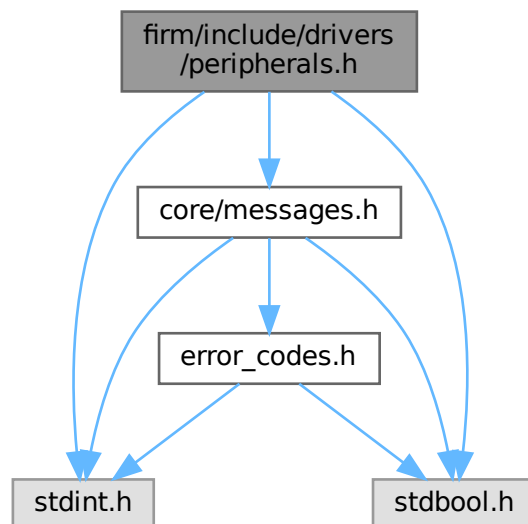
Driver agnostico unificato per feedback (LED, Buzzer) e sensori.

```
#include <stdbool.h>
```

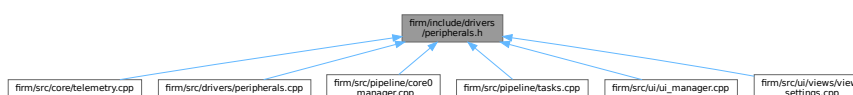
```
#include <stdint.h>
```

```
#include "core/messages.h"
```

Include dependency graph for peripherals.h:



This graph shows which files directly or indirectly include this file:



Enumerations

- enum `FeedbackType` { `FEEDBACK_SUCCESS` , `FEEDBACK_ERROR` , `FEEDBACK_NEUTRAL` , `FEEDBACK_OFF` }

Tipologie di feedback utente per azioni UI.

- enum `LedAnimState` {
`LED_ANIM_NONE` , `LED_ANIM_BOOT` , `LED_ANIM_BOOT_DONE` , `LED_ANIM_SEARCHING` ,
`LED_ANIM_FIX_OK` , `LED_ANIM_SIGNAL_LOW` }

Stati di animazione LED per sistema di feedback visuale.

Functions

Inizializzazione

- void `peripherals_init` (void)
- void `peripherals_init_leds` (void)

Controllo Fisico Base

- void `peripherals_set_leds` (bool enabled)
- void `peripherals_set_buzzer` (bool enabled)

Feedback Visivo e Acustico

- void `peripherals_update_led_fx` (`LedAnimState` state, float val)
Aggiorna l'animazione LED basata sullo stato di sistema.
- void `peripherals_auto_feedback` (const `SystemDataPacket` *data)
Esegue un feedback automatico basato sui dati di telemetria.
- void `peripherals_trigger_feedback` (`FeedbackType` type)
Attiva un feedback sonoro/visivo basato su un evento.
- void `peripherals_set_led` (uint8_t r, uint8_t g, uint8_t b)
Imposta colore RGB statico per i LED.
- void `peripherals_beep` (uint32_t freq_hz, uint32_t duration_ms)
Esegue un breve suono (beep).
- void `peripherals_play_click` (void)
Esegue il tipico suono di "click" per conferma UI.

Acquisizione Dati Ambientali

- float `peripherals_get_temperature` (void)
- float `peripherals_get_battery_voltage` (void)

7.21.1 Detailed Description

Driver agnostico unificato per feedback (LED, Buzzer) e sensori.

Fornisce un'interfaccia di alto livello per gestire le periferiche di feedback visivo e acustico, e per l'acquisizione di dati ambientali.

Definition in file [peripherals.h](#).

7.22 peripherals.h

[Go to the documentation of this file.](#)

```
00001
00010 #ifndef PERIPHERALS_H
00011 #define PERIPHERALS_H
00012
00013 #include <stdbool.h>
00014 #include <stdint.h>
00015 #include "core/messages.h"
00016
00020 typedef enum {
00021     FEEDBACK_SUCCESS,
00022     FEEDBACK_ERROR,
00023     FEEDBACK_NEUTRAL,
00024     FEEDBACK_OFF
00025 } FeedbackType;
00026
00030 typedef enum {
00031     LED_ANIM_NONE,
```

```

00032     LED_ANIM_BOOT,
00033     LED_ANIM_BOOT_DONE,
00034     LED_ANIM_SEARCHING,
00035     LED_ANIM_FIX_OK,
00036     LED_ANIM_SIGNAL_LOW
00037 } LedAnimState;
00038
00043 void peripherals_init(void);
00044 void peripherals_init_leds(void);
00051 void peripherals_set_leds(bool enabled);
00052 void peripherals_set_buzzer(bool enabled);
00065 void peripherals_update_led_fx(LedAnimState state, float val);
00066
00071 void peripherals_auto_feedback(const SystemDataPacket* data);
00072
00077 void peripherals_trigger_feedback(FeedbackType type);
00078
00082 void peripherals_set_led(uint8_t r, uint8_t g, uint8_t b);
00083
00089 void peripherals_beep(uint32_t freq_hz, uint32_t duration_ms);
00090
00094 void peripherals_play_click(void);
00101 float peripherals_get_temperature(void);
00102 float peripherals_get_battery_voltage(void);
// fine del gruppo Peripherals 00106
00107 #endif /* PERIPHERALS_H */

```

7.23 firm/include/drivers/radio_uart.h File Reference

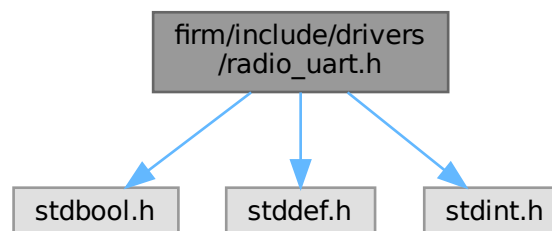
Driver agnostico per la gestione delle interfacce seriali (UART0 e UART1).

```
#include <stdbool.h>
```

```
#include <stddef.h>
```

```
#include <stdint.h>
```

Include dependency graph for radio_uart.h:



Functions

- void **radio_uart_init** (void)
Configura le periferiche UART0 e UART1 dell'RP2040.

Interfaccia GPS (UART1 - SAM-M8Q)

- bool **radio_gps_available** (void)
Verifica se ci sono byte pronti nel buffer di ricezione del GPS.
- uint8_t **radio_gps_read** (void)
Legge un singolo byte dal modulo GPS.
- void **radio_gps_send** (const uint8_t *buffer, size_t len)
Invia un comando o un pacchetto di configurazione al GPS.

Interfaccia WiFi (UART0 - ESP-01)

- bool **radio_esp_available** (void)
Verifica se ci sono dati in arrivo dal modulo WiFi.
- uint8_t **radio_esp_read** (void)
Legge un singolo byte dal modulo WiFi.
- void **radio_esp_send** (const uint8_t *buffer, size_t len)
Invia dati o comandi AT al modulo ESP.

7.23.1 Detailed Description

Driver agnostico per la gestione delle interfacce seriali (UART0 e UART1).

Astrae le comunicazioni con il modulo GPS (SAM-M8Q) e WiFi (ESP-01). Fornisce un'interfaccia a basso livello per il buffering dei dati. Repository: <https://github.com/4137314/ppse-lab-firmware>

Definition in file [radio_uart.h](#).

7.23.2 Function Documentation

7.23.2.1 radio_esp_send()

```
void radio_esp_send (
    const uint8_t * buffer,
    size_t len )
```

Invia dati o comandi AT al modulo ESP.

*

Parameters

<i>buffer</i>	Puntatore ai dati (es. stringa AT o pacchetto UDP).
<i>len</i>	Lunghezza dei dati.

7.23.2.2 radio_gps_send()

```
void radio_gps_send (
    const uint8_t * buffer,
    size_t len )
```

Invia un comando o un pacchetto di configurazione al GPS.

*

Parameters

<i>buffer</i>	Puntatore ai dati da inviare.
<i>len</i>	Lunghezza dei dati.

7.23.2.3 radio_uart_init()

```
void radio_uart_init (
    void )
```

Configura le periferiche UART0 e UART1 dell'RP2040.

Imposta i baudrate specifici: solitamente 9600 per il GPS e 115200 per l'ESP-01.

7.24 radio_uart.h

[Go to the documentation of this file.](#)

```
00001
00009 #ifndef RADIO_UART_H
00010 #define RADIO_UART_H
00011
00012 #include <stdbool.h>
```

```

00013 #include <stddef.h>
00014 #include <stdint.h>
00015
00021 void radio_uart_init(void);
00022
00027 bool radio_gps_available(void);
00028
00030 uint8_t radio_gps_read(void);
00031
00036 void radio_gps_send(const uint8_t* buffer, size_t len);
00037
00044 bool radio_esp_available(void);
00045
00047 uint8_t radio_esp_read(void);
00048
00053 void radio_esp_send(const uint8_t* buffer, size_t len);
00054
00057 #endif /* RADIO_UART_H */

```

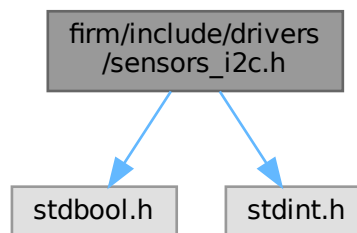
7.25 firm/include/drivers/sensors_i2c.h File Reference

Driver agnostico per il bus I2C0 e la sensoristica inerziale/ambientale.

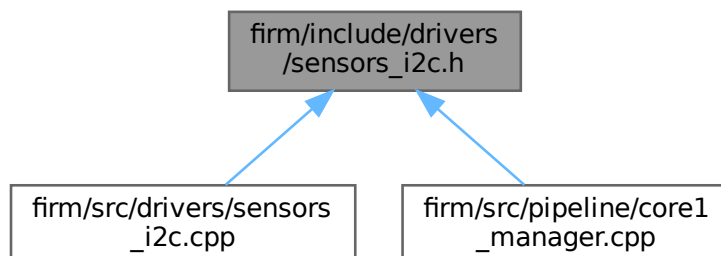
```
#include <stdbool.h>
```

```
#include <stdint.h>
```

Include dependency graph for sensors_i2c.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [Vector3D_t](#)

Struttura dati per vettori inerziali (Accelerazione/Giroscopio).

Functions

- bool `sensors_i2c_init` (void)
Inizializza il bus I2C0 e i sensori collegati.
- bool `sensors_accel_get_data` (Vector3D_t *data)
Legge i dati dall'accelerometro.
- bool `sensors_i2c_is_healthy` (void)
Esegue una scansione del bus I2C0 per diagnostica.
- void `sensors_accel_calibrate` (void)
Calibra il sensore inerziale.
- float `sensors_read_temperature_c` (void)
Legge la temperatura corrente.
- float `sensors_read_battery_v` (void)
Legge la tensione della batteria.

7.25.1 Detailed Description

Driver agnostico per il bus I2C0 e la sensoristica inerziale/ambientale.
Gestisce l'astrazione dei sensori (IMU, Termometro, ADC) e del bus di comunicazione.
Definition in file `sensors_i2c.h`.

7.26 sensors_i2c.h

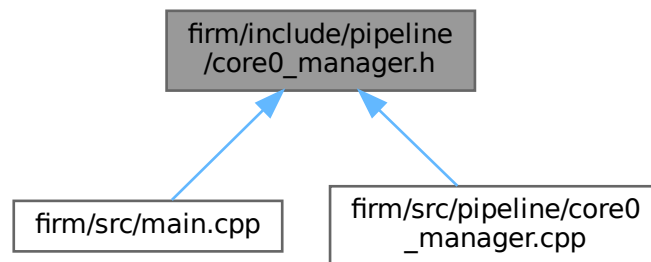
[Go to the documentation of this file.](#)

```
00001
00009 #ifndef SENSORS_I2C_H
00010 #define SENSORS_I2C_H
00011
00012 #include <stdbool.h>
00013 #include <stdint.h>
00014
00019 typedef struct {
00020     float x;
00021     float y;
00022     float z;
00023     uint32_t timestamp;
00024 } Vector3D_t;
00025
00030 bool sensors_i2c_init(void);
00031
00038 bool sensors_accel_get_data(Vector3D_t* data);
00039
00045 bool sensors_i2c_is_healthy(void);
00046
00051 void sensors_accel_calibrate(void);
00052
00057 float sensors_read_temperature_c(void);
00058
00063 float sensors_read_battery_v(void);
00064
00066 // fine del gruppo Sensors 00066
00067 #endif /* SENSORS_I2C_H */
```

7.27 firm/include/pipeline/core0_manager.h File Reference

Punto di ingresso e ciclo di controllo principale per il Core 0.

This graph shows which files directly or indirectly include this file:



Functions

- void `core0_setup` (void)
Setup iniziale del Core 0.
- void `core0_loop` (void)
Ciclo principale di esecuzione del Core 0.

7.27.1 Detailed Description

Punto di ingresso e ciclo di controllo principale per il Core 0.

Il Core 0 gestisce la logica di alto livello, la UI, la gestione dei file (LittleFS) e la comunicazione seriale.

Definition in file `core0_manager.h`.

7.28 `core0_manager.h`

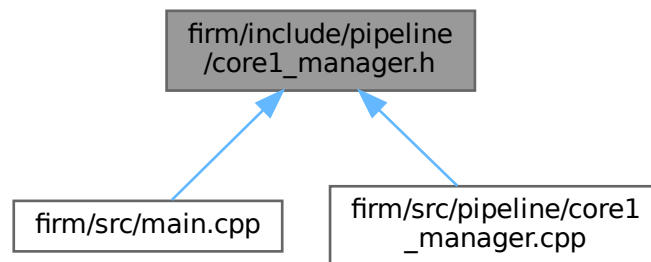
[Go to the documentation of this file.](#)

```
00001
00010 #pragma once
00011
00017 void core0_setup(void);
00018
00024 void core0_loop(void);
00025
// fine del gruppo Core0
```

7.29 `firm/include/pipeline/core1_manager.h` File Reference

Punto di ingresso e ciclo di elaborazione per il Core 1.

This graph shows which files directly or indirectly include this file:



Functions

- void `core1_setup` (void)
Setup iniziale del Core 1.
- void `core1_loop` (void)
Ciclo principale di elaborazione del Core 1.

7.29.1 Detailed Description

Punto di ingresso e ciclo di elaborazione per il Core 1.

Il Core 1 funge da "Worker" dedicato, gestendo task a bassa latenza come l'acquisizione GPS, la lettura dei sensori IMU e il calcolo dei cicli di controllo.

Definition in file `core1_manager.h`.

7.30 core1_manager.h

[Go to the documentation of this file.](#)

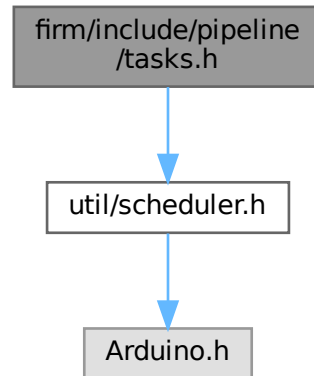
```
00001
00010 #pragma once
00011
00017 void core1_setup(void);
00018
00024 void core1_loop(void);
00025
// fine del gruppo Core1
```

7.31 firm/include/pipeline/tasks.h File Reference

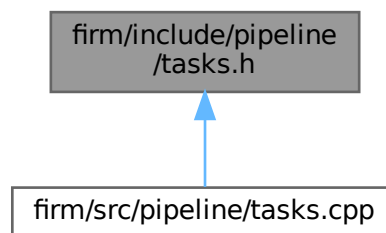
Dichiarazione dei task di sistema per lo scheduler.

```
#include "util/scheduler.h"
```

Include dependency graph for tasks.h:



This graph shows which files directly or indirectly include this file:



Variables

- [Task task_serial](#)
Task di gestione comunicazione seriale.
- [Task task_input](#)
Task di acquisizione input utente.
- [Task task_ui](#)
Task di aggiornamento dell'interfaccia utente (UI).
- [Task task_health](#)
Task di monitoraggio integrità sistema (Health Check).

7.31.1 Detailed Description

Dichiarazione dei task di sistema per lo scheduler.

Questo file espone i task globali che compongono l'architettura multi-tasking del firmware. Ogni task rappresenta un'unità di esecuzione periodica gestita dallo scheduler.

Definition in file [tasks.h](#).

7.32 tasks.h

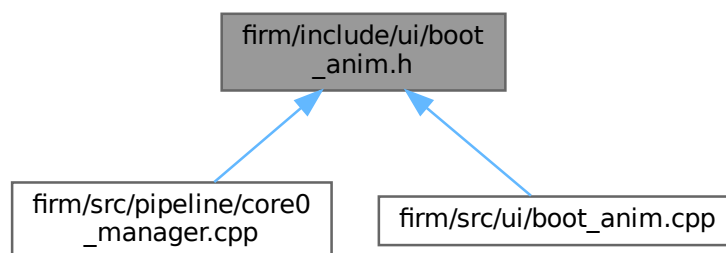
[Go to the documentation of this file.](#)

```
00001
00011 #ifndef TASKS_H
00012 #define TASKS_H
00013
00014 #include "util/scheduler.h"
00015
00020 extern Task task_serial;
00021
00026 extern Task task_input;
00027
00032 extern Task task_ui;
00033
00039 extern Task task_health;
00040
// fine del gruppo TaskManager 00042
00043 #endif // TASKS_H
```

7.33 firm/include/ui/boot_anim.h File Reference

Gestione delle sequenze grafiche di avvio (Boot Animations).

This graph shows which files directly or indirectly include this file:



Functions

- void [run_hacker_boot_sequence](#) (void(*update_callback)(void))
Esegue la sequenza di avvio "Hacker-style".

7.33.1 Detailed Description

Gestione delle sequenze grafiche di avvio (Boot Animations).

Fornisce primitive per eseguire animazioni di sistema all'accensione senza bloccare l'esecuzione dei task critici.

Definition in file [boot_anim.h](#).

7.34 boot_anim.h

[Go to the documentation of this file.](#)

```
00001
00010 #ifndef BOOT_ANIM_H
00011 #define BOOT_ANIM_H
00012
00021 void run_hacker_boot_sequence(void (*update_callback)(void));
00022
// fine del gruppo BootAnim 00024
00025 #endif // BOOT_ANIM_H
```

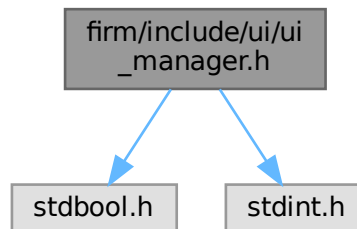
7.35 firm/include/ui/ui_manager.h File Reference

Orchestratore agnostico dell'interfaccia utente.

```
#include <stdbool.h>
```

```
#include <stdint.h>
```

Include dependency graph for ui_manager.h:



This graph shows which files directly or indirectly include this file:



Enumerations

- enum [view_id_t](#) {
[VIEW_ID_HOME](#) = 0 , [VIEW_ID_GPS](#) , [VIEW_ID_METEO](#) , [VIEW_ID_SETTINGS](#) ,
[VIEW_ID_INFO](#) , [VIEW_ID_COUNT](#) }

Identificativi univoci per le schermate (view) del sistema.

Functions

- void [ui_manager_init](#) (void)
Inizializza il sistema UI.
- void [ui_manager_update](#) (const void *system_data)
Task di aggiornamento periodico dell'interfaccia.
- void [ui_manager_dispatch_input](#) (uint8_t input_id)
Dispaccia un evento di input alla schermata attiva.
- void [ui_manager_navigate_to](#) (view_id_t new_view_id)
Cambia la schermata attiva (Navigazione).
- [view_id_t ui_manager_get_current_view](#) (void)
Restituisce la schermata correntemente visualizzata.

7.35.1 Detailed Description

Orchestratore agnostico dell'interfaccia utente.

Gestisce la macchina a stati (FSM) delle View, il routing degli eventi di input e l'aggiornamento grafico del display.

Definition in file [ui_manager.h](#).

7.36 ui_manager.h

[Go to the documentation of this file.](#)

```

00001
00010 #ifndef UI_MANAGER_H
00011 #define UI_MANAGER_H
00012
00013 #include <stdbool.h>
00014 #include <stdint.h>
00015
00019 typedef enum {
00020     VIEW_ID_HOME = 0,
00021     VIEW_ID_GPS,
00022     VIEW_ID_METEO,
00023     VIEW_ID_SETTINGS,
00024     VIEW_ID_INFO,
00025     VIEW_ID_COUNT
00026 } view_id_t;
00027
00033 void ui_manager_init(void);
00034
00041 void ui_manager_update(const void* system_data);
00042
00049 void ui_manager_dispatch_input(uint8_t input_id);
00050
00055 void ui_manager_navigate_to(view_id_t new_view_id);
00056
00061 view_id_t ui_manager_get_current_view(void);
00062
// fine del gruppo UIManager 00064
00065 #endif /* UI_MANAGER_H */

```

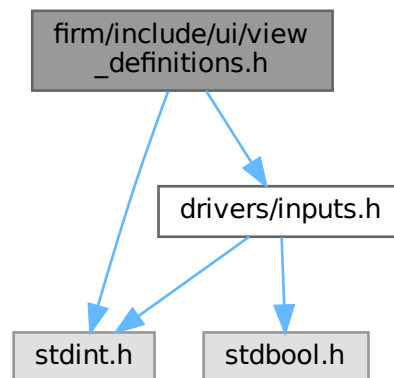
7.37 firm/include/ui/view_definitions.h File Reference

Definizione del contratto per le View dell'interfaccia utente.

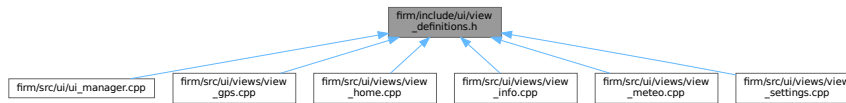
```
#include <stdint.h>
```

```
#include "drivers/inputs.h"
```

Include dependency graph for view_definitions.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [view_interface_t](#)

Struttura che definisce l'interfaccia funzionale di una schermata (View).

Variables

Istanze delle Viste di Sistema

Esportazione delle interfacce implementate nei relativi moduli.

- const [view_interface_t](#) view_home
- const [view_interface_t](#) view_gps
- const [view_interface_t](#) view_meteo
- const [view_interface_t](#) view_settings
- const [view_interface_t](#) view_info

7.37.1 Detailed Description

Definizione del contratto per le View dell'interfaccia utente.

Definisce l'interfaccia ([view_interface_t](#)) che ogni schermata deve implementare per essere gestita dall'orchestratore UI.

Definition in file [view_definitions.h](#).

7.38 view_definitions.h

[Go to the documentation of this file.](#)

```

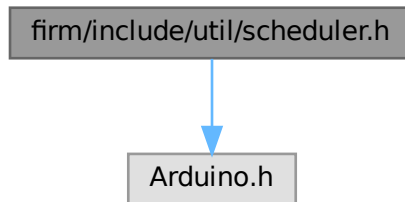
00001
00010 #ifndef VIEW_DEFINITIONS_H
00011 #define VIEW_DEFINITIONS_H
00012
00013 #include <stdint.h>
00014 #include "drivers/inputs.h"
00015
00021 typedef struct {
00023     void (*on_enter)(void);
00024
00028     void (*on_update)(const void* data);
00029
00034     void (*on_input)(button_t btn, button_state_t state);
00035
00037     void (*on_exit)(void);
00038 } view_interface_t;
00039
00045 extern const view_interface_t view_home;
00046 extern const view_interface_t view_gps;
00047 extern const view_interface_t view_meteo;
00048 extern const view_interface_t view_settings;
00049 extern const view_interface_t view_info;
// fine del gruppo ViewDefinitions 00053
00054 #endif /* VIEW_DEFINITIONS_H */
  
```

7.39 firm/include/util/scheduler.h File Reference

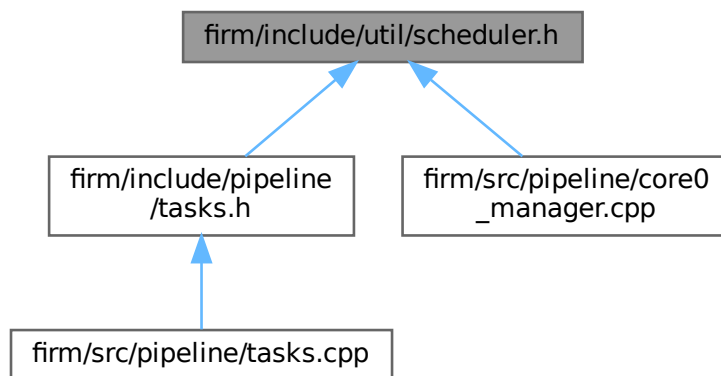
Motore di esecuzione multitasking cooperativo per task periodici.


```
#include <Arduino.h>
```

Include dependency graph for scheduler.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct [Task](#)

Struttura dati che definisce un task periodico.

Typedefs

- typedef void(* [TaskCallback](#)) (void)

Firma della funzione callback per un task.

Functions

- void [run_task](#) ([Task](#) *t)

Verifica ed esegue il task se l'intervallo è scaduto.

7.39.1 Detailed Description

Motore di esecuzione multitasking cooperativo per task periodici.

Gestisce l'invocazione di funzioni (callback) basata su intervalli temporali (polling) senza l'uso di un RTOS complesso.

Definition in file [scheduler.h](#).

7.40 scheduler.h

[Go to the documentation of this file.](#)

```
00001
00010 #ifndef SCHEDULER_H
00011 #define SCHEDULER_H
00012
00013 #include <Arduino.h>
00014
00018 typedef void (*TaskCallback)(void);
00019
00023 typedef struct {
00024     TaskCallback callback;
00025     uint32_t interval;
00026     uint32_t last_run;
00027 } Task;
00028
00035 inline void run_task(Task* t) {
00036     uint32_t now = millis();
00037     if (now - t->last_run >= t->interval) {
00038         t->callback();
00039         t->last_run = now;
00040     }
00041 }
00042
// fine del gruppo Scheduler 00044
00045 #endif // SCHEDULER_H
```

7.41 target.h

```
00001 #pragma once
00002
00003 #ifndef UNIT_TEST
00004 #include <Arduino.h>
00005
00006 #include "pico/util/queue.h"
00007 #else
00008 #include <stdarg.h>
00009 #include <stdbool.h>
00010 #include <stdint.h>
00011 #include <stdio.h>
00012 #include <string.h>
00013
00014 // --- Costanti Arduino ---
00015 #define HIGH 0x1
00016 #define LOW 0x0
00017 #define INPUT 0x0
00018 #define OUTPUT 0x1
00019 #define INPUT_PULLUP 0x2
00020
00021 // La macro F() su PC non serve (non c'è memoria Flash separata)
00022 #define F(s) s
00023
00024 typedef uint8_t byte;
00025 typedef bool boolean;
00026
00027 #ifdef __cplusplus
00028 extern "C" {
00029 #endif
00030 inline uint32_t millis() {
00031     return 0;
00032 }
00033 inline void delay(uint32_t ms) {}
00034 inline void pinMode(int pin, int mode) {}
00035 inline int digitalRead(int pin) {
00036     return LOW;
00037 }
00038 inline void digitalWrite(int pin, int val) {}
00039
00040 typedef struct {
00041     int dummy;
00042 } queue_t;
00043 inline void queue_init(queue_t* q, size_t size, unsigned int depth) {}
00044 inline bool queue_try_add(queue_t* q, const void* data) {
00045     return true;
00046 }
```

```

00047 inline bool queue_try_remove(queue_t* q, void* data) {
00048     return false;
00049 }
00050 inline bool queue_is_empty(queue_t* q) {
00051     return true;
00052 }
00053 #ifdef __cplusplus
00054 }
00055 #endif
00056
00057 #ifdef __cplusplus
00058 // --- Mock Serial ---
00059 class MockSerial {
00060 public:
00061     void begin(long s) {}
00062     bool available() {
00063         return false;
00064     }
00065     int read() {
00066         return -1;
00067     }
00068     void print(const char* s) {
00069         printf("%s", s);
00070     }
00071     void print(int n) {
00072         printf("%d", n);
00073     } // Gestisce print di numeri
00074     void println(const char* s) {
00075         printf("%s\n", s);
00076     }
00077     void printf(const char* format, ...) {
00078         va_list args;
00079         va_start(args, format);
00080         vprintf(format, args);
00081         va_end(args);
00082     }
00083 };
00084 extern MockSerial Serial;
00085 extern MockSerial Serial1;
00086
00087 // --- Mock Adafruit GFX & SSD1306 ---
00088 #define WHITE 1
00089 #define BLACK 0
00090 #define SSD1306_WHITE 1
00091 #define SSD1306_BLACK 0
00092
00093 class Adafruit_GFX {
00094 public:
00095     virtual void drawPixel(int16_t x, int16_t y, uint16_t color) = 0;
00096     void setCursor(int16_t x, int16_t y) {}
00097     void setTextColor(uint16_t c) {}
00098     void setTextColor(uint16_t c, uint16_t bg) {} // Supporto testo invertito
00099     void setTextSize(uint8_t s) {}
00100
00101     // Gestione print e printf
00102     void print(const char* s) {}
00103     void print(int n) {}
00104     void println(const char* s) {}
00105     void printf(const char* format, ...) {
00106         va_list args;
00107         va_start(args, format);
00108         vprintf(format, args);
00109         va_end(args);
00110     }
00111
00112     // Funzioni di disegno mancanti
00113     void drawFastHLine(int16_t x, int16_t y, int16_t w, uint16_t color) {}
00114     void drawFastVLine(int16_t x, int16_t y, int16_t h, uint16_t color) {}
00115 };
00116
00117 class Adafruit_SSD1306 : public Adafruit_GFX {
00118 public:
00119     Adafruit_SSD1306(int w, int h, void* wire = NULL, int rst = -1) {}
00120     bool begin(uint8_t vcc, uint8_t addr) {
00121         return true;
00122     }
00123     void display() {}
00124     void clearDisplay() {}
00125     void drawPixel(int16_t x, int16_t y, uint16_t color) override {}
00126 };
00127 #endif
00128 #endif

```

7.42 config.cpp

```

00001 #include "core/config.h"
00002
00003 SystemConfig global_cfg;
00004
00005 void config_init(void) {
00006     global_cfg.oled_brightness = 150;
00007     global_cfg.leds_enabled = true;
00008     global_cfg.gps_refresh_rate = 1;
00009
00010     // Se compiliamo con BUZZER_SILENT=1, il buzzer parte spento
00011     #ifdef DEFAULT_BUZZER_OFF
00012         global_cfg.buzzer_enabled = false;
00013     #else
00014         global_cfg.buzzer_enabled = true;
00015     #endif
00016 }
```

7.43 power_manager.cpp

7.44 storage.cpp

```

00001 #include "core/storage.h"
00002 #include "core/system_manager.h" // Per segnalare errori al bus
00003 #include <LittleFS.h>
00004 #include <Arduino.h>
00005
00006 static bool driveConnected = false;
00007 static const char* LOG_FILENAME = "/telemetry_log.csv";
00008 static const char* CONFIG_FILENAME = "/config.bin";
00009
00010 bool storage_init(void) {
00011     //LittleFS.remove("/gps.log");
00012     if (!LittleFS.begin()) {
00013         if (!LittleFS.format()) {
00014             sys_manager_report_error(ERR_CAT_STORAGE, ERR_STG_FS_CORRUPT, true);
00015             return false;
00016         }
00017         if (!LittleFS.begin()) return false;
00018     }
00019
00020     if (!LittleFS.exists(LOG_FILENAME)) {
00021         File f = LittleFS.open(LOG_FILENAME, "w");
00022         if (f) {
00023             f.println("timestamp,lat,lon,speed,temp,satellites");
00024             f.close();
00025         } else {
00026             sys_manager_report_error(ERR_CAT_STORAGE, ERR_STG_WRITE_FAIL, false);
00027         }
00028     }
00029     return true;
00030 }
00031
00032 // Logging Asincrono: ora segnala fallimenti al sistema
00033 void storage_log_append(const char* entry) {
00034     if (driveConnected) return;
00035
00036     File f = LittleFS.open(LOG_FILENAME, "a");
00037     if (f) {
00038         f.println(entry);
00039         f.close();
00040     } else {
00041         // Segnala l'errore al bus di sistema, che lo passerà alla UI
00042         sys_manager_report_error(ERR_CAT_STORAGE, ERR_STG_WRITE_FAIL, false);
00043     }
00044 }
00045
00046 bool storage_save_params(const void* data, size_t size) {
00047     if (driveConnected) return false;
00048
00049     File f = LittleFS.open(CONFIG_FILENAME, "w");
00050     if (f) {
00051         size_t written = f.write((const uint8_t*)data, size);
00052         f.close();
00053         if (written != size) {
00054             sys_manager_report_error(ERR_CAT_STORAGE, ERR_STG_WRITE_FAIL, true);
00055             return false;
00056         }
00057         return true;
00058     }
00059     sys_manager_report_error(ERR_CAT_STORAGE, ERR_STG_WRITE_FAIL, true);
00060     return false;
}
```

```

00061 }
00062
00063 bool storage_load_params(void* data, size_t size) {
00064     File f = LittleFS.open(CONFIG_FILENAME, "r");
00065     if (f) {
00066         size_t read = f.read((uint8_t*)data, size);
00067         f.close();
00068         return (read == size);
00069     }
00070     return false;
00071 }
00072
00073 bool storage_is_busy_by_usb(void) {
00074     return driveConnected;
00075 }
00076
00077 void storage_log_gps_fix(float lat, float lon) {
00078     // FILTRO DI SICUREZZA:
00079     // Se le coordinate sono (0,0), ignoriamo la scrittura per non sporcare la cache.
00080     // Usiamo una piccola tolleranza per sicurezza (es. < 0.0001)
00081     if (lat > -0.0001 && lat < 0.0001 && lon > -0.0001 && lon < 0.0001) {
00082         return;
00083     }
00084
00085     File f = LittleFS.open("/gps.log", "a");
00086     if (!f) {
00087         Serial.println("[STORAGE] Errore apertura gps.log");
00088         return;
00089     }
00090
00091     f.printf("%lu,%.6f,%.6f\n", millis() / 1000, lat, lon);
00092     f.close();
00093
00094     Serial.printf("[STORAGE] Posizione salvata: %.6f, %.6f\n", lat, lon);
00095 }
00096
00097 bool storage_read_last_gps(float* lat, float* lon) {
00098     File f = LittleFS.open("/gps.log", "r");
00099     if (!f) {
00100         Serial.println("[STORAGE DEBUG] ERRORE: Impossibile aprire /gps.log");
00101         return false;
00102     }
00103
00104     String last_line = "";
00105     int line_count = 0;
00106
00107     Serial.println("[STORAGE DEBUG] Inizio scansione file...");
00108
00109     while (f.available()) {
00110         String line = f.readStringUntil('\n');
00111         if (line.length() > 5) {
00112             last_line = line;
00113             line_count++;
00114         }
00115     }
00116     f.close();
00117
00118     Serial.printf("[STORAGE DEBUG] Scansione terminata. Righe trovate: %d\n", line_count);
00119     Serial.printf("[STORAGE DEBUG] Ultima riga letta: '%s'\n", last_line.c_str());
00120
00121     if (last_line == "") {
00122         Serial.println("[STORAGE DEBUG] ERRORE: File vuoto o corrotto");
00123         return false;
00124     }
00125
00126     int first_comma = last_line.indexOf(',');
00127     int second_comma = last_line.indexOf(',', first_comma + 1);
00128
00129     if (first_comma == -1 || second_comma == -1) {
00130         Serial.println("[STORAGE DEBUG] ERRORE: Formato riga non valido");
00131         return false;
00132     }
00133
00134     *lat = last_line.substring(first_comma + 1, second_comma).toFloat();
00135     *lon = last_line.substring(second_comma + 1).toFloat();
00136
00137     Serial.printf("[STORAGE DEBUG] Parse riuscito: LAT %.6f, LON %.6f\n", *lat, *lon);
00138     return true;
00139 }
00140
00141 void debug_dump_gps_log() {
00142     Serial.println("\n--- TENTATIVO LETTURA FILE GPS.LOG ---");
00143
00144     if (!LittleFS.exists("/gps.log")) {
00145         Serial.println("DEBUG: Errore: Il file /gps.log non esiste!");
00146         return;
00147     }

```

```

00148     }
00149
00150     File f = LittleFS.open("/gps.log", "r");
00151     if (!f) {
00152         Serial.println("DEBUG: Errore: Impossibile aprire il file!");
00153         return;
00154     }
00155
00156     Serial.println("--- CONTENUTO FILE ---");
00157     while (f.available()) {
00158         Serial.write(f.read());
00159     }
00160     Serial.println("\n--- FINE DUMP ---");
00161     f.close();
00162 }
00163
00164 bool storage_format_all(void) {
00165     LittleFS.end();
00166     if (LittleFS.format()) {
00167         return LittleFS.begin(); // Rinizializza
00168     }
00169     return false;
00170 }
00171
00172 static const char* WEATHER_FILENAME = "/weather.bin";
00173
00174 bool storage_save_weather(const WeatherDataPacket* w) {
00175     if (!driveConnected) return false;
00176     if (!w || !w->valid) return false;
00177
00178     File f = LittleFS.open(WEATHER_FILENAME, "w");
00179     if (f) {
00180         size_t written = f.write((const uint8_t*)w, sizeof(WeatherDataPacket));
00181         f.close();
00182         if (written != sizeof(WeatherDataPacket)) {
00183             sys_manager_report_error(ERR_CAT_STORAGE, ERR_STG_WRITE_FAIL, false);
00184             return false;
00185         }
00186         Serial.println("[STORAGE] Dati meteo salvati in Flash (LittleFS).");
00187         return true;
00188     }
00189     sys_manager_report_error(ERR_CAT_STORAGE, ERR_STG_WRITE_FAIL, false);
00190     return false;
00191 }
00192
00193 bool storage_load_weather(WeatherDataPacket* w) {
00194     if (!w) return false;
00195
00196     if (!LittleFS.exists(WEATHER_FILENAME)) {
00197         Serial.println("[STORAGE] Nessun dump meteo presente in memoria.");
00198         w->valid = false;
00199         return false;
00200     }
00201
00202     File f = LittleFS.open(WEATHER_FILENAME, "r");
00203     if (f) {
00204         size_t read_bytes = f.read((uint8_t*)w, sizeof(WeatherDataPacket));
00205         f.close();
00206         if (read_bytes == sizeof(WeatherDataPacket)) {
00207             w->valid = true;
00208             Serial.printf("[STORAGE] Cache meteo ripristinata per la città: %s\n", w->city);
00209             return true;
00210         }
00211     }
00212     w->valid = false;
00213     return false;
00214 }

```

7.45 system_manager.cpp

```

00001 #include "core/system_manager.h"
00002 #include "core/messages.h"
00003 #include <Arduino.h>
00004 #include <pico/sync.h>
00005 #include "core/storage.h"
00006
00007 static SystemDataPacket shared_data;
00008 static spin_lock_t* data_lock = spin_lock_init(spin_lock_claim_unused(true));
00009 static volatile bool _core1_ready = false;
00010
00011 void sys_manager_init() {
00012     uint32_t save = spin_lock_blocking(data_lock);
00013     memset(&shared_data, 0, sizeof(SystemDataPacket));
00014     spin_unlock(data_lock, save);
00015     _core1_ready = false;

```

```

00016
00017 // === MODIFICA 1: CARICAMENTO DELLA CACHE ALL'AVVIO ===
00018 WeatherDataPacket cached_weather;
00019 if (storage_load_weather(&cached_weather)) {
00020     uint32_t save_lock = spin_lock_blocking(data_lock);
00021     shared_data.weather = cached_weather;
00022     spin_unlock(data_lock, save_lock);
00023 }
00024 }
00025
00026 bool sys_manager_send_data(SystemDataPacket* packet) {
00027     uint32_t save = spin_lock_blocking(data_lock);
00028     shared_data = *packet;
00029     spin_unlock(data_lock, save);
00030     return true;
00031 }
00032
00033 bool sys_manager_receive_data(SystemDataPacket* buffer) {
00034     uint32_t save = spin_lock_blocking(data_lock);
00035     *buffer = shared_data;
00036     spin_unlock(data_lock, save);
00037     return true;
00038 }
00039
00040 void sys_manager_report_error(error_category_t cat, error_code_t code, bool critical) {
00041     uint32_t save = spin_lock_blocking(data_lock);
00042
00043     // Se l'errore è già attivo e identico, non fare nulla.
00044     // Questo evita di forzare un aggiornamento continuo dell'UI.
00045     if (shared_data.flags.error_active && shared_data.last_error.code == code) {
00046         spin_unlock(data_lock, save);
00047         return;
00048     }
00049
00050     shared_data.last_error = {cat, code, millis(), critical};
00051     shared_data.flags.error_active = 1;
00052     spin_unlock(data_lock, save);
00053 }
00054
00055 void sys_manager_clear_error() {
00056     uint32_t save = spin_lock_blocking(data_lock);
00057     shared_data.flags.error_active = 0;
00058
00059     // Eseguì il casting esplicito per soddisfare il controllo di tipo del compilatore
00060     shared_data.last_error.code = (error_code_t)0;
00061     shared_data.last_error.category = (error_category_t)0;
00062
00063     spin_unlock(data_lock, save);
00064 }
00065
00066 bool sys_is_corel_ready() { return _corel_ready; }
00067 void sys_set_corel_ready(bool ready) { _corel_ready = ready; }
00068
00069 void sys_manager_handle_serial(void) {
00070     if (Serial.available() > 0) {
00071         String cmd = Serial.readStringUntil('\n');
00072         cmd.trim();
00073
00074         // Comando per recuperare la posizione GPS
00075         if (cmd == "GET_GPS") {
00076             SystemDataPacket frame;
00077             sys_manager_receive_data(&frame);
00078
00079             // LOGICA RESILIENTE: controlla se il fix è attivo, altrimenti usa la cache
00080             if (frame.gps_status) {
00081                 // GPS LIVE: abbiamo segnale attuale
00082                 Serial.printf("GPS_DATA:LIVE,%.6f,%.6f\n", frame.latitude, frame.longitude);
00083             } else {
00084                 // GPS NON DISPONIBILE: leggiamo dalla Flash
00085                 float lat, lon;
00086                 if (storage_read_last_gps(&lat, &lon)) {
00087                     Serial.printf("GPS_DATA:CACHE,%.6f,%.6f\n", lat, lon);
00088                 } else {
00089                     // Fallback totale in caso di errore di lettura
00090                     Serial.printf("GPS_DATA:CACHE,0.000000,0.000000\n");
00091                 }
00092             }
00093         }
00094
00095         // 2. Comando per ricezione dati meteo (WXC, City, Temp, Wind, Hum, Code)
00096         else if (cmd.startsWith("WXC,")) {
00097             int p1 = cmd.indexOf(',');
00098             int p2 = cmd.indexOf(',', p1 + 1);
00099             int p3 = cmd.indexOf(',', p2 + 1);
00100             int p4 = cmd.indexOf(',', p3 + 1);
00101             int p5 = cmd.indexOf(',', p4 + 1);
00102

```

```

00103         if (p5 != -1) {
00104             WeatherDataPacket w;
00105
00106             cmd.substring(p1 + 1, p2).toCharArray(w.city, 32);
00107             w.temp_ext = cmd.substring(p2 + 1, p3).toFloat();
00108             w.wind_speed = cmd.substring(p3 + 1, p4).toFloat();
00109             w.humidity = cmd.substring(p4 + 1, p5).toInt();
00110             w.weather_code = cmd.substring(p5 + 1).toInt();
00111             w.valid = true;
00112
00113             sys_manager_update_weather(w);
00114
00115             // === MODIFICA 2: PERSISTENZA IMMEDIATA IN LITTLEFS ===
00116             storage_save_weather(&w);
00117
00118             Serial.println("OK_WXC");
00119         }
00120     }
00121
00122     // 3. Comando per stato diagnostico
00123     else if (cmd == "GET_STATUS") {
00124         SystemDataPacket frame;
00125         sys_manager_receive_data(&frame);
00126
00127         Serial.println("--- STATO SISTEMA ---");
00128         Serial.printf("Error Active: %d\n", frame.flags.error_active);
00129
00130         if (frame.flags.error_active) {
00131             Serial.printf("Ultimo Errore: Cat 0x%02X, Code 0x%02X\n",
00132                 frame.last_error.category, frame.last_error.code);
00133         } else {
00134             Serial.println("System Healthy.");
00135         }
00136     }
00137 }
00138 }
00139
00140 // Funzione chirurgica per il Meteo (Usata solo dal Core 0/Seriale)
00141 bool sys_manager_update_weather(const WeatherDataPacket& weather) {
00142     uint32_t save = spin_lock_blocking(data_lock);
00143     shared_data.weather = weather;
00144     shared_data.weather.valid = true;
00145     spin_unlock(data_lock, save);
00146     return true;
00147 }
00148
00149 bool sys_manager_update_gps(float lat, float lon, bool fix, uint8_t sats) {
00150     uint32_t save = spin_lock_blocking(data_lock);
00151     shared_data.latitude = lat;
00152     shared_data.longitude = lon;
00153     shared_data.gps_status = fix; // Aggiorna lo stato fix
00154     shared_data.satellites = sats; // Aggiorna il numero satelliti
00155     spin_unlock(data_lock, save);
00156     return true;
00157 }

```

7.46 telemetry.cpp

```

00001 #include "core/telemetry.h"
00002 #include "core/system_manager.h"
00003 #include "drivers/config_pins.h"
00004 #include "drivers/peripherals.h"
00005 #include <Arduino.h>
00006 #include <minmea.h>
00007 #include <pico/sync.h>
00008 #include "core/storage.h"
00009
00010 static SystemDataPacket telemetry_data;
00011 static spin_lock_t* telemetry_lock = NULL; // Inizializzato in telemetry_init
00012
00013 static char nmea_buffer[128];
00014 static uint8_t nmea_idx = 0;
00015 static uint32_t last_fix_timestamp = 0;
00016 static bool error_reported = false;
00017
00018 static void gps_hw_enable() {
00019     pinMode(SAM_EN_PIN, OUTPUT);
00020     digitalWrite(SAM_EN_PIN, LOW);
00021
00022     pinMode(SAM_RST_PIN, OUTPUT);
00023     digitalWrite(SAM_RST_PIN, LOW);
00024     delay(100);
00025     digitalWrite(SAM_RST_PIN, HIGH);
00026
00027     delay(1000);

```



```

00028 }
00029
00030 void telemetry_init() {
00031     // Inizializzazione sicura del lock
00032     if (telemetry_lock == NULL) {
00033         telemetry_lock = spin_lock_instance(spin_lock_claim_unused(true));
00034     }
00035
00036     memset(&telemetry_data, 0, sizeof(SystemDataPacket));
00037     gps_hw_enable();
00038
00039     Serial2.setTX(GPS_TX_PIN);
00040     Serial2.setRX(GPS_RX_PIN);
00041     Serial2.begin(GPSBAUD);
00042 }
00043
00044 // static void process_nmea(const char* line) {
00045 //     struct minmea_sentence_gga gga;
00046 //     if (minmea_parse_gga(&gga, line)) {
00047 //         uint32_t save = spin_lock_blocking(telemetry_lock);
00048 //         telemetry_data.latitude = minmea_tocoord(&gga.latitude);
00049 //         telemetry_data.longitude = minmea_tocoord(&gga.longitude);
00050 //         telemetry_data.satellites = gga.satellites_tracked;
00051 //         telemetry_data.gps_status = (gga.fix_quality > 0);
00052 //         if (telemetry_data.gps_status) last_fix_timestamp = millis();
00053 //         spin_unlock(telemetry_lock, save);
00054 //         return;
00055 //     }
00056 //
00057 //     struct minmea_sentence_rmc rmc;
00058 //     if (minmea_parse_rmc(&rmc, line)) {
00059 //         if (rmc.valid) {
00060 //             uint32_t save = spin_lock_blocking(telemetry_lock);
00061 //             telemetry_data.gps_status = true;
00062 //             telemetry_data.latitude = minmea_tocoord(&rmc.latitude);
00063 //             telemetry_data.longitude = minmea_tocoord(&rmc.longitude);
00064 //             last_fix_timestamp = millis();
00065 //             spin_unlock(telemetry_lock, save);
00066 //         }
00067 //     }
00068 // }
00069
00070 // static void process_nmea(const char* line) {
00071 //     struct minmea_sentence_gga gga;
00072 //     if (minmea_parse_gga(&gga, line)) {
00073 //         // CONTROLLO RIGIDO: Se fix_quality è 0, il fix NON ESISTE.
00074 //         if (gga.fix_quality > 0) {
00075 //             uint32_t save = spin_lock_blocking(telemetry_lock);
00076 //             telemetry_data.latitude = minmea_tocoord(&gga.latitude);
00077 //             telemetry_data.longitude = minmea_tocoord(&gga.longitude);
00078 //             telemetry_data.satellites = gga.satellites_tracked;
00079 //             telemetry_data.gps_status = true;
00080 //             last_fix_timestamp = millis();
00081 //             telemetry_data.flags.error_active = 0;
00082 //             error_reported = false;
00083 //             spin_unlock(telemetry_lock, save);
00084 //         } else {
00085 //             // Se arrivano dati GGA ma il fix è 0, azzera lo stato!
00086 //             uint32_t save = spin_lock_blocking(telemetry_lock);
00087 //             telemetry_data.gps_status = false;
00088 //             telemetry_data.satellites = 0;
00089 //             spin_unlock(telemetry_lock, save);
00090 //         }
00091 //         return;
00092 //     }
00093 // }
00094
00095
00096 static void process_nmea(const char* line) {
00097     bool has_real_fix = false;
00098     float lat = 0.0f, lon = 0.0f;
00099     int sats = 0;
00100
00101     struct minmea_sentence_gga gga;
00102     if (minmea_parse_gga(&gga, line)) {
00103         // Il fix_quality > 0 è l'unico vero segnale di posizione valida
00104         if (gga.fix_quality > 0) {
00105             lat = minmea_tocoord(&gga.latitude);
00106             lon = minmea_tocoord(&gga.longitude);
00107             sats = gga.satellites_tracked;
00108             has_real_fix = true;
00109         }
00110     }
00111
00112     // Se abbiamo un fix, aggiorniamo e logghiamo
00113     if (has_real_fix) {
00114         uint32_t save = spin_lock_blocking(telemetry_lock);

```

```

00115     telemetry_data.latitude = lat;
00116     telemetry_data.longitude = lon;
00117     telemetry_data.gps_status = true;
00118     telemetry_data.satellites = sats;
00119     last_fix_timestamp = millis();
00120     spin_unlock(&telemetry_lock, save);
00121
00122     // LOGGING SOLO QUI: Con fix garantito e coordinate non nulle
00123     check_and_log_gps(lat, lon);
00124 }
00125 }
00126 // void telemetry_update() {
00127 //     // Aggiornamento lento per non bloccare il loop
00128 //     static uint32_t last_slow_update = 0;
00129 //     if (millis() - last_slow_update > 500) {
00130 //         telemetry_data.temp_c = peripherals_get_temperature();
00131 //         telemetry_data.uptime_s = millis() / 1000;
00132 //         last_slow_update = millis();
00133 //     }
00134 //
00135 //     if (!telemetry_is_healthy()) {
00136 //         if (!error_reported) {
00137 //             sys_manager_report_error(ERR_CAT_SENSORS, ERR_SENS_GPS_NO_DATA, false);
00138 //             error_reported = true;
00139 //         }
00140 //     }
00141 //
00142 //     // Processing rate-limited: max 64 byte a ciclo per evitare corruzione
00143 //     int read_count = 0;
00144 //     while (Serial2.available() && read_count < 64) {
00145 //         char c = (char)Serial2.read();
00146 //         read_count++;
00147 //
00148 //         if (c == '$') nmea_idx = 0;
00149 //         if (nmea_idx < sizeof(nmea_buffer) - 1) nmea_buffer[nmea_idx++] = c;
00150 //
00151 //         if (c == '\n') {
00152 //             nmea_buffer[nmea_idx] = '\0';
00153 //             if (nmea_idx > 5) {
00154 //                 Serial.print("RAW: "); Serial.print(nmea_buffer); // DEBUG
00155 //                 process_nmea(nmea_buffer);
00156 //             }
00157 //             nmea_idx = 0;
00158 //         }
00159 //     }
00160 // }
00161
00162 void telemetry_update() {
00163     // 1. LOGICA DI TIMEOUT: Se è passato troppo tempo dall'ultimo fix, azzerare lo stato.
00164     // Questo garantisce che la UI e i LED tornino allo stato "Searching" correttamente.
00165     uint32_t save = spin_lock_blocking(&telemetry_lock);
00166     if (telemetry_data.gps_status && (millis() - last_fix_timestamp > 3000)) {
00167         telemetry_data.gps_status = false;
00168         telemetry_data.satellites = 0;
00169         // Opzionale: potresti voler azzerare anche lat/lon qui se non vuoi mostrare l'ultima
        posizione nota
00170     }
00171     spin_unlock(&telemetry_lock, save);
00172
00173     // 2. Aggiornamento lento (Temp/Uptime) - Eseguito solo ogni 500ms
00174     static uint32_t last_slow_update = 0;
00175     if (millis() - last_slow_update > 500) {
00176         telemetry_data.temp_c = peripherals_get_temperature();
00177         telemetry_data.uptime_s = millis() / 1000;
00178         last_slow_update = millis();
00179     }
00180
00181     // 3. Gestione errori (Controllo salute)
00182     if (!telemetry_is_healthy()) {
00183         if (!error_reported) {
00184             sys_manager_report_error(ERR_CAT_SENSORS, ERR_SENS_GPS_NO_DATA, false);
00185             error_reported = true;
00186         }
00187     }
00188
00189     // 4. Processing rate-limited: max 64 byte a ciclo per evitare saturazione del bus
00190     int read_count = 0;
00191     while (Serial2.available() && read_count < 64) {
00192         char c = (char)Serial2.read();
00193         read_count++;
00194
00195         if (c == '$') nmea_idx = 0;
00196         if (nmea_idx < sizeof(nmea_buffer) - 1) nmea_buffer[nmea_idx++] = c;
00197
00198         if (c == '\n') {
00199             nmea_buffer[nmea_idx] = '\0';
00200             if (nmea_idx > 5) {

```

```

00201         Serial.print("RAW: "); Serial.print(nmea_buffer); // DEBUG - opzionale
00202         process_nmea(nmea_buffer);
00203     }
00204     nmea_idx = 0;
00205 }
00206 }
00207 }
00208
00209 bool telemetry_get_frame(SystemDataPacket* dest) {
00210     if (!dest || !telemetry_lock) return false;
00211     uint32_t save = spin_lock_blocking(telemetry_lock);
00212     *dest = telemetry_data;
00213     spin_unlock(telemetry_lock, save);
00214     return true;
00215 }
00216
00217 // bool telemetry_is_healthy() {
00218 //     if (last_fix_timestamp == 0) return (millis() < 60000);
00219 //     return (millis() - last_fix_timestamp) < 30000;
00220 // }
00221 //
00222 bool telemetry_is_healthy() {
00223     // Se non abbiamo ancora un fix, il sistema è "sano" (sta cercando)
00224     if (last_fix_timestamp == 0) return true;
00225     // Se sono passati più di 5 secondi dall'ultimo fix, segnala errore
00226     return (millis() - last_fix_timestamp) < 5000;
00227 }
00228
00229 static void check_and_log_gps(float lat, float lon) {
00230     static float last_lat = 0, last_lon = 0;
00231     // Calcolo semplice distanza (Euclideo semplificato per brevità)
00232     if (abs(lat - last_lat) > 0.005 || abs(lon - last_lon) > 0.005) {
00233         storage_log_gps_fix(lat, lon);
00234         last_lat = lat;
00235         last_lon = lon;
00236     }
00237 }

```

7.47 firm/src/drivers/display_ssd1306.cpp File Reference

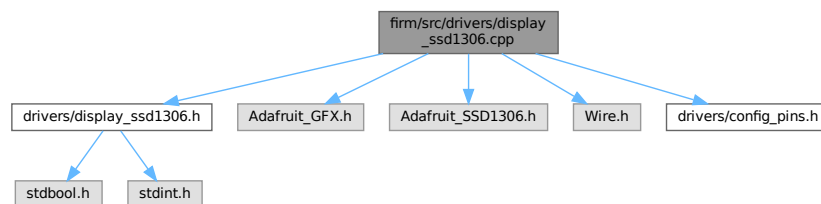
Implementazione driver OLED via I2C0 nativo per PCB PPSE Lab.

```

#include "drivers/display_ssd1306.h"
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Wire.h>
#include "drivers/config_pins.h"

```

Include dependency graph for display_ssd1306.cpp:



Functions

- bool [display_hw_init](#) ()
Inizializza l'hardware I2C e il controller OLED.
- bool [display_is_alive](#) ()
Verifica se il display è fisicamente connesso al bus I2C.
- void [display_clear](#) ()
Pulisce il buffer interno (RAM) del display.
- void [display_show](#) ()

Invia il buffer corrente al display fisico (Refresh).

- void `display_set_power` (bool on)

Gestione energetica del pannello.

- void `display_set_brightness` (uint8_t level)

Regola il contrasto (luminosità) dei pixel OLED.

- void `display_draw_pixel` (int16_t x, int16_t y, uint16_t color)

Funzione primitiva di disegno agnostica.

- void * `get_display_driver` ()

Fornisce l'accesso al driver sottostante (uso avanzato).

7.47.1 Detailed Description

Implementazione driver OLED via I2C0 nativo per PCB PPSE Lab.

Definition in file `display_ssd1306.cpp`.

7.48 display_ssd1306.cpp

[Go to the documentation of this file.](#)

```
00001
00006 #include "drivers/display_ssd1306.h"
00007 #include <Adafruit_GFX.h>
00008 #include <Adafruit_SSD1306.h>
00009 #include <Wire.h>
00010 #include "drivers/config_pins.h"
00011
00012 // Istanziamento statico del driver
00013 static Adafruit_SSD1306 display_instance(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
00014 static bool is_initialized = false;
00015
00016 bool display_hw_init() {
00017     // Configurazione hardware pin I2C0
00018     Wire.setSDA(I2C0_SDA_PIN);
00019     Wire.setSCL(I2C0_SCL_PIN);
00020     Wire.begin();
00021     Wire.setClock(400000); // Passiamo a 400kHz per una UI più fluida (OLED SSD1306 supporta fino a
400kHz)
00022
00023     // Verifica presenza hardware
00024     Wire.beginTransmission(SCREEN_ADDRESS);
00025     if (Wire.endTransmission() != 0) {
00026         Serial.println("[OLED] CRITICO: Hardware non rilevato!");
00027         return false;
00028     }
00029
00030     if (!display_instance.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADDRESS)) {
00031         return false;
00032     }
00033
00034     display_instance.clearDisplay();
00035     display_instance.display();
00036     is_initialized = true;
00037     return true;
00038 }
00039
00040 bool display_is_alive() {
00041     if (!is_initialized) return false;
00042     Wire.beginTransmission(SCREEN_ADDRESS);
00043     return (Wire.endTransmission() == 0);
00044 }
00045
00046 void display_clear() {
00047     if (is_initialized) display_instance.clearDisplay();
00048 }
00049
00050 void display_show() {
00051     if (is_initialized) display_instance.display();
00052 }
00053
00054 void display_set_power(bool on) {
00055     if (!is_initialized) return;
00056     display_instance.ssd1306_command(on ? SSD1306_DISPLAYON : SSD1306_DISPLAYOFF);
00057 }
00058
00059 void display_set_brightness(uint8_t level) {
00060     if (!is_initialized) return;
00061     display_instance.ssd1306_command(SSD1306_SETCONTRAST);
```

```

00062     display_instance.ssd1306_command(level);
00063 }
00064
00065 void display_draw_pixel(int16_t x, int16_t y, uint16_t color) {
00066     if (is_initialized) display_instance.drawPixel(x, y, color);
00067 }
00068
00069 void* get_display_driver() {
00070     // Restituiamo il puntatore all'istanza.
00071     // Il cast avverrà nel file che chiama questa funzione.
00072     return (void*)&display_instance;
00073 }

```

7.49 inputs.cpp

```

00001 #include "drivers/inputs.h"
00002 #include <Arduino.h>
00003 #include "drivers/config_pins.h"
00004
00005 static volatile ButtonId pending_button = BTN_NONE;
00006 static volatile uint32_t last_interrupt_time = 0;
00007 const uint32_t DEBOUNCE_DELAY_MS = 200;
00008
00009 // Rimosso IRAM_ATTR per compatibilità immediata
00010 void handle_button_interrupt() {
00011     uint32_t current_time = millis();
00012     if (current_time - last_interrupt_time > DEBOUNCE_DELAY_MS) {
00013         if (digitalRead(BTN_UP_PIN) == LOW) pending_button = BTN_UP;
00014         else if (digitalRead(BTN_DOWN_PIN) == LOW) pending_button = BTN_DOWN;
00015         else if (digitalRead(BTN_OK_PIN) == LOW) pending_button = BTN_OK;
00016         else if (digitalRead(BTN_BACK_PIN) == LOW) pending_button = BTN_BACK;
00017
00018         last_interrupt_time = current_time;
00019     }
00020 }
00021
00022 void inputs_init() {
00023     pinMode(BTN_UP_PIN, INPUT_PULLUP);
00024     pinMode(BTN_DOWN_PIN, INPUT_PULLUP);
00025     pinMode(BTN_OK_PIN, INPUT_PULLUP);
00026     pinMode(BTN_BACK_PIN, INPUT_PULLUP);
00027
00028     attachInterrupt(digitalPinToInterrupt(BTN_UP_PIN), handle_button_interrupt, FALLING);
00029     attachInterrupt(digitalPinToInterrupt(BTN_DOWN_PIN), handle_button_interrupt, FALLING);
00030     attachInterrupt(digitalPinToInterrupt(BTN_OK_PIN), handle_button_interrupt, FALLING);
00031     attachInterrupt(digitalPinToInterrupt(BTN_BACK_PIN), handle_button_interrupt, FALLING);
00032 }
00033
00034 // void inputs_update() {
00035 //     // Ora gestito totalmente dagli interrupt
00036 // }
00037
00038 ButtonId inputs_get_last_press() {
00039     ButtonId event = pending_button;
00040     pending_button = BTN_NONE;
00041     return event;
00042 }
00043
00044 bool inputs_is_pressed(ButtonId btn) {
00045     switch (btn) {
00046         case BTN_UP: return (digitalRead(BTN_UP_PIN) == LOW);
00047         case BTN_DOWN: return (digitalRead(BTN_DOWN_PIN) == LOW);
00048         case BTN_OK: return (digitalRead(BTN_OK_PIN) == LOW);
00049         case BTN_BACK: return (digitalRead(BTN_BACK_PIN) == LOW);
00050         default: return false;
00051     }
00052 }

```

7.50 firm/src/drivers/peripherals.cpp File Reference

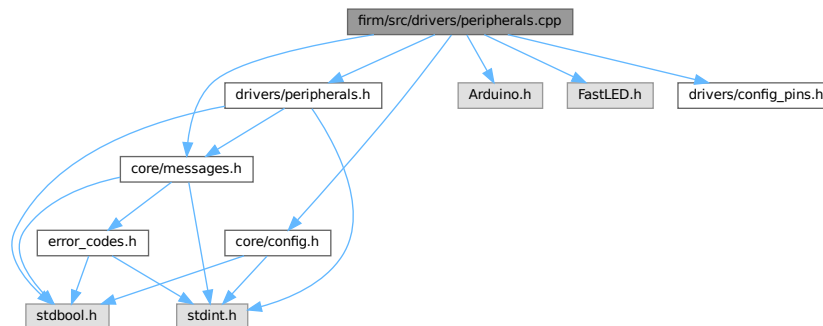
Implementazione driver con logica di feedback a stati e boot sequence.

```

#include "drivers/peripherals.h"
#include <Arduino.h>
#include <FastLED.h>
#include "core/config.h"
#include "core/messages.h"
#include "drivers/config_pins.h"

```

Include dependency graph for peripherals.cpp:



Functions

- void [peripherals_init](#) ()
- void [peripherals_init_leds](#) ()
- void [peripherals_set_leds](#) (bool enabled)
- void [peripherals_set_buzzer](#) (bool enabled)
- void [peripherals_play_click](#) ()
Esegue il tipico suono di "click" per conferma UI.
- void [peripherals_update_led_fx](#) ([LedAnimState](#) state, float val)
Gestore avanzato effetti LED.
- void [peripherals_auto_feedback](#) (const [SystemDataPacket](#) *data)
Esegue un feedback automatico basato sui dati di telemetria.
- void [peripherals_beep](#) (uint32_t freq, uint32_t duration)
Esegue un breve suono (beep).
- void [peripherals_trigger_feedback](#) ([FeedbackType](#) type)
Attiva un feedback sonoro/visivo basato su un evento.
- void [peripherals_set_led](#) (uint8_t r, uint8_t g, uint8_t b)
Imposta colore RGB statico per i LED.
- float [peripherals_get_temperature](#) ()

Variables

- [SystemConfig global_cfg](#)

7.50.1 Detailed Description

Implementazione driver con logica di feedback a stati e boot sequence.
Definition in file [peripherals.cpp](#).

7.50.2 Variable Documentation

7.50.2.1 global_cfg

[SystemConfig](#) global_cfg [extern]

Definition at line 3 of file [config.cpp](#).

7.51 peripherals.cpp

[Go to the documentation of this file.](#)

```

00001
00006 #include "drivers/peripherals.h"
00007 #include <Arduino.h>
00008 #include <FastLED.h>
00009 #include "core/config.h"
00010 #include "core/messages.h"
00011 #include "drivers/config_pins.h"
00012
00013 static CRGB leds[NUM_LEDS];
00014 extern SystemConfig global_cfg;
00015
00016 // --- Stato interno ---
00017 static bool boot_finished = false;
00018 static uint32_t boot_start_time = 0;
00019
00020 void peripherals_init() {
00021     pinMode(LED_ALIVE_PIN, OUTPUT);
00022     for (int i = 0; i < 3; i++) {
00023         digitalWrite(LED_ALIVE_PIN, HIGH);
00024         delay(100);
00025         digitalWrite(LED_ALIVE_PIN, LOW);
00026         delay(100);
00027     }
00028     digitalWrite(LED_ALIVE_PIN, HIGH);
00029     pinMode(BUCK_5V_EN_PIN, OUTPUT);
00030     digitalWrite(BUCK_5V_EN_PIN, HIGH);
00031     pinMode(TEMP_SENS_PIN, INPUT);
00032     boot_start_time = millis();
00033 }
00034
00035 void peripherals_init_leds() {
00036     FastLED.addLeds<WS2812B, RGB_LED_PIN, GRB>(leds, NUM_LEDS);
00037     FastLED.setBrightness(40);
00038     fill_solid(leds, NUM_LEDS, CRGB::Black);
00039     FastLED.show();
00040 }
00041
00042 /* --- Controllo Fisico Periferiche --- */
00043 void peripherals_set_leds(bool enabled) {
00044     if (!enabled) {
00045         fill_solid(leds, NUM_LEDS, CRGB::Black);
00046         FastLED.show();
00047     }
00048     global_cfg.leds_enabled = enabled;
00049 }
00050
00051 // DEFINIZIONE UNICA
00052 void peripherals_set_buzzer(bool enabled) {
00053     global_cfg.buzzer_enabled = enabled;
00054     if (!enabled) {
00055         noTone(BUZZER_PIN);
00056     }
00057 }
00058
00059 void peripherals_play_click() {
00060     if (global_cfg.buzzer_enabled) {
00061         peripherals_beep(2000, 30);
00062     }
00063 }
00064
00065 void peripherals_update_led_fx(LedAnimState state, float val) {
00066     bool needs_show = true;
00067     static LedAnimState last_state = (LedAnimState)-1;
00068     if (last_state != state) {
00069         fill_solid(leds, NUM_LEDS, CRGB::Black);
00070         last_state = state;
00071     }
00072
00073     switch (state) {
00074         case LED_ANIM_BOOT: {
00075             static uint8_t pos = 0;
00076             fadeToBlackBy(leds, NUM_LEDS, 20);
00077             leds[pos % NUM_LEDS] = CRGB(150, 150, 150);
00078             pos++;
00079             break;
00080         }
00081         case LED_ANIM_SEARCHING: {
00082             if ((millis() / 500) % 2 == 0) fill_solid(leds, NUM_LEDS, CRGB::Red);
00083             else fill_solid(leds, NUM_LEDS, CRGB::Black);
00084             break;
00085         }
00086         case LED_ANIM_FIX_OK: {
00087             float brightness_factor = constrain((float)val * 2.0f, 0.2f, 1.0f);

```

```

00091         uint8_t brightness = (uint8_t)(brightness_factor * 255);
00092         fill_solid(leds, NUM_LEDS, CRGB(0, brightness, 0));
00093         break;
00094     }
00095     case LED_ANIM_SIGNAL_LOW: {
00096         fill_solid(leds, NUM_LEDS, CRGB(150, 50, 0));
00097         break;
00098     }
00099     default:
00100         needs_show = false;
00101         break;
00102 }
00103 if (needs_show) FastLED.show();
00104 }
00105
00106 // void peripherals_auto_feedback(const SystemDataPacket* data) {
00107 //     if (!global_cfg.leds_enabled) {
00108 //         fill_solid(leds, NUM_LEDS, CRGB::Black);
00109 //         FastLED.show();
00110 //         return;
00111 //     }
00112 //
00113 //     if (!boot_finished) {
00114 //         peripherals_update_led_fx(LED_ANIM_BOOT, 0.0f);
00115 //         if (millis() - boot_start_time > 3000) boot_finished = true;
00116 //     } else {
00117 //         // Corretto confronto usando l'enum per robustezza
00118 //         if (data->gps_status < GPS_STATUS_FIX_2D) {
00119 //             peripherals_update_led_fx(LED_ANIM_SEARCHING, 0.0f);
00120 //         } else if (data->battery_v < 3.4f) {
00121 //             peripherals_update_led_fx(LED_ANIM_SIGNAL_LOW, 0.0f);
00122 //         } else {
00123 //             float sat_quality = constrain((float)data->satellites / 12.0f, 0.1f, 1.0f);
00124 //             peripherals_update_led_fx(LED_ANIM_FIX_OK, sat_quality);
00125 //         }
00126 //     }
00127 // }
00128
00129
00130 void peripherals_auto_feedback(const SystemDataPacket* data) {
00131     if (!global_cfg.leds_enabled) {
00132         fill_solid(leds, NUM_LEDS, CRGB::Black);
00133         FastLED.show();
00134         return;
00135     }
00136
00137     if (!boot_finished) {
00138         peripherals_update_led_fx(LED_ANIM_BOOT, 0.0f);
00139         if (millis() - boot_start_time > 3000) boot_finished = true;
00140     } else {
00141         // MODIFICA: Invece di usare il confronto <, usiamo un confronto esplicito
00142         // Assumendo che 0 sia NO_FIX e 1 sia FIX_2D/3D (come da log)
00143         if (data->gps_status == 0) {
00144             peripherals_update_led_fx(LED_ANIM_SEARCHING, 0.0f);
00145         } else if (data->battery_v < 3.4f) {
00146             peripherals_update_led_fx(LED_ANIM_SIGNAL_LOW, 0.0f);
00147         } else {
00148             // Qui entriamo se status >= 1
00149             float sat_quality = constrain((float)data->satellites / 8.0f, 0.1f, 1.0f);
00150             peripherals_update_led_fx(LED_ANIM_FIX_OK, sat_quality);
00151         }
00152     }
00153 }
00154
00155
00156 /* --- Feedback Acustico --- */
00157 // void peripherals_beep(uint32_t freq, uint32_t duration) {
00158 //     if(global_cfg.buzzer_enabled) {
00159 //         tone(BUZZER_PIN, freq, duration);
00160 //     }
00161 // }
00162
00163 // void peripherals_beep(uint32_t freq, uint32_t duration) {
00164 //     #if defined(BUZZER_SILENT) && BUZZER_SILENT == 1
00165 //         return; // Ignora i beep se siamo in modalità debug silenziosa
00166 //     #endif
00167 //
00168 //     if(global_cfg.buzzer_enabled) {
00169 //         tone(BUZZER_PIN, freq, duration);
00170 //     }
00171 // }
00172
00173 /* --- Feedback Acustico --- */
00174 void peripherals_beep(uint32_t freq, uint32_t duration) {
00175     // 1. Controllo a COMPILAZIONE (più veloce e sicuro)
00176     #ifndef DEFAULT_BUZZER_OFF
00177         return;
00178     #endif

```



```

00178     #endif
00179
00180     // 2. Controllo a RUNTIME (basato sulle impostazioni dell'utente nel menu)
00181     if (global_cfg.buzzer_enabled) {
00182         tone(BUZZER_PIN, freq, duration);
00183     }
00184 }
00185
00186 // void peripherals_trigger_feedback(FeedbackType type) {
00187 //     switch (type) {
00188 //         case FEEDBACK_SUCCESS: peripherals_beep(1500, 80); break;
00189 //         case FEEDBACK_ERROR:   peripherals_beep(400, 300); break;
00190 //         default: break;
00191 //     }
00192 // }
00193
00194 void peripherals_trigger_feedback(FeedbackType type) {
00195     // Questa funzione ora userà automaticamente la logica sicura definita sopra
00196     switch (type) {
00197         case FEEDBACK_SUCCESS: peripherals_beep(1500, 80); break;
00198         case FEEDBACK_ERROR:   peripherals_beep(400, 300); break;
00199         default: break;
00200     }
00201 }
00202
00203 void peripherals_set_led(uint8_t r, uint8_t g, uint8_t b) {
00204     fill_solid(leds, NUM_LEDS, CRGB(r, g, b));
00205     FastLED.show();
00206 }
00207
00208 float peripherals_get_temperature() {
00209     uint32_t raw_sum = 0;
00210     for (uint8_t i = 0; i < 10; i++) {
00211         raw_sum += analogRead(TEMP_SENS_PIN);
00212         delayMicroseconds(50);
00213     }
00214     return ((raw_sum / 10.0f) * (3.3f / 4095.0f)) - 0.5f * 100.0f;
00215 }

```

7.52 radio_uart.cpp

7.53 sensors_i2c.cpp

```

00001 #include "drivers/sensors_i2c.h"
00002 #include "drivers/config_pins.h"
00003 #include <Arduino.h>
00004 #include <Wire.h>
00005
00006 bool sensors_i2c_init() {
00007     // I2C è già stato inizializzato nel driver display,
00008     // qui possiamo fare solo un check di presenza sensori
00009     return sensors_i2c_is_healthy();
00010 }
00011
00012 bool sensors_i2c_is_healthy() {
00013     // Esempio: scansione bus
00014     Wire.beginTransmission(0x19); // Esempio indirizzo LIS3DH
00015     return (Wire.endTransmission() == 0);
00016 }
00017
00018 // --- FUNZIONI DI LETTURA REALE (AGGIUNTE) ---
00019
00020 float sensors_read_temperature_c() {
00021     // Legge il sensore ADC0 (GPIO 26)
00022     int raw = analogRead(TEMP_SENS_PIN);
00023     // Formula di conversione (da calibrare in base al tuo sensore termico)
00024     return (raw * 3.3f / 4096.0f) * 100.0f;
00025 }
00026
00027 float sensors_read_battery_v() {
00028     int raw = analogRead(ANALOG_BATT_PIN);
00029     // Supponendo che il calcolo ti dia, per esempio, 2.5 come fattore:
00030     const float FATTORE_CALIBRAZIONE = 2.5f;
00031     return (raw * 3.3f / 4095.0f) * FATTORE_CALIBRAZIONE;
00032 }
00033
00034 bool sensors_accel_get_data(Vector3D_t* data) {
00035     // Qui andrebbe l'implementazione I2C per leggere il LIS3DH
00036     return false;
00037 }

```

7.54 main.cpp

```

00001 #include <Arduino.h>
00002 #include "core/messages.h"
00003 #include "core/config.h"
00004 #include "pipeline/core0_manager.h"
00005 #include "pipeline/core1_manager.h"
00006 #include "pico/multicore.h"
00007
00008 // La variabile globale è definita in config.cpp, qui è solo un riferimento
00009 extern SystemConfig global_cfg;
00010
00011 void setup1() {
00012     core1_setup();
00013 }
00014
00015 void loop1() {
00016     core1_loop();
00017 }
00018
00019 void setup() {
00020     // disable the watchdog timer
00021     watchdog_hw->ctrl &= ~(1<<30);
00022     core0_setup();
00023 }
00024
00025 void loop() {
00026     core0_loop();
00027 }

```

7.55 core0_manager.cpp

```

00001 #include "pipeline/core0_manager.h"
00002
00003 #include <Arduino.h>
00004
00005 #include "core/system_manager.h"
00006 #include "core/telemetry.h"
00007 #include "drivers/config_pins.h"
00008 #include "drivers/display_ssd1306.h"
00009 #include "drivers/inputs.h"
00010 #include "drivers/peripherals.h"
00011 #include "ui/boot_anim.h"
00012 #include "ui/ui_manager.h"
00013 #include "util/scheduler.h"
00014 #include "core/config.h"
00015 #include "core/storage.h"
00016 #include <LittleFS.h>
00017
00018 // DICHIARA L'ESISTENZA DELLA VARIABILE GLOBALE
00019 extern SystemConfig global_cfg;
00020
00021 // Variabili per tracciamento tempo
00022 static uint32_t last_uptime_tick = 0;
00023
00024 // Task schedulati
00025 static Task task_serial = {[]() { sys_manager_handle_serial(); }, 10, 0};
00026
00027 static Task task_input = {[]() {
00028     ButtonId b = inputs_get_last_press();
00029     if (b != BTN_NONE)
00030         ui_manager_dispatch_input(b);
00031 },
00032     20, 0};
00033
00034 // Task Uptime: aggiorna il contatore nel pacchetto condiviso
00035 static Task task_uptime = {[]() {
00036     if (millis() - last_uptime_tick >= 1000) {
00037         last_uptime_tick = millis();
00038         SystemDataPacket frame;
00039         if (sys_manager_receive_data(&frame)) {
00040             frame.uptime_s++;
00041             sys_manager_send_data(&frame);
00042         }
00043     }
00044 },
00045     100, 0};
00046
00047 // Task UI separato per fluidità: aggiornamento a ~30Hz
00048 static Task task_ui = {[]() {
00049     SystemDataPacket frame;
00050     if (sys_manager_receive_data(&frame)) {
00051         ui_manager_update(&frame);
00052     }

```

```

00053         },
00054         33, 0);
00055
00056 static Task task_leds = {[]() {
00057     SystemDataPacket frame;
00058     if (sys_manager_receive_data(&frame)) {
00059         peripherals_auto_feedback(&frame);
00060     }
00061 },
00062     100, 0);
00063
00064 // static Task task_health = {
00065 //     []() {
00066 //         SystemDataPacket frame;
00067 //         sys_manager_receive_data(&frame);
00068 //         if (!frame.flags.error_active) {
00069 //             Serial.printf("[HEARTBEAT] System Healthy | Uptime: %lus\n", frame.uptime_s);
00070 //         } else {
00071 //             Serial.printf("[HEARTBEAT] WARNING: Error Active! | Code: 0x%02X\n",
00072 //                 frame.last_error.code);
00073 //         }
00074 //     },
00075 //     10000, 0);
00076
00077 static Task task_health = {
00078     []() {
00079         #if defined(DEBUG) && DEBUG == 1
00080             SystemDataPacket frame;
00081             sys_manager_receive_data(&frame);
00082             if (!frame.flags.error_active) {
00083                 Serial.printf("[HEARTBEAT] System Healthy | Uptime: %lus\n", frame.uptime_s);
00084             } else {
00085                 Serial.printf("[HEARTBEAT] WARNING: Error Active! | Code: 0x%02X\n",
00086                     frame.last_error.code);
00087             }
00088         #endif
00089     },
00090     10000, 0);
00091
00092 void boot_led_update_wrapper() {
00093     static uint32_t last_led_update = 0;
00094     if (millis() - last_led_update > 50) {
00095         peripherals_update_led_fx(LED_ANIM_BOOT, 0.0f);
00096         last_led_update = millis();
00097     }
00098 }
00099
00100 void core0_setup() {
00101     Serial.begin(115200);
00102     uint32_t start_time = millis();
00103     while (!Serial && (millis() - start_time < 1000)) delay(100);
00104
00105     // Messaggi di boot condizionali (visibili solo in Debug)
00106     #if defined(DEBUG) && DEBUG == 1
00107         Serial.println("\n--- PPSE FIRMWARE SYSTEM BOOTING ---");
00108     #endif
00109
00110     // 1. Inizializzazione configurazione (Applica i default/flag di compilazione)
00111     config_init();
00112
00113     // if (LittleFS.begin()) {
00114     //     Serial.println("\n[SYSTEM] Filesystem montato.");
00115     //     debug_dump_gps_log();
00116     // } else {
00117     //     Serial.println("\n[SYSTEM] ERRORE: LittleFS non montato.");
00118     // }
00119     delay(3000); // Aspetta 3 secondi, non 1
00120
00121     Serial.println("\n\n--- INIZIO TEST DI BOOT FORZATO ---");
00122
00123     // Prova ad inizializzare SOLO la Flash
00124     if (LittleFS.begin()) {
00125         Serial.println("LittleFS OK.");
00126     } else {
00127         Serial.println("LittleFS FALLITO.");
00128         // Se fallisce qui, il file system è corrotto a livello hardware
00129     }
00130
00131     // Ora prova a scrivere un file di test per vedere se la flash risponde
00132     File f = LittleFS.open("/test.txt", "w");
00133     if (f) {
00134         f.println("Test scrittura riuscito");
00135         f.close();
00136         Serial.println("Scrittura test.txt OK.");
00137     } else {
00138         Serial.println("Scrittura test.txt FALLITA.");
00139     }

```

```

00140
00141 // 2. Inizializzazione sottosistemi core
00142 sys_manager_init();
00143 telemetry_init();
00144
00145
00146 // 3. Setup hardware di base
00147 pinMode(LED_ALIVE_PIN, OUTPUT);
00148 digitalWrite(LED_ALIVE_PIN, HIGH);
00149
00150 pinMode(BUCK_5V_EN_PIN, OUTPUT);
00151 digitalWrite(BUCK_5V_EN_PIN, HIGH);
00152
00153 // 4. Inizializzazione periferiche (ora leggono correttamente global_cfg)
00154 peripherals_init();
00155 peripherals_init_leds();
00156 inputs_init();
00157
00158 // 5. Setup UI
00159 if (!display_hw_init()) {
00160     sys_manager_report_error(ERR_CAT_HW, ERR_HW_DISPLAY_LOST, true);
00161 } else {
00162     run_hacker_boot_sequence(boot_led_update_wrapper);
00163 }
00164
00165 ui_manager_init();
00166
00167 // Messaggio finale condizionale
00168 #if defined(DEBUG) && DEBUG == 1
00169     Serial.println("--- PPSE FIRMWARE SYSTEM READY ---");
00170 #endif
00171 }
00172
00173 void core0_loop() {
00174     telemetry_update();
00175
00176     run_task(&task_serial);
00177     run_task(&task_input);
00178     run_task(&task_uptime); // Gestione uptime integrata
00179     run_task(&task_ui);
00180     run_task(&task_leds);
00181     run_task(&task_health);
00182
00183     digitalWrite(LED_ALIVE_PIN, (millis() / 500) % 2);
00184 }

```

7.56 core1_manager.cpp

```

00001 #include "pipeline/core1_manager.h"
00002 #include <Arduino.h>
00003 #include "drivers/sensors_i2c.h"
00004 #include "core/messages.h"
00005 #include "core/telemetry.h"
00006 #include "core/system_manager.h" // Necessario per sys_manager_send_data
00007 #include "drivers/config_pins.h"
00008
00009 void core1_setup() {
00010     Serial.println("CORE 1: Avvio alimentazione GPS...");
00011
00012     pinMode(SAM_EN_PIN, OUTPUT);
00013     digitalWrite(SAM_EN_PIN, LOW); // PMOS: LOW = ON
00014
00015     sensors_i2c_init();
00016     telemetry_init();
00017
00018     Serial.println("CORE 1: Init completato.");
00019 }
00020
00021 // void core1_loop() {
00022 //     telemetry_update();
00023 //
00024 //     SystemDataPacket frame;
00025 //     telemetry_get_frame(&frame);
00026 //
00027 //     // CAMBIAMENTO: Non inviare più tutto il pacchetto frame.
00028 //     // Invia solo le coordinate GPS usando la funzione chirurgica.
00029 //     sys_manager_update_gps(frame.latitude, frame.longitude, frame.gps_status, frame.satellites);
00030 //     // Debug
00031 //     static uint32_t last_log = 0;
00032 //     if (millis() - last_log > 5000) {
00033 //         Serial.println("CORE 1: GPS Update Sent...");
00034 //         last_log = millis();
00035 //     }
00036 //
00037 //     delay(50);

```

```

00038 // }
00039 //
00040 void core1_loop() {
00041     telemetry_update();
00042
00043     SystemDataPacket frame;
00044     if (telemetry_get_frame(&frame)) {
00045         // Invio chirurgico: aggiorna solo se c'è stato un nuovo dato valido
00046         // o se il timeout ha forzato lo stato a false
00047         Serial.printf("DEBUG: Status %d, Sats %d\n", frame.gps_status, frame.satellites);
00048         sys_manager_update_gps(frame.latitude, frame.longitude, frame.gps_status, frame.satellites);
00049         //sys_manager_update_gps(frame.latitude, frame.longitude, frame.gps_status, frame.satellites);
00050     }
00051
00052     delay(100); // Aumentato leggermente per ridurre il carico sul bus I2C/SPI
00053 }

```

7.57 tasks.cpp

```

00001 #include "pipeline/tasks.h"
00002 #include "core/system_manager.h"
00003 #include "ui/ui_manager.h"
00004 #include "drivers/peripherals.h"
00005 #include "drivers/inputs.h"
00006
00007 Task task_serial = { []() { sys_manager_handle_serial(); }, 10, 0 };
00008
00009 Task task_input = { []() {
00010     ButtonId b = inputs_get_last_press();
00011     if (b != BTN_NONE) ui_manager_dispatch_input(b);
00012 }, 20, 0 };
00013
00014 Task task_ui = { []() {
00015     SystemDataPacket frame;
00016     if (sys_manager_receive_data(&frame)) {
00017         ui_manager_update(&frame);
00018         peripherals_auto_feedback(&frame);
00019     }
00020 }, 33, 0 };
00021
00022 Task task_health = { []() {
00023     SystemDataPacket frame;
00024     sys_manager_receive_data(&frame);
00025     if (!frame.flags.error_active) {
00026         Serial.printf("[HEARTBEAT] System Healthy | Uptime: %lus\n", millis() / 1000);
00027     } else {
00028         Serial.printf("[HEARTBEAT] WARNING: Error Active! | Code: 0x%02X\n", frame.last_error.code);
00029     }
00030 }, 10000, 0 };

```

7.58 firm/src/ui/boot_anim.cpp File Reference

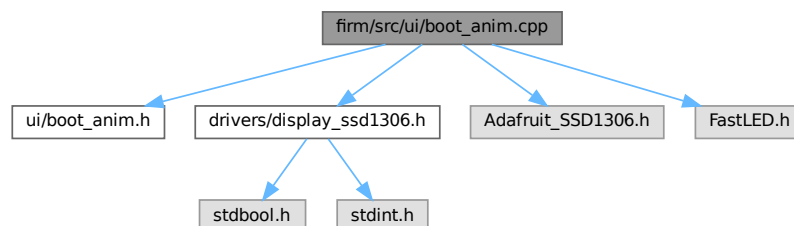
Animazione di boot hacker-style con rendering asincrono dei LED.

```

#include "ui/boot_anim.h"
#include "drivers/display_ssd1306.h"
#include <Adafruit_SSD1306.h>
#include <FastLED.h>

```

Include dependency graph for boot_anim.cpp:



Functions

- void [run_hacker_boot_sequence](#) (void(*update_callback)())

7.58.1 Detailed Description

Animazione di boot hacker-style con rendering asincrono dei LED.

Definition in file [boot_anim.cpp](#).

7.58.2 Function Documentation

7.58.2.1 run_hacker_boot_sequence()

```
void run_hacker_boot_sequence (
    void(*)() update_callback )
```

Definition at line 11 of file [boot_anim.cpp](#).

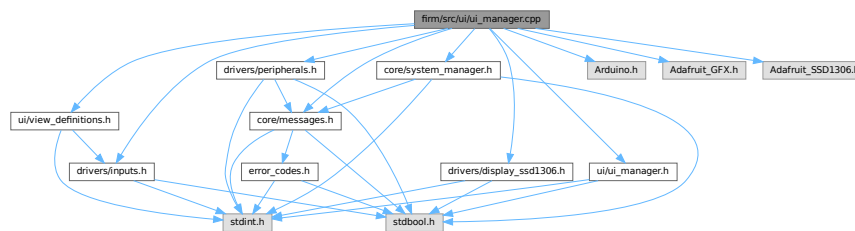
7.59 boot_anim.cpp

[Go to the documentation of this file.](#)

```
00001
00006 #include "ui/boot_anim.h"
00007 #include "drivers/display_ssd1306.h"
00008 #include <Adafruit_SSD1306.h>
00009 #include <FastLED.h> // Incluso per FastLED.delay()
00010
00011 void run_hacker_boot_sequence(void (*update_callback)()) {
00012     Adafruit_SSD1306* canvas = (Adafruit_SSD1306*)get_display_driver();
00013     if (!canvas) return;
00014
00015     const char* modules[] = {"CORE", "RADIO", "GPS", "SENSORS", "UI"};
00016     int num_modules = 5;
00017
00018     for (int i = 0; i <= num_modules; i++) {
00019         canvas->clearDisplay();
00020         canvas->setTextColor(SSD1306_WHITE);
00021
00022         // --- Render UI ---
00023         canvas->setCursor(0, 0);
00024         canvas->print("INIT SYSTEM: ");
00025         canvas->println(i == num_modules ? "READY" : "LOADING");
00026         canvas->drawLine(0, 10, 128, 10, SSD1306_WHITE);
00027
00028         canvas->setCursor(0, 15);
00029         for (int j = 0; j < i; j++) {
00030             canvas->printf("> %s_READY\n", modules[j]);
00031         }
00032
00033         if (i < num_modules) {
00034             canvas->print("> ");
00035             canvas->print(modules[i]);
00036             canvas->print((millis() / 300) % 2 == 0 ? "_" : "");
00037         }
00038
00039         canvas->drawRect(0, 56, 128, 8, SSD1306_WHITE);
00040         canvas->fillRect(2, 58, (i * 124 / num_modules), 4, SSD1306_WHITE);
00041
00042         canvas->display();
00043
00044         // --- Loop di attesa fluido ---
00045         // Invece di un delay bloccante, aggiorniamo i LED continuamente
00046         uint32_t wait_duration = 600 + (rand() % 400);
00047         uint32_t start_wait = millis();
00048
00049         while (millis() - start_wait < wait_duration) {
00050             if (update_callback) {
00051                 update_callback(); // Aggiorna i LED velocemente
00052             }
00053             FastLED.delay(10); // Gestisce il timing FastLED per animazioni fluide
00054         }
00055     }
00056
00057     // Pausa finale di conferma prima di passare alla UI principale
00058     delay(500);
00059 }
```

7.60 firm/src/ui/ui_manager.cpp File Reference

```
#include "ui/ui_manager.h"
#include <Arduino.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include "core/messages.h"
#include "core/system_manager.h"
#include "drivers/display_ssd1306.h"
#include "drivers/inputs.h"
#include "ui/view_definitions.h"
#include "drivers/peripherals.h"
Include dependency graph for ui_manager.cpp:
```



Functions

- void **ui_manager_init** (void)
Inizializza il sistema UI.
- void **ui_manager_update** (const void *system_data)
Task di aggiornamento periodico dell'interfaccia.
- void **ui_manager_dispatch_input** (uint8_t btn_raw)
Dispaccia un evento di input alla schermata attiva.
- void **ui_manager_navigate_to** (view_id_t new_view_id)
Cambia la schermata attiva (Navigazione).
- view_id_t **ui_manager_get_current_view** (void)
Restituisce la schermata correntemente visualizzata.

7.61 ui_manager.cpp

[Go to the documentation of this file.](#)

```
00001
00005 #include "ui/ui_manager.h"
00006 #include <Arduino.h>
00007 #include <Adafruit_GFX.h>
00008 #include <Adafruit_SSD1306.h>
00009 #include "core/messages.h"
00010 #include "core/system_manager.h"
00011 #include "drivers/display_ssd1306.h"
00012 #include "drivers/inputs.h"
00013 #include "ui/view_definitions.h"
00014 #include "drivers/peripherals.h"
00015
00016 static const view_interface_t* current_view = &view_home;
00017 static view_id_t current_view_id = VIEW_ID_HOME;
00018
00019 static bool error_acknowledged = false;
00020 static uint8_t last_error_code = 0;
00021
00022 void ui_manager_init(void) {
00023     if (current_view && current_view->on_enter) {
00024         current_view->on_enter();
00025     }
00026 }
```

```

00027
00028 static void render_error_overlay(const SystemDataPacket* data, Adafruit_SSD1306* canvas) {
00029     canvas->fillRect(0, 0, 128, 64, SSD1306_BLACK);
00030     canvas->setTextColor(SSD1306_WHITE);
00031
00032     canvas->setCursor(0, 0);
00033     canvas->print("CRITICAL ERROR");
00034     canvas->drawLine(0, 10, 128, 10, SSD1306_WHITE);
00035
00036     canvas->setCursor(0, 20);
00037     canvas->printf("CAT: 0x%02X", data->last_error.category);
00038     canvas->setCursor(0, 30);
00039     canvas->printf("CODE: 0x%02X", data->last_error.code);
00040
00041     if (data->last_error.is_critical) {
00042         canvas->setCursor(0, 50);
00043         canvas->print("SYSTEM HALTED");
00044     } else {
00045         canvas->setCursor(0, 50);
00046         canvas->print("Premere tasto...");
00047     }
00048 }
00049
00050 void ui_manager_update(const void* system_data) {
00051     const SystemDataPacket* data = (const SystemDataPacket*)system_data;
00052     Adafruit_SSD1306* canvas = (Adafruit_SSD1306*)get_display_driver();
00053     if (!canvas) return;
00054
00055     if (data->last_error.code != last_error_code) {
00056         error_acknowledged = false;
00057         last_error_code = data->last_error.code;
00058         // Feedback acustico istantaneo all'errore
00059         #ifndef DEFAULT_BUZZER_OFF
00060             if (last_error_code != 0) peripherals_trigger_feedback(FEEDBACK_ERROR);
00061         #endif
00062     }
00063
00064     canvas->clearDisplay();
00065
00066     if (data->flags.error_active && !error_acknowledged) {
00067         render_error_overlay(data, canvas);
00068     } else {
00069         if (current_view && current_view->on_update) {
00070             current_view->on_update(data);
00071         }
00072     }
00073     canvas->display();
00074 }
00075
00076 void ui_manager_dispatch_input(uint8_t btn_raw) {
00077     button_t btn = (button_t)btn_raw;
00078     if (btn == BTN_NONE || !current_view) return;
00079
00080     if (last_error_code != 0 && !error_acknowledged) {
00081         error_acknowledged = true;
00082         sys_manager_clear_error();
00083         #ifndef DEFAULT_BUZZER_OFF
00084             peripherals_play_click();
00085         #endif
00086         return;
00087     }
00088
00089     // Feedback acustico differenziato
00090     switch (btn) {
00091         case BTN_UP:
00092         case BTN_DOWN:
00093             #ifndef DEFAULT_BUZZER_OFF
00094                 peripherals_beep(2400, 15);
00095             #endif
00096             break;
00097         case BTN_BACK:
00098             #ifndef DEFAULT_BUZZER_OFF
00099                 peripherals_beep(1800, 20);
00100             #endif
00101             break;
00102         case BTN_OK:
00103             break;
00104         default: break;
00105     }
00106
00107     if (current_view->on_input) {
00108         current_view->on_input(btn, BTN_RELEASED);
00109     }
00110 }
00111
00112 void ui_manager_navigate_to(view_id_t new_view_id) {
00113     if (current_view && current_view->on_exit) {

```



```

00114     current_view->on_exit();
00115 }
00116
00117 current_view_id = new_view_id;
00118
00119 // Feedback di navigazione menu
00120 #ifndef DEFAULT_BUZZER_OFF
00121     peripherals_trigger_feedback(FEEDBACK_SUCCESS);
00122 #endif
00123
00124 switch (new_view_id) {
00125     case VIEW_ID_HOME:    current_view = &view_home;    break;
00126     case VIEW_ID_GPS:    current_view = &view_gps;      break;
00127     case VIEW_ID_METEO:  current_view = &view_meteo;    break;
00128     case VIEW_ID_SETTINGS: current_view = &view_settings; break;
00129     case VIEW_ID_INFO:   current_view = &view_info;     break;
00130     default:             current_view = &view_home;     break;
00131 }
00132
00133 if (current_view && current_view->on_enter) {
00134     current_view->on_enter();
00135 }
00136 }
00137
00138 view_id_t ui_manager_get_current_view(void) {
00139     return current_view_id;
00140 }

```

7.62 view_gps.cpp

```

00001 #include <Adafruit_GFX.h>
00002 #include <Adafruit_SSD1306.h>
00003 #include <Arduino.h>
00004
00005 #include "core/messages.h"
00006 #include "drivers/display_ssd1306.h"
00007 #include "drivers/inputs.h"
00008 #include "ui/ui_manager.h"
00009 #include "ui/view_definitions.h"
00010
00011 static void gps_on_input(button_t btn, button_state_t state) {
00012     if (state != BTN_RELEASED) return;
00013
00014     // Gestione ritorno alla HOME
00015     if (btn == BTN_BACK) {
00016         ui_manager_navigate_to(VIEW_ID_HOME);
00017     }
00018 }
00019
00020 static void gps_render(const void* data) {
00021     const SystemDataPacket* pkt = (const SystemDataPacket*)data;
00022     Adafruit_SSD1306* canvas = (Adafruit_SSD1306*)get_display_driver();
00023
00024     if (!canvas) return;
00025
00026     // --- INTESTAZIONE ---
00027     canvas->setCursor(25, 0);
00028     canvas->setTextSize(1);
00029     canvas->println(F("GPS TELEMETRY"));
00030     canvas->drawFastHLine(0, 9, 128, SSD1306_WHITE);
00031
00032     // --- STATO FIX E SATELLITI ---
00033     canvas->setCursor(0, 12);
00034     canvas->printf("FIX: %s", pkt->gps_status ? "YES" : "NO");
00035     canvas->setCursor(75, 12);
00036     canvas->printf("SATS: %02d", pkt->satellites);
00037
00038     // --- COORDINATE ---
00039     canvas->setCursor(0, 24);
00040     canvas->print(F("LAT: "));
00041     canvas->printf("%.6f", pkt->latitude);
00042
00043     canvas->setCursor(0, 34);
00044     canvas->print(F("LON: "));
00045     canvas->printf("%.6f", pkt->longitude);
00046
00047     // --- VELOCITÀ ---
00048     canvas->setCursor(0, 44);
00049     float speed_kmh = pkt->speed_ms * 3.6f;
00050     canvas->printf("SPD: %.1f km/h", speed_kmh);
00051
00052     // --- FOOTER ---
00053     canvas->drawFastHLine(0, 54, 128, SSD1306_WHITE);
00054 }
00055

```

```

00056 const view_interface_t view_gps = {
00057     .on_enter = NULL,
00058     .on_update = gps_render,
00059     .on_input = gps_on_input,
00060     .on_exit = NULL
00061 };

```

7.63 view_home.cpp

```

00001 #include <Adafruit_GFX.h>
00002 #include <Adafruit_SSD1306.h>
00003 #include <Arduino.h>
00004
00005 #include "core/messages.h"
00006 #include "drivers/display_ssd1306.h"
00007 #include "drivers/inputs.h"
00008 #include "ui/ui_manager.h"
00009 #include "ui/view_definitions.h"
00010
00011 // Menu semplificato e unificato
00012 enum { HOME_FLIGHT, HOME_WEATHER, HOME_CONFIG, HOME_COUNT };
00013 static uint8_t menu_cursor = 0;
00014 static const char* labels[] = {"GPS", "METEO", "SETTINGS"};
00015
00016 static void home_input(button_t btn, button_state_t state) {
00017     if (state != BTN_RELEASED) return;
00018
00019     if (btn == BTN_UP) menu_cursor = (menu_cursor == 0) ? HOME_COUNT - 1 : menu_cursor - 1;
00020     if (btn == BTN_DOWN) menu_cursor = (menu_cursor + 1) % HOME_COUNT;
00021
00022     if (btn == BTN_OK) {
00023         // Mappatura pulita: ogni voce porta alla sua destinazione univoca
00024         view_id_t targets[] = {VIEW_ID_GPS, VIEW_ID_METEO, VIEW_ID_SETTINGS};
00025         ui_manager_navigate_to(targets[menu_cursor]);
00026     }
00027 }
00028
00029 static void home_render(const void* data) {
00030     Adafruit_SSD1306* canvas = (Adafruit_SSD1306*)get_display_driver();
00031
00032     canvas->clearDisplay();
00033     canvas->setTextSize(1);
00034     canvas->setTextColor(SSD1306_WHITE);
00035
00036     // Titolo della vista
00037     canvas->setCursor(0, 0);
00038     canvas->print(F("SYSTEM::ROOT"));
00039     canvas->drawFastHLine(0, 8, 128, SSD1306_WHITE);
00040
00041     // Menu Centrale
00042     for (uint8_t i = 0; i < HOME_COUNT; i++) {
00043         uint8_t y = 18 + (i * 12);
00044
00045         if (menu_cursor == i) {
00046             // Riga selezionata (inversa)
00047             canvas->fillRect(0, y - 1, 128, 10, SSD1306_WHITE);
00048             canvas->setTextColor(SSD1306_BLACK);
00049             canvas->setCursor(2, y);
00050             canvas->print(labels[i]);
00051             canvas->setTextColor(SSD1306_WHITE);
00052         } else {
00053             // Riga normale
00054             canvas->setCursor(2, y);
00055             canvas->print(labels[i]);
00056         }
00057     }
00058
00059     canvas->display();
00060 }
00061
00062 const view_interface_t view_home = {
00063     .on_enter = NULL,
00064     .on_update = home_render,
00065     .on_input = home_input,
00066     .on_exit = NULL
00067 };

```

7.64 view_info.cpp

```

00001 #include <Adafruit_GFX.h>
00002 #include <Adafruit_SSD1306.h>
00003 #include <Arduino.h>

```

```

00004
00005 #include "core/messages.h"
00006 #include "drivers/display_ssd1306.h"
00007 #include "drivers/inputs.h"
00008 #include "ui/ui_manager.h"
00009 #include "ui/view_definitions.h"
00010
00011 // Rimuoviamo la define statica manuale.
00012 // FW_VERSION viene passato dal Makefile tramite il flag -DFW_VERSION="\"...\"
00013 #ifndef FW_VERSION
00014 #define FW_VERSION "v0.0.0-unknown"
00015 #endif
00016
00017 static void info_input(button_t btn, button_state_t state) {
00018     if (state == BTN_RELEASED) {
00019         // Navigazione: il tasto BACK riporta alla vista SETTINGS
00020         if (btn == BTN_BACK) {
00021             ui_manager_navigate_to(VIEW_ID_SETTINGS);
00022         }
00023     }
00024 }
00025
00026 static void info_render(const void* data) {
00027     const SystemDataPacket* pkt = (const SystemDataPacket*)data;
00028     Adafruit_SSD1306* canvas = (Adafruit_SSD1306*)get_display_driver();
00029
00030     canvas->clearDisplay();
00031     canvas->setTextSize(1);
00032     canvas->setTextColor(SSD1306_WHITE);
00033
00034     // 0: Header
00035     canvas->setCursor(0, 0);
00036     canvas->print(F("INFO SISTEMA"));
00037     canvas->drawFastHLine(0, 9, 128, SSD1306_WHITE);
00038
00039     // 1: FW Version (Inniettata a compilation time)
00040     canvas->setCursor(0, 16);
00041     canvas->print(F("VERSIONE: "));
00042     canvas->print(FW_VERSION);
00043
00044     // 2: Uptime (Reattivo: il valore viene estratto dal pacchetto pkt)
00045     canvas->setCursor(0, 32);
00046     canvas->print(F("UPTIME: "));
00047     canvas->printf("%02lu:%02lu:%02lu",
00048         (pkt->uptime_s / 3600),
00049         (pkt->uptime_s % 3600) / 60,
00050         (pkt->uptime_s % 60));
00051
00052     // 3: Navigazione (Footer)
00053     canvas->setCursor(0, 56);
00054     canvas->print(F("< SETTINGS"));
00055
00056     canvas->display();
00057 }
00058
00059 const view_interface_t view_info = {
00060     .on_enter = NULL,
00061     .on_update = info_render,
00062     .on_input = info_input,
00063     .on_exit = NULL
00064 };

```

7.65 view_meteo.cpp

```

00001 #include <Adafruit_GFX.h>
00002 #include <Adafruit_SSD1306.h>
00003 #include <Arduino.h>
00004
00005 #include "core/messages.h"
00006 #include "drivers/display_ssd1306.h"
00007 #include "ui/ui_manager.h"
00008 #include "ui/view_definitions.h"
00009
00010 static void meteo_render(const void* data) {
00011     const SystemDataPacket* sys = (const SystemDataPacket*)data;
00012     Adafruit_SSD1306* canvas = (Adafruit_SSD1306*)get_display_driver();
00013     if (!canvas || !sys) return;
00014
00015     // Pulisce il buffer prima di ogni frame per evitare artefatti
00016     canvas->clearDisplay();
00017     canvas->setTextColor(SSD1306_WHITE);
00018
00019     if (sys->weather.valid) {
00020         // --- Header ---
00021         canvas->setTextSize(1);

```

```

00022     canvas->setCursor(0, 0);
00023     canvas->printf("CITY: %s", sys->weather.city);
00024     canvas->drawFastHLine(0, 9, 128, SSD1306_WHITE);
00025
00026     // --- Temperatura (Grande, sinistra) ---
00027     canvas->setTextSize(2);
00028     canvas->setCursor(5, 18);
00029     canvas->printf("%.1f C", sys->weather.temp_ext);
00030
00031     // --- Dati Vento e Umidità (Colonna destra, allineata) ---
00032     canvas->setTextSize(1);
00033     canvas->setCursor(80, 15);
00034     canvas->printf("W:%02.0f", sys->weather.wind_speed); // %02.0f evita che la scritta si sposti
00035     canvas->setCursor(80, 25);
00036     canvas->printf("H:%02d%", sys->weather.humidity); // %02d forza 2 cifre per l'umidità
00037
00038     // --- Descrizione Meteo ---
00039     canvas->setCursor(5, 42);
00040     if (sys->weather.weather_code == 0) canvas->print(F("Clear Sky"));
00041     else if (sys->weather.weather_code <= 3) canvas->print(F("Partly Cloudy"));
00042     else if (sys->weather.weather_code >= 60) canvas->print(F("Rainy"));
00043     else canvas->print(F("Condition check"));
00044 } else {
00045     // --- Stato Errore ---
00046     canvas->setTextSize(1);
00047     canvas->setCursor(0, 0);
00048     canvas->print(F("CITY: UNKNOWN"));
00049     canvas->drawFastHLine(0, 9, 128, SSD1306_WHITE);
00050     canvas->setCursor(20, 25);
00051     canvas->println(F("NO SYNC DATA"));
00052     canvas->setCursor(10, 38);
00053     canvas->println(F("Run make sync-meteo"));
00054 }
00055
00056 // --- Footer fisso ---
00057 canvas->drawFastHLine(0, 54, 128, SSD1306_WHITE);
00058 canvas->setCursor(0, 56);
00059 canvas->print(F("Sync via Serial/PC"));
00060
00061 // IMPORTANTE: invia il buffer al display
00062 canvas->display();
00063 }
00064
00065 static void meteo_on_input(button_t btn, button_state_t state) {
00066     if (state == BTN_RELEASED && btn == BTN_BACK) {
00067         ui_manager_navigate_to(VIEW_ID_HOME);
00068     }
00069 }
00070
00071 const view_interface_t view_meteo = {
00072     .on_enter = NULL,
00073     .on_update = meteo_render,
00074     .on_input = meteo_on_input,
00075     .on_exit = NULL
00076 };

```

7.66 view_settings.cpp

```

00001 #include <Adafruit_GFX.h>
00002 #include <Adafruit_SSD1306.h>
00003 #include <Arduino.h>
00004 #include <LittleFS.h>
00005 #include "core/config.h"
00006 #include "core/storage.h"
00007 #include "ui/ui_manager.h"
00008 #include "ui/view_definitions.h"
00009 #include "drivers/peripherals.h"
00010 #include "drivers/display_ssd1306.h"
00011
00012 enum { SETTING_BRIGHTNESS, SETTING_BUZZER, SETTING_LEDS, SETTING_ERASE, SETTING_INFO, SETTING_COUNT };
00013 static uint8_t cursor = 0;
00014 static bool erase_confirm = false;
00015 static uint32_t confirm_timer = 0; // Timer per nascondere "OK"
00016
00017 static void settings_input(button_t btn, button_state_t state) {
00018     if (state != BTN_RELEASED) return;
00019
00020     switch (btn) {
00021     case BTN_UP:
00022         cursor = (cursor == 0) ? SETTING_COUNT - 1 : cursor - 1;
00023         break;
00024     case BTN_DOWN:
00025         cursor = (cursor + 1) % SETTING_COUNT;
00026         break;
00027     case BTN_BACK:

```

```

00028         ui_manager_navigate_to(VIEW_ID_HOME);
00029         break;
00030     case BTN_OK:
00031         #ifndef DEFAULT_BUZZER_OFF
00032             peripherals_play_click();
00033         #endif
00034
00035         switch(cursor) {
00036             case SETTING_BRIGHTNESS:
00037                 global_cfg.oled_brightness = (global_cfg.oled_brightness >= 250) ? 50 :
00038                 ((global_cfg.oled_brightness / 50) + 1) * 50;
00039                 display_set_brightness(global_cfg.oled_brightness);
00040                 break;
00041             case SETTING_BUZZER:
00042                 global_cfg.buzzer_enabled = !global_cfg.buzzer_enabled;
00043                 peripherals_set_buzzer(global_cfg.buzzer_enabled);
00044                 break;
00045             case SETTING_LEDS:
00046                 global_cfg.leds_enabled = !global_cfg.leds_enabled;
00047                 peripherals_set_leds(global_cfg.leds_enabled);
00048                 break;
00049             case SETTING_ERASE:
00050                 LittleFS.end();
00051                 LittleFS.format();
00052                 LittleFS.begin();
00053                 erase_confirm = true;
00054                 confirm_timer = millis(); // Avvia il timer di 2 secondi
00055                 break;
00056             case SETTING_INFO:
00057                 ui_manager_navigate_to(VIEW_ID_INFO);
00058                 break;
00059         }
00060         break;
00061     default: break;
00062 }
00063
00064 static void settings_render(const void* data) {
00065     Adafruit_SSD1306* canvas = (Adafruit_SSD1306*)get_display_driver();
00066     const char* labels[] = {"Bright", "Buzzer", "LEDs", "Erase Flash", "Info"};
00067
00068     // Reset automatico della scritta OK dopo 2 secondi
00069     if (erase_confirm && (millis() - confirm_timer > 2000)) {
00070         erase_confirm = false;
00071     }
00072
00073     canvas->clearDisplay();
00074     canvas->fillRect(0, 0, 128, 12, SSD1306_WHITE);
00075     canvas->setTextColor(SSD1306_BLACK, SSD1306_WHITE);
00076     canvas->setCursor(35, 2);
00077     canvas->print(F("SETTINGS"));
00078
00079     canvas->setTextColor(SSD1306_WHITE);
00080     for (uint8_t i = 0; i < SETTING_COUNT; i++) {
00081         uint8_t y = 16 + (i * 9);
00082         if (cursor == i) {
00083             canvas->fillRect(0, y - 1, 128, 9, SSD1306_WHITE);
00084             canvas->setTextColor(SSD1306_BLACK, SSD1306_WHITE);
00085         } else {
00086             canvas->setTextColor(SSD1306_WHITE, SSD1306_BLACK);
00087         }
00088
00089         canvas->setCursor(2, y);
00090         canvas->print(labels[i]);
00091
00092         if (cursor == i) {
00093             canvas->setCursor(80, y);
00094             if (i == SETTING_BRIGHTNESS) canvas->printf("%d", global_cfg.oled_brightness);
00095             else if (i == SETTING_BUZZER) canvas->print(global_cfg.buzzer_enabled ? F("ON") :
00096             F("OFF"));
00097             else if (i == SETTING_LEDS) canvas->print(global_cfg.leds_enabled ? F("ON") : F("OFF"));
00098             else if (i == SETTING_ERASE && erase_confirm) canvas->print(F("OK"));
00099         }
00100     }
00101     canvas->display();
00102 }
00103 const view_interface_t view_settings = {
00104     .on_enter = NULL,
00105     .on_update = settings_render,
00106     .on_input = settings_input,
00107     .on_exit = NULL
00108 };

```

